



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Deep Neural Network

AIML Module 1

BITS Pilani

The Lecture Materials gratefully acknowledge the authors who made their course materials freely available online.

Course Logistics

What we Learn.... (Module Structure)



1. Fundamentals of Neural Network
2. Multilayer Perceptron
3. Deep Feedforward Neural Network
4. Improve the DNN performance by Optimization and Regularization
5. Convolutional Neural Networks
6. Sequence Models
7. Attention Mechanism
8. Neural Network Search
9. Time series Modelling and Forecasting
10. Other Learning Techniques

Textbook

- Dive into Deep Learning by Aston Zhang, Zack C. Lipton, Mu Li, Alex J. Smola. https://d2l.ai/chapter_introduction/index.html

Reference book

- Deep Learning by Ian Goodfellow, Yoshua Bengio, Aaron Courville
<https://www.deeplearningbook.org/>

Evaluation Components and Schedule



	Evaluation Component	Mode	Duration	Weightage	Date and Time
EC1	Quiz I*	Online	30 Mins	5%	Refer to LMS
	Quiz II*		30 Mins	5%	
	Assignment 01		4 Weeks	10%	
	Assignment 02		4 Weeks	15%	
EC2	Mid-sem exam (Closed book)		2 Hours	30%	
EC3	Comprehensive exam (Open book)		2.5 Hours	40%	

* Best of Two

- L1 Introduction to PyTorch
- L2 Deep Neural Network with Back-propagation and optimization
- L3 CNN
- L4 RNN
- L5 LSTM
- L6 Auto-encoders

- Refer Taxila for the following
 - Handout
 - Schedule for Webinar
 - Schedule of Quiz, and Assignments.
 - Evaluation scheme
 - Session Slide Deck
 - Demo Lab Sheets
 - Quiz-I, Quiz-II
 - Assignment-I, Assignment-II
 - Sample QPs
- Lecture Recordings
 - Available on Microsoft teams

Honour Code



All submissions for graded components must be the result of your original effort. It is strictly prohibited to copy and paste verbatim from any sources, whether online or from your peers. The use of unauthorized sources or materials, as well as collusion or unauthorized collaboration to gain an unfair advantage, is also strictly prohibited. Please note that we will not distinguish between the person sharing their resources and the one receiving them for plagiarism, and the consequences will apply to both parties equally.

In cases where suspicious circumstances arise, such as identical verbatim answers or a significant overlap of unreasonable similarities in a set of submissions, will be investigated, and severe punishments will be imposed on all those found guilty of plagiarism.

In case of Queries regarding the course....

Step 1: Post in the discussion forum.

- Read through the existing post and if you find any topic similar to your concern, add on to the existing discussion.
- Avoid duplication of queries or issues.

Step 2: Email the IC at bharatesh.c@wilp.bits-pilani.ac.in if the query or issue is not resolved within 1 week. Turn around for a response to the email is 48 hours.

- In the subject pl mention the phrase "DNN" clearly.
- **Use BITS email id for correspondence.** Emails from personal emails will be ignored without any reply.

PATIENCE is highly APPRECIATED

What is Deep Learning?

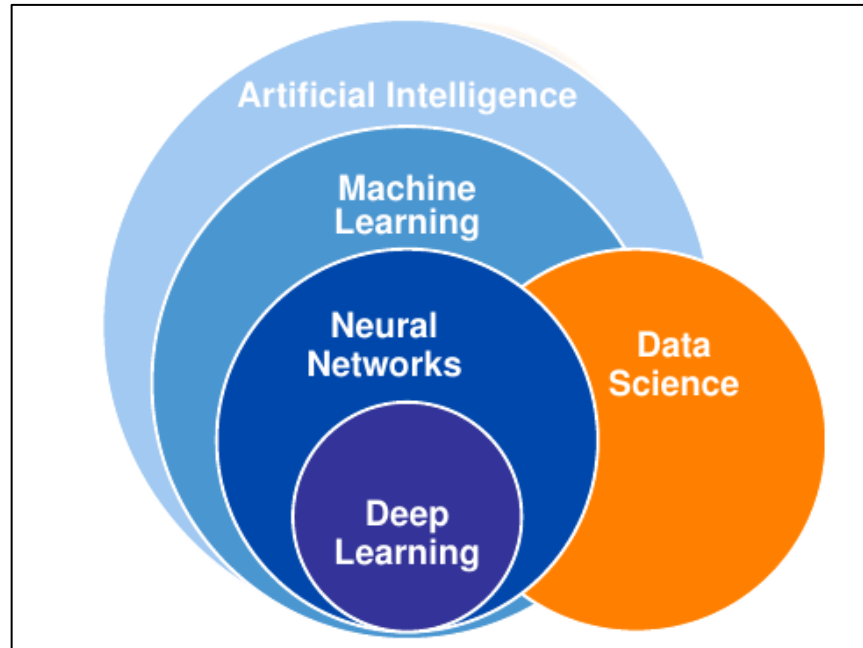
Definitions of Deep Learning

- Deep Learning is a type of **machine learning** based on **artificial neural networks** in which multiple layers of processing are used to extract progressively higher-level features from data.
- Deep learning is a method in artificial intelligence (AI) that teaches computers to process data in a way that is **inspired by the human brain**.
- Deep learning is a machine learning technique that teaches computers to do what comes naturally to humans: **learn by example**.
- Deep learning is a subset of machine learning, which is essentially a neural network with **three or more layers**.
- Deep Learning gets its name from the fact that we add more "**Layers**" to learn from the data.

Where in AI sits DL?



AI is a general field that encompasses machine learning and deep learning, but that also includes many more approaches that don't involve any learning.



AI : Artificial intelligence is the science of making things smart. The aim is make machines perform human tasks. Eg: Robot cleaning a room.

ML : Machine learning is an approach to AI. The machine learns or perform tasks through learning by experience.

DL : Deep Learning is a technique for implementing machine learning to recognise patterns.

Deep (Machine) Learning



- Deep learning is a specific subfield of machine learning.
- Learning representations from data that puts an emphasis on learning successive layers of increasingly meaningful representations.
- The **deep** in deep learning stands for this idea of successive layers of representations.
- The number of layers that contribute to model the data is called the **depth** of the model.
- In deep learning, the layered representations are learned via models called **neural networks**, structured in literal layers stacked on top of each other.

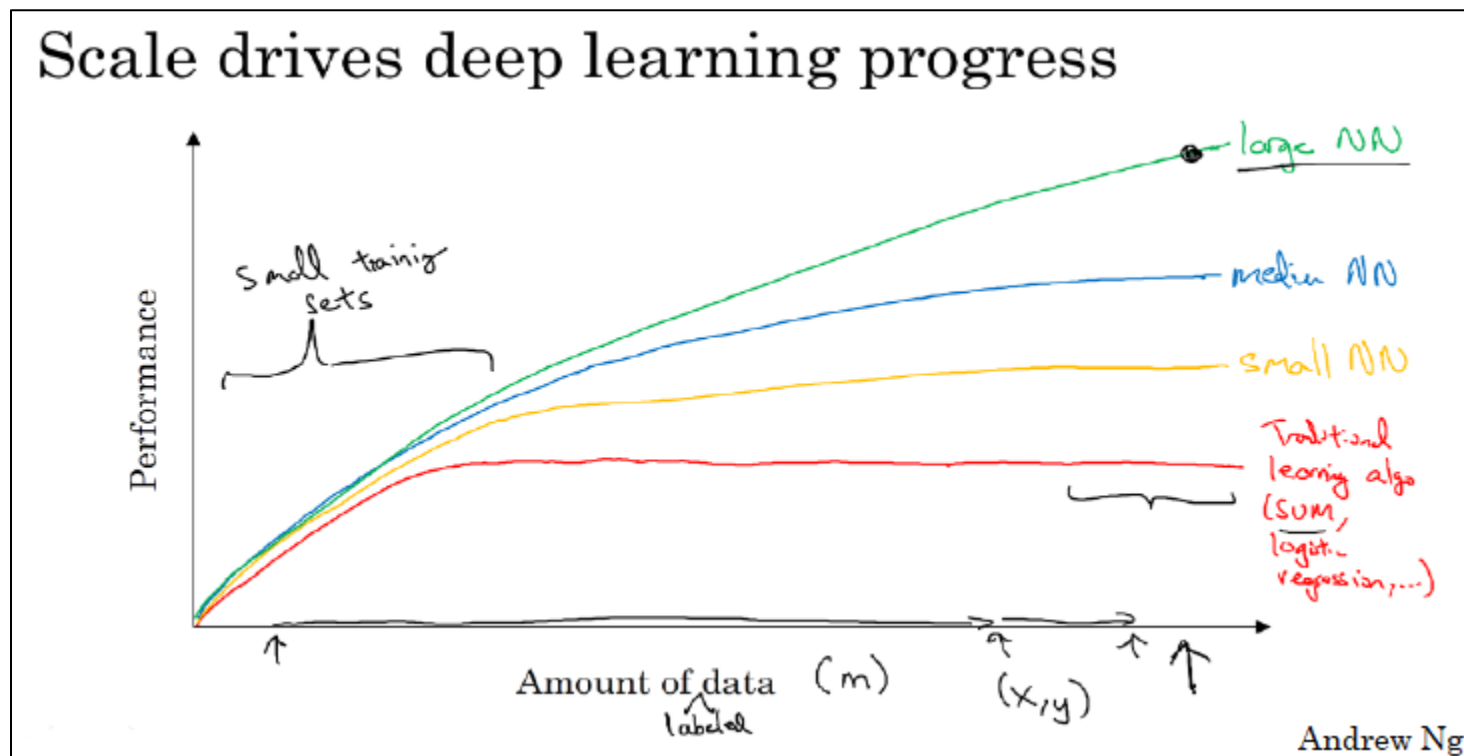
Why Deep Learning?

Why Deep Learning?

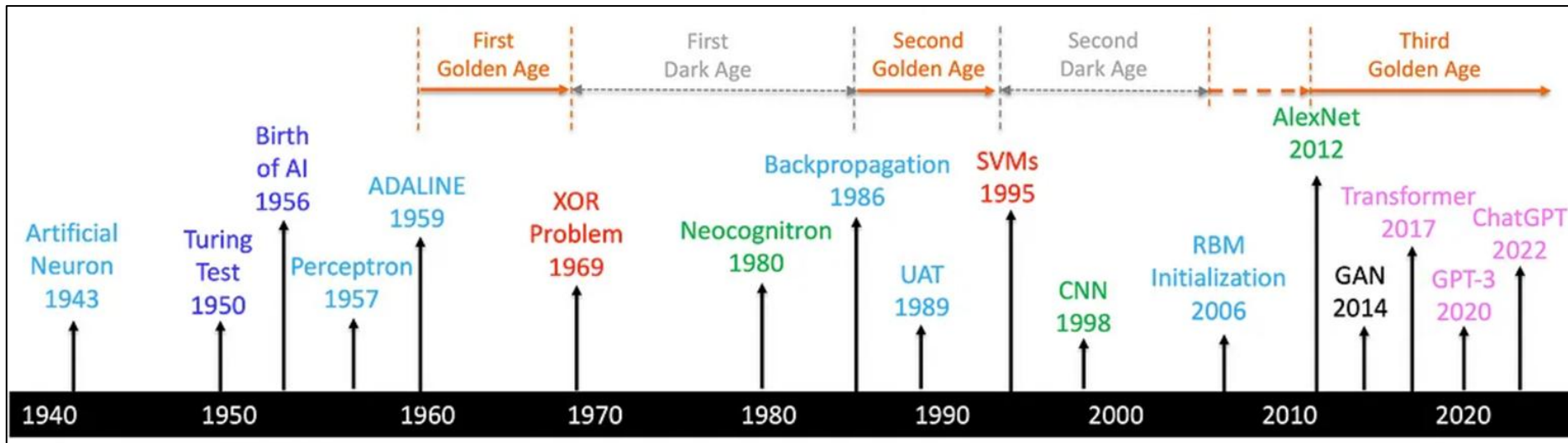


- Large amounts of data
- Lots and lots of unstructured data like images, text, audio, video
- Cheap, high-quality sensors
- Cheap computation - CPU, GPU, Distributed clusters
- Cheap data storage
- Learn by examples
- Automated feature generation
- Better learning capabilities
- Scalability
- Advance analytics can be applied

Why deep learning?



Deep Learning Timeline



Applications of Deep Learning

Microsoft AI Beats Humans at Speech Recognition

By Richard Adhikari

Oct 20, 2016 11:40 AM PT

Print
Email



Image: Adobe Stock

Microsoft's Artificial Intelligence and Research Unit earlier this week reported that its speech recognition technology had surpassed the performance of human transcriptionists.



Most Popular Newsletters News Alert

How do you feel about Black Friday and Cyber Monday?

- They're great -- I get a lot of bargains!
- The deals are too spread out -- I'd prefer just one day.
- They're a fun way to kick off the holiday season.
- I don't like the commercialization of Thanksgiving Day.
- They're crucial for the retail industry and the economy.
- The deals typically aren't that good.

Vote to See Results

E-Commerce Times

Black Friday Shoppers Hungry for New Experiences, New Tech

Pay TV's Newest Innovation: Giving Users Control

Apple Celebrates Itself in \$300 Coffee Table Tome

AWS Enjoys Top Perch in IaaS, PaaS Markets

US Comptroller Gears Up for Blockchain and

TRANSLATE

NOV 15, 2016

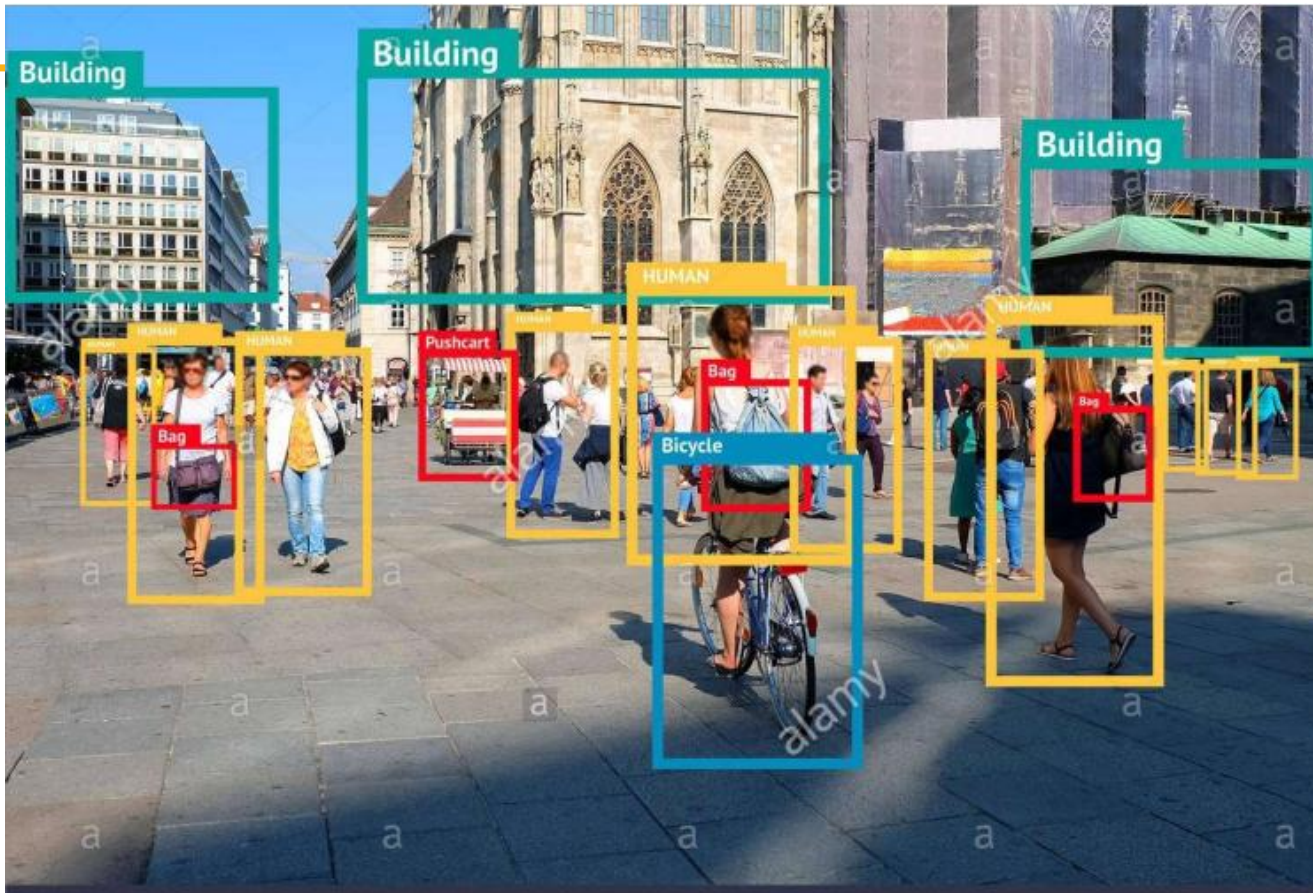
Found in translation: More accurate, fluent sentences in Google Translate

Barak Turovsky
PRODUCT LEAD, GOOGLE TRANSLATE

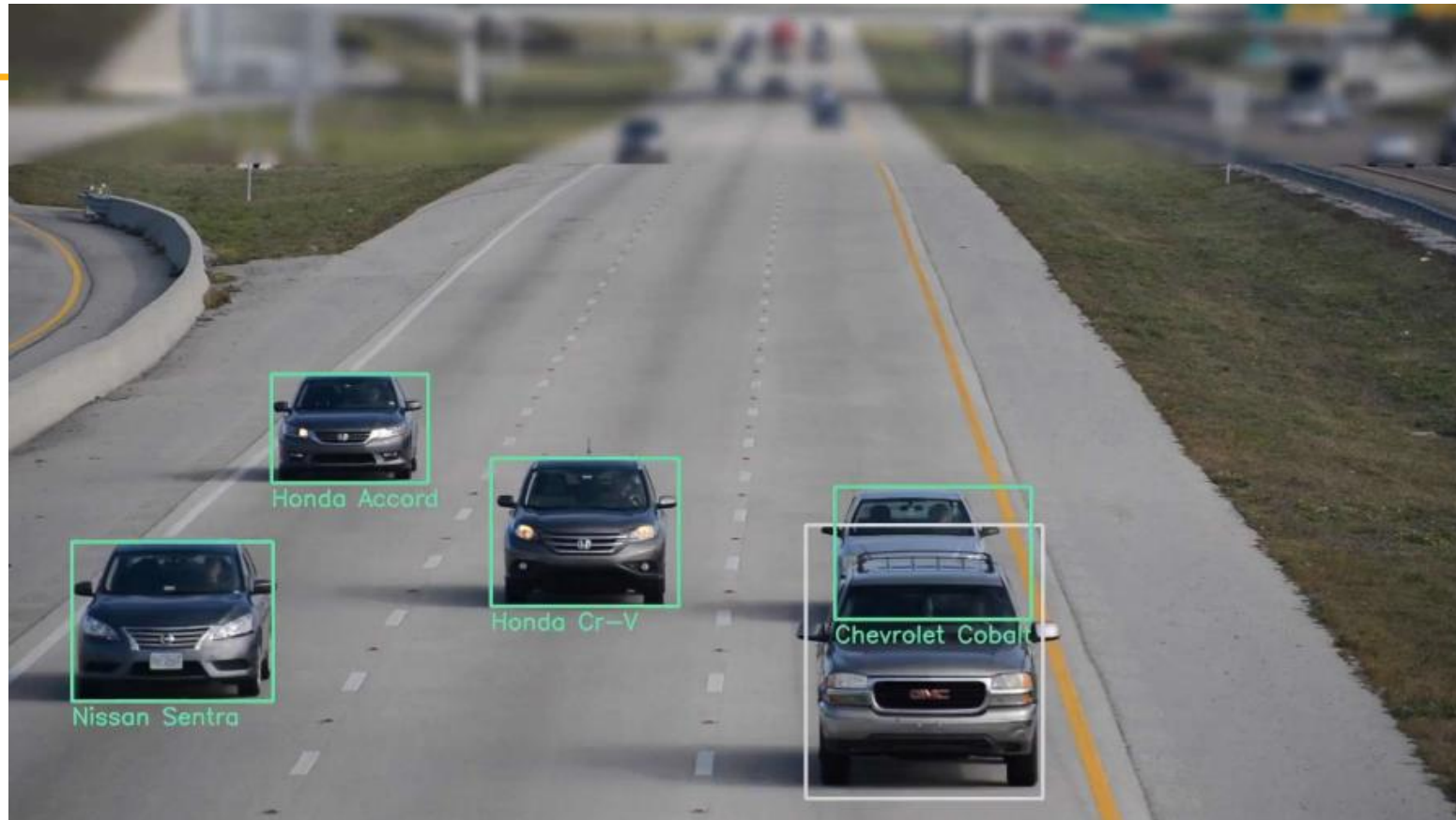
In 10 years, Google Translate has gone from supporting just a few languages to 103, connecting strangers, teaching across language barriers and even helping



Breakthroughs with Neural Networks



Breakthroughs with Neural Networks



Breakthroughs with Neural Networks

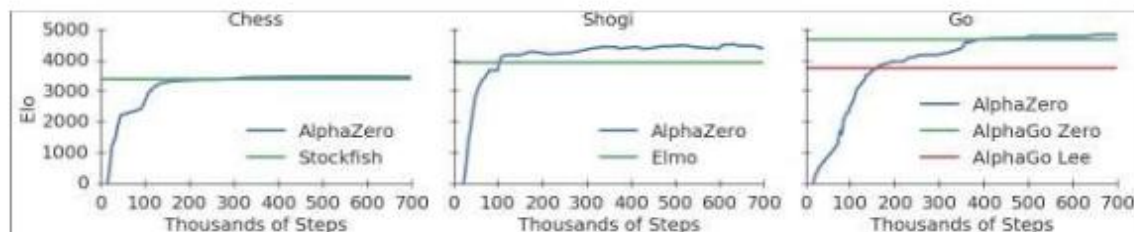


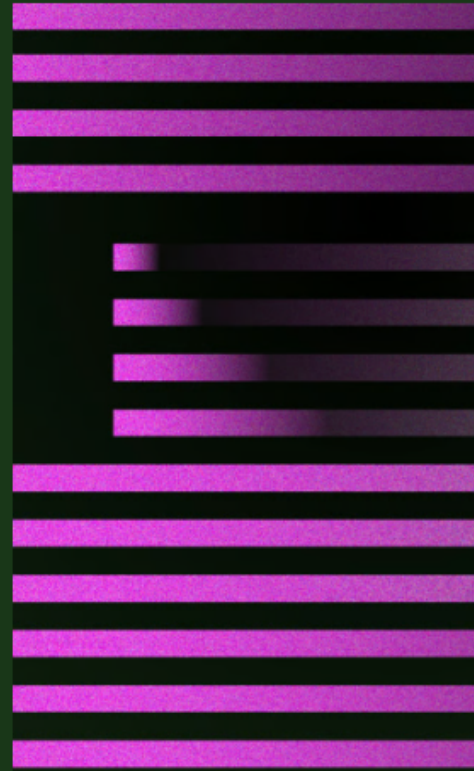
Figure 1: Training *AlphaZero* for 700,000 steps. Elo ratings were computed from evaluation games between different players when given one second per move. **a** Performance of *AlphaZero* in chess, compared to 2016 TCEC world-champion program *Stockfish*. **b** Performance of *AlphaZero* in shogi, compared to 2017 CSA world-champion program *Elmo*. **c** Performance of *AlphaZero* in Go, compared to *AlphaGo Lee* and *AlphaGo Zero* (20 block / 3 day) (29).

Introducing ChatGPT

We've trained a model called ChatGPT which interacts in a conversational way. The dialogue format makes it possible for ChatGPT to answer followup questions, admit its mistakes, challenge incorrect premises, and reject inappropriate requests.

[Try ChatGPT ↗](#)

[Read about ChatGPT Plus](#)



Applications of Deep Learning



Application	Input	Output	Neural Network
Real Estate	House features	House Price	Std NN
Photo Tagging	Image	Text	CNN
Object detection	Image	Bounding box	CNN
Speech Recognition	Audio	Text transcript	RNN
Translation	English Text	French Text	RNN
Autonomous driving	Image, Sensors Radars	Position of other cars, objects, signals	Hybrid NN

Many more applications....



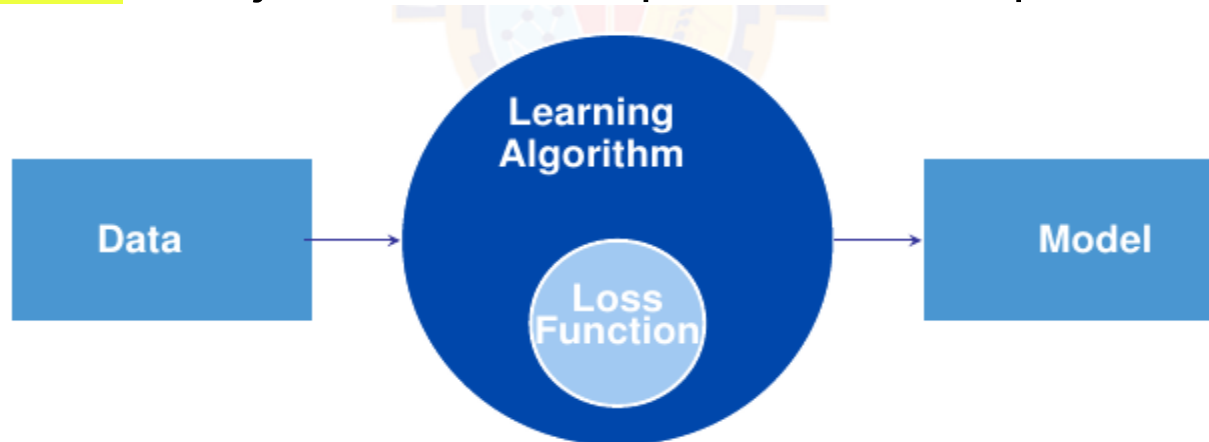
- a program that predicts tomorrow's weather given geographic information, satellite images, and a trailing window of past weather.
- a program that takes in a question, expressed in free-form text, and answers it correctly.
- a program that given an image can identify all the people it contains, drawing outlines around each.
- a program that presents users with products that they are likely to enjoy but unlikely, in the natural course of browsing, to encounter.

Key components of DL problem

Core components of DL problem



1. The **data** that we can learn from.
2. A **model** of how to transform the data.
3. An **objective function** that quantifies how well (or badly) the model is doing.
4. An **algorithm** to adjust the model's parameters to optimize the objective



1. Data

- Collection of examples.
- Data has to be converted to an useful and a suitable numerical **representation**.
- Each example (or data point, data instance, sample) typically consists of a set of attributes called **features** (or covariates), from which the model must make its predictions.
- In the supervised learning problems, the attribute to be predicted is designated as the **label** (or target).
- Mathematically, a set of m examples,
- We need **right** data.

$$Data = \mathcal{D} = \{X, t\}$$

1. Data



- **Dimensionality** of data

- Each example has the same number of numerical values. This data consist of fixed-length vectors. Eg: Image
- The constant length of the vectors as the dimensionality of the data.
- Text data has varying-length data.

2. Model



- Model denotes the **computational machinery** for ingesting data of one type, and spitting out predictions of a possibly different type.
- Deep learning models consist of many successive transformations of the data that are chained together top to bottom, thus the name deep learning.

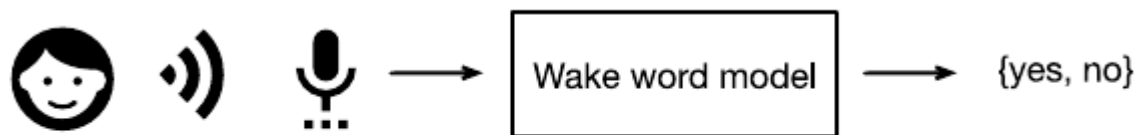


Fig. 1.1.1: Identify a wake word.

3. Objective Function



- **Learning means improving at some task over time.**
- A formal mathematical system of learning machines is defined using formal measures of how good (or bad) the models are. These formal measures are called as **objective functions**.
- By convention, objective functions are defined so that lower is better.
- Because lower is better, these functions are sometimes called **loss functions**.

3. Loss Functions



- To predict numerical values (regression), the most common loss function is squared error.
- For classification, the most common objective is to minimize error rate, i.e., the fraction of examples on which our predictions disagree with the ground truth.

3. Loss Functions



- Loss function is defined **with respect to the model parameters** and **depends upon the dataset**.
- We learn the best values of our model parameters by **minimizing the loss** incurred on a set consisting of some number of examples collected for training. However, doing well on the training data does not guarantee that we will do well on unseen data. I.e **Model has to generalize better**.
- When a model performs well on the training set but fails to generalize to unseen data, we say that it is **overfitting**.

4. Optimization Algorithms



- Optimization Algorithm is an algorithm capable of **searching for the best possible parameters** for **minimizing the loss function**.
- Popular optimization algorithms for deep learning are based on an approach called **gradient descent**.

Example of the Framework



- We have to tell a computer explicitly how to map from inputs to outputs.
- We have to define the problem precisely, pinning down the exact nature of the inputs and outputs, and choosing an appropriate model family.
- Collect a huge dataset containing examples of audio and label those that do and that do not contain the wake word.

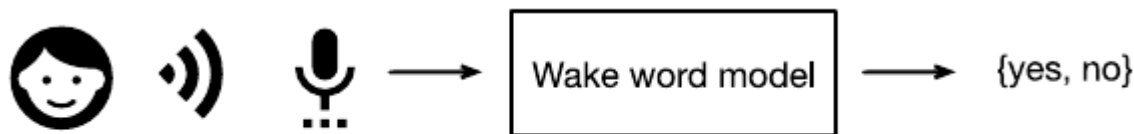


Fig. 1.1.1: Identify a wake word.

Example of the Framework



- Create a Model

- Define a flexible program whose behavior is determined by a number of parameters.
- To determine the best possible set of parameters, use the data. The parameters should improve the performance of the program with respect to some measure of performance on the task of interest.
- After fixing the parameters, we call the program a model.
 - Eg: The model receives a snippet of audio as input, and the model generates a selection among yes, no as output.
- The set of all distinct programs (input-output mappings) that we can produce just by manipulating the parameters is called a family of models.
 - Eg: We expect that the same model family should be suitable for "Alexa" recognition and "Hey Siri" recognition because they seem, intuitively, to be similar tasks.

Example of the Framework



- The meta-program that uses our dataset to choose the parameters is called a learning algorithm.
- In machine learning, the learning is the process by which we discover the right setting of the parameter coercing the desired behavior from our model.
- Train the model with data.

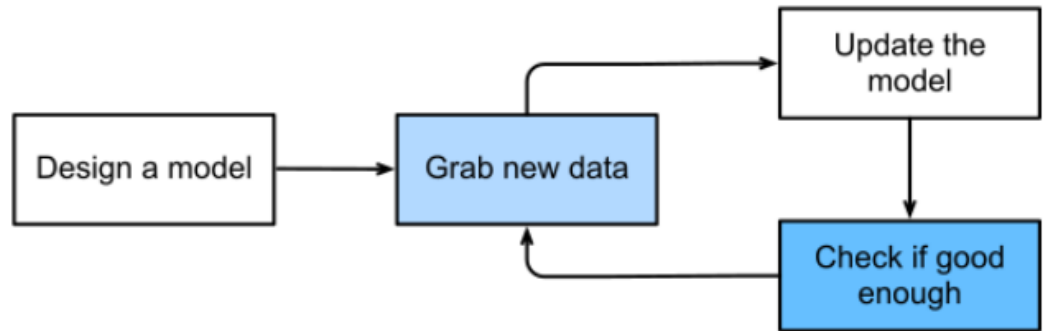


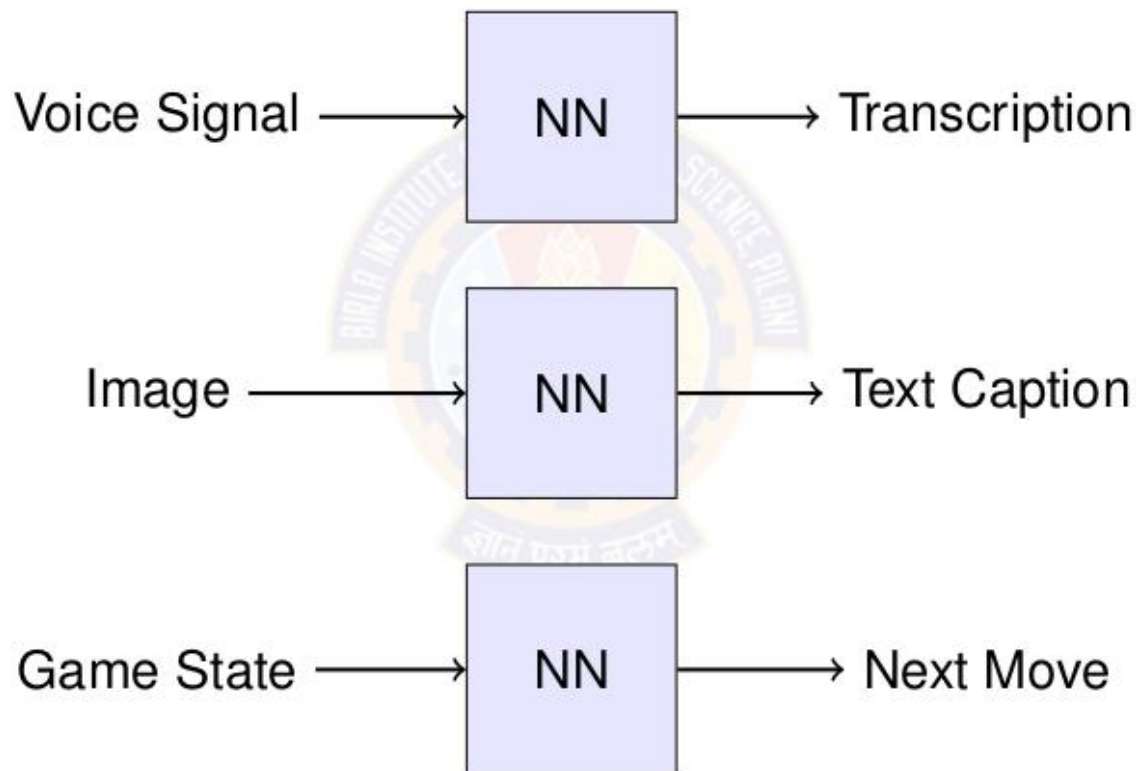
Fig. 1.1.2: A typical training process.

Reading from TB Dive into Deep Learning

- Chapter 1
- Chapter 2 for Python Prelims, Linear Algebra, Calculus, Probability

Artificial Neural Network

What are Neural Networks?



What are Neural Networks?



- It begins with human brain.



- Humans learn, solve problems, recognize patterns, create, think deeply about something, meditate and many many more.....
- Humans learn through **association**. [Refer to Associationism for more details.]

Observation: The Brain



- The brain is a mass of interconnected neurons.
- Number of neurons is approximately 10^{10} .
- Connections per neuron is approximately $10^{(4 \text{ to } 5)}$.
- Neuron switching time is approximately 0.001 second.
- Scene recognition time is 1 second.
- 100 inference steps doesn't seem like enough. Lot of parallel computation.



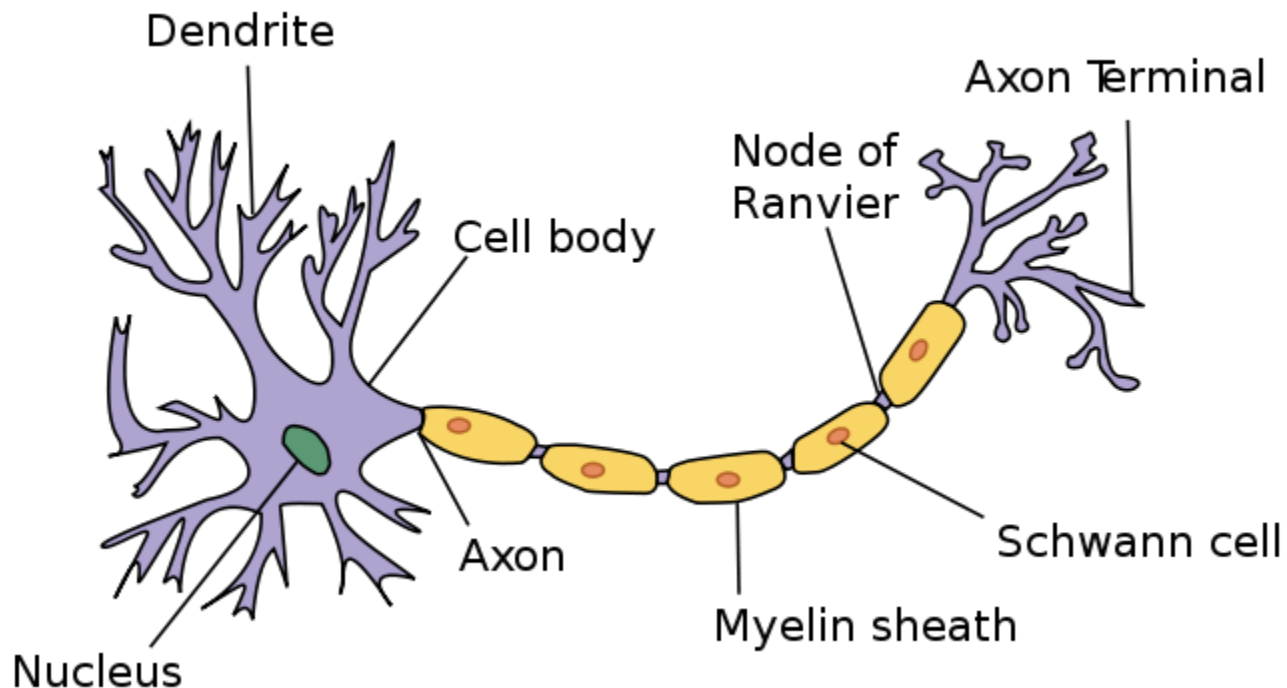
Brain: Interconnected Neurons



- Many neurons connect in to each neuron.
- Each neuron connects out to many neurons.



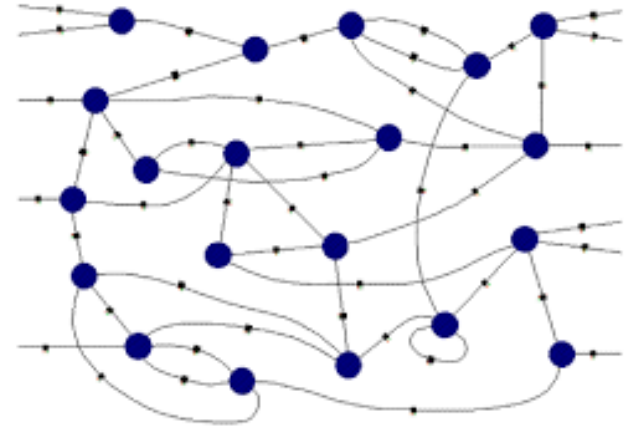
Biological Neuron



Connectionist Machines



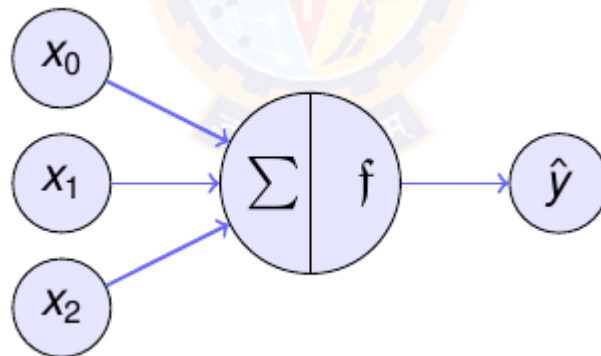
- Network of processing elements, called artificial neural unit.
- The neurons are interconnected to form a network.
- All world knowledge is stored in the connections between these elements.
- Neural networks are connectionist machines.



What are Artificial Neurons?



- Neuron is a processing element inspired by how the brain works.
- Similar to biological neuron, each artificial neuron will be do some computation. Each neuron is interconnected to other neurons.
- Similar to brain, the interconnections between neurons store the knowledge it learns. The knowledge is stored as parameters.



Properties of Artificial Neural Nets (ANNs)

- Many neuron-like threshold switching units.
- Many weighted interconnections among units.
- Highly parallel, distributed process.
- Emphasis on tuning parameters or weights automatically.

When to consider Neural Networks?



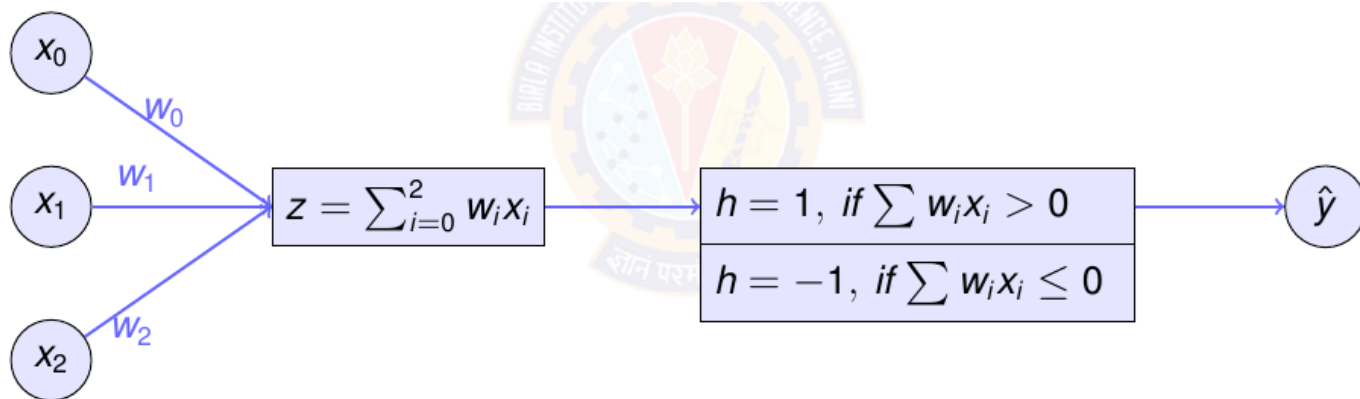
- Input is high-dimensional discrete or real-valued (e.g. raw sensor input).
- Possibly noisy data. Data has lots of errors.
- Output is discrete or real valued or a vector of values.
- Form of target function is unknown.
- Human readability, in other words, explainability, of result is unimportant.
- Examples:
 - Speech phoneme recognition
 - Image classification
 - Financial prediction

Perceptron

Perceptron



- One type of ANN system is based on a unit called a perceptron.
- **A perceptron takes a vector of real-valued inputs, calculates a linear combination of these inputs, and then outputs a 1 if the result is greater than some threshold and -1 otherwise.**



Inputs

Weights

Hypothesis

Activation

Output

Representing Logic Gates using Perceptron

- **Perceptron** can represent all of the primitive Boolean functions AND, OR, NAND, and NOR – Linearly separable data
- some Boolean functions cannot be represented by a single perceptron, such as the XOR function – Linearly nonseparable data
- Perceptron learning rule finds a successful weight vector when the training examples are linearly separable, **it can fail to converge if the examples are not linearly separable.**

NOT Logic Gate



Question:

- How to represent NOT gate using a perceptron?
- What are the parameters for the NOT perceptron?
- Data is given below.



A	B
0	1
1	0

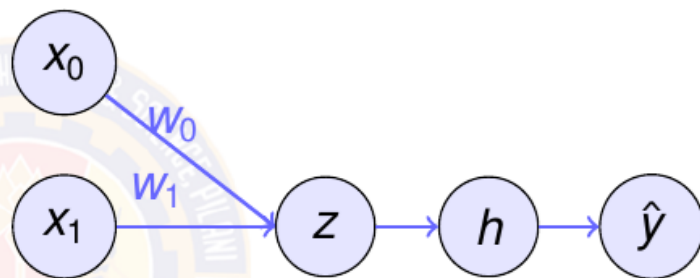
Rewrite as

x_1	t
-1	1
1	-1

Perceptron for NOT gate



- Perceptron equation is $\hat{y} = w_0x_0 + w_1x_1$.
- $x_0 = 1$ always.
- $h > 0$ for output to be 1.
- For each row of truth table, the equations are given.
- One solution is $w_0 = 1$ and $w_1 = -1$. (Intuitive solution)
- This give a beautiful linear decision boundary.



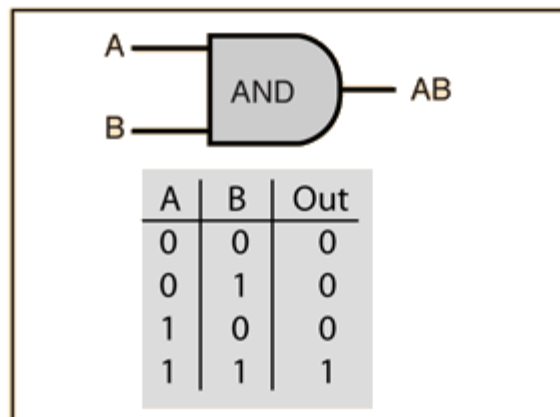
x_1	t_1	h	h
-1	1	$w_0x_0 + w_1(-1) > 0$	$w_0 - w_1 > 0$
1	-1	$w_0x_0 + w_1(1) < 0$	$w_0 + w_1 < 0$

AND Logic Gate



Question:

- How to represent AND gate using a perceptron?
- What are the parameters for the AND perceptron?
- Data is given below.



Perceptron for AND gate



- Perceptron equation is
 $\hat{y} = w_0x_0 + w_1x_1 + w_2x_2.$

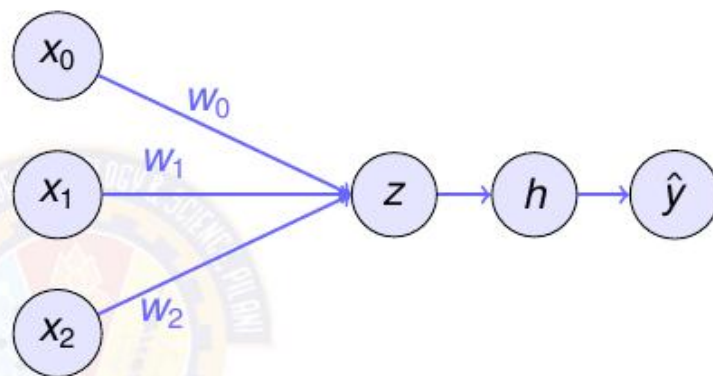
- $h > 0$ for output to be 1.

- For each row of the truth table, the equations are given.

- Solve for the inequalities.

- One solution is
 $w_1 = w_2 = 2, w_0 = (-1).$

- This give a beautiful linear decision boundary.



x_1	x_2	t	h
-1	-1	-1	$w_0 + w_1(-1) + w_2(-1) < 0$
-1	1	-1	$w_0 + w_1(-1) + w_2(1) < 0$
1	-1	-1	$w_0 + w_1(1) + w_2(-1) < 0$
1	1	1	$w_0 + w_1(1) + w_2(1) > 0$

Perceptron for AND gate

innovate

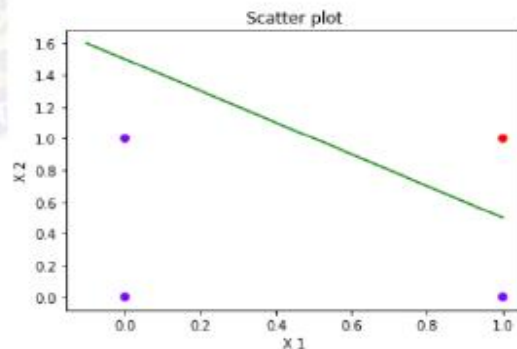
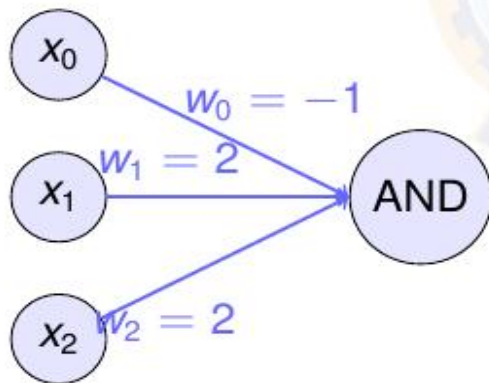
achieve

lead



A	B	AB
0	0	0
0	1	0
1	0	0
1	1	1

DECISION BOUNDARY



Exercise

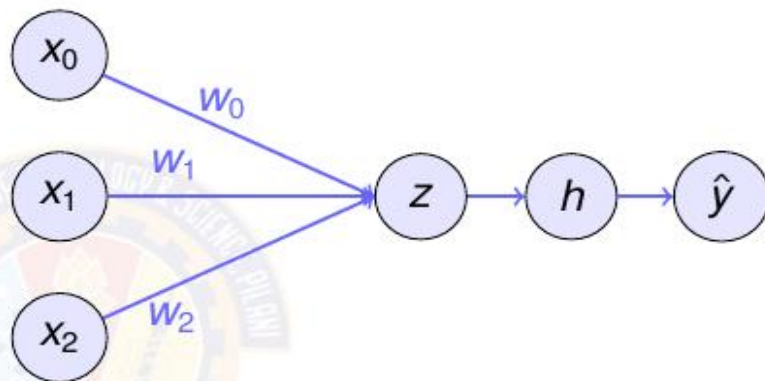


1. Represent OR gate using Perceptron. Compute the parameters of the perceptron.
2. Represent NOR gate using Perceptron. Compute the parameters of the perceptron.
3. Represent NAND gate using Perceptron. Compute the parameters of the perceptron.

Perceptron for OR gate



- Perceptron equation is $\hat{y} = w_0x_0 + w_1x_1 + w_2x_2$.
- $h > 0$ for output to be 1.
- For each row of the truth table, the equations are given.
- Solve for the inequalities.
- One solution is $w_0 = w_1 = w_2 = 2$.
- This give a beautiful linear decision boundary.



x_1	x_2	t	h
-1	-1	-1	$w_0 + w_1(-1) + w_2(-1) < 0$
-1	1	-1	$w_0 + w_1(-1) + w_2(1) > 0$
1	-1	-1	$w_0 + w_1(1) + w_2(-1) > 0$
1	1	1	$w_0 + w_1(1) + w_2(1) > 0$

Perceptron for OR gate

innovate

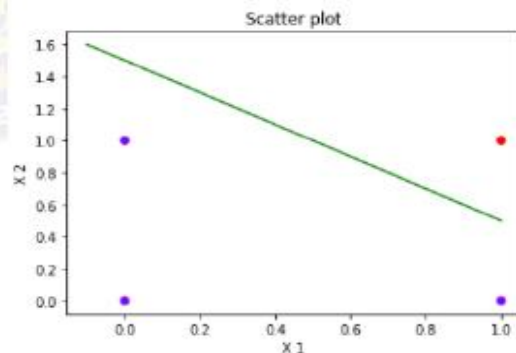
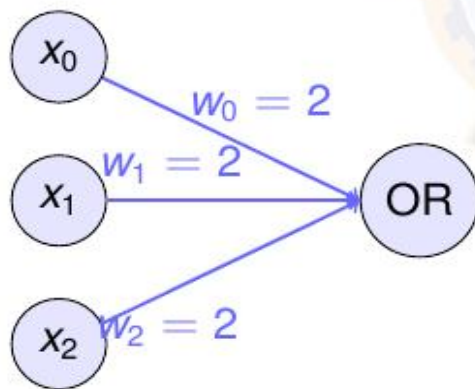
achieve

lead



A	B	AB
0	0	0
0	1	1
1	0	1
1	1	1

DECISION BOUNDARY



Perceptron Learning Algorithm

Perceptron Learning Algorithm



$$w_i \leftarrow w_i + \Delta w_i$$

where

$$\Delta w_i = \eta(t - o)x_i$$

Where:

- $t = c(\vec{x})$ is target value
- o is perceptron output
- η is small constant (e.g., .1) called *learning rate*

Convergence of Perceptron Learning Algorithm

It can be proved that the algorithm will converge

- If training data is linearly separable.
- Learning rate is sufficiently small
 - The role of the learning rate is to moderate the degree to which weights are changed at each step.
 - It is usually set to some small value (e.g., 0.1) and is sometimes made to decay as the number of weight-tuning iterations increases.

Perceptron Learning Algorithm for NOT gate

- Assume $w_0 = w_1 = 0$. Let the learning rate $= \eta = 1$.
- Equations are

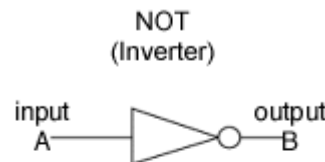
Hypothesis $z = w_0 + w_1 x_1$

Activation $h = \text{sign}(z)$

Compute output $\hat{y} = h = \text{sign}(w_0 + w_1 x_1)$

Compute $\Delta w = \eta(t - \hat{y})x$

Update $w_{\text{new}} \leftarrow w_{\text{old}} + \Delta w$



A	B
0	1
1	0

Perceptron Learning Algorithm for NOT gate

$w_1=w_0=0, \eta =1$

X1	t	W1	W0	z	h	Isequal(t,h)	Δw	New w
-1	1	0	0	$0+0=0$	-1	No	$\Delta w_1=1(1+1)*(-1)=-2$ $\Delta w_0=1(1+1) =2$	$w_1 \Rightarrow 0-2=-2$ $w_0 \Rightarrow 0+2=2$
1	-1	-2	2	$2-2=0$	-1	Yes	$\Delta w_1=1(-1+1)1=0$ $\Delta w_0=1(-1+1) =0$	$w_1 \Rightarrow -2+0=-2$ $w_0 \Rightarrow 2+0=2$

X1	t	W1	W0	z	h	Isequal(t,h)	Δw	New w
-1	1	-2	2	$2+2=4$	1	Yes	$\Delta w_1=1(1-1)*(-1)=0$ $\Delta w_0=1(1-1) =0$	$w_1 \Rightarrow -2+0=-2$ $w_0 \Rightarrow 2+0=2$

Exercise

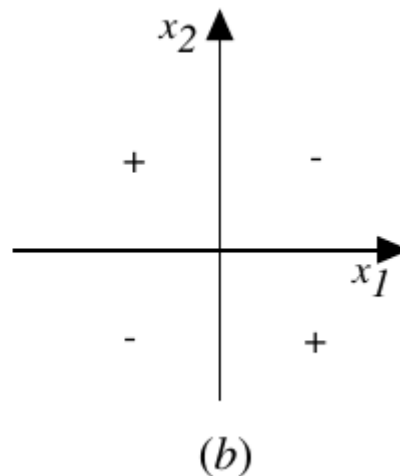
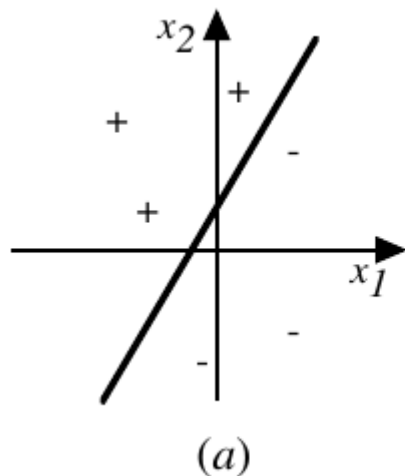


1. Represent OR gate using Perceptron. Compute the parameters of the perceptron using perceptron learning algorithm.
2. Represent AND gate using Perceptron. Compute the parameters of the perceptron using perceptron learning algorithm.

Representational Power of Perceptrons



- A perceptron represents a hyperplane decision surface in the n -dimensional space of examples.
- The perceptron outputs a 1 for examples lying on one side of the hyperplane and outputs a -1 for examples lying on the other side.

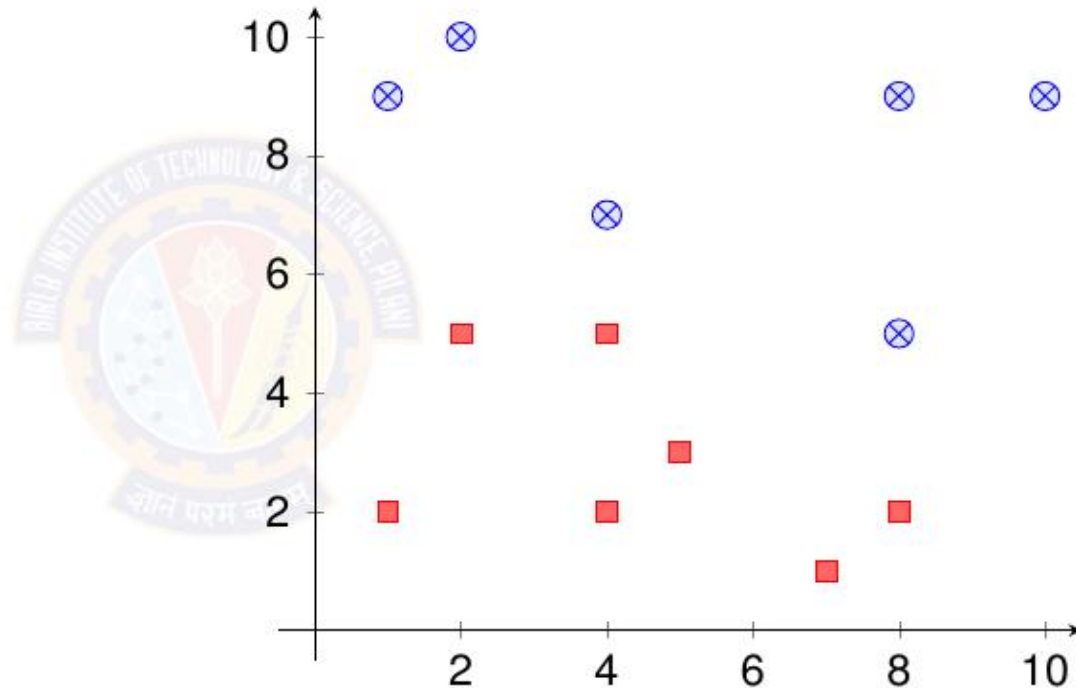


Single Layer Perceptron

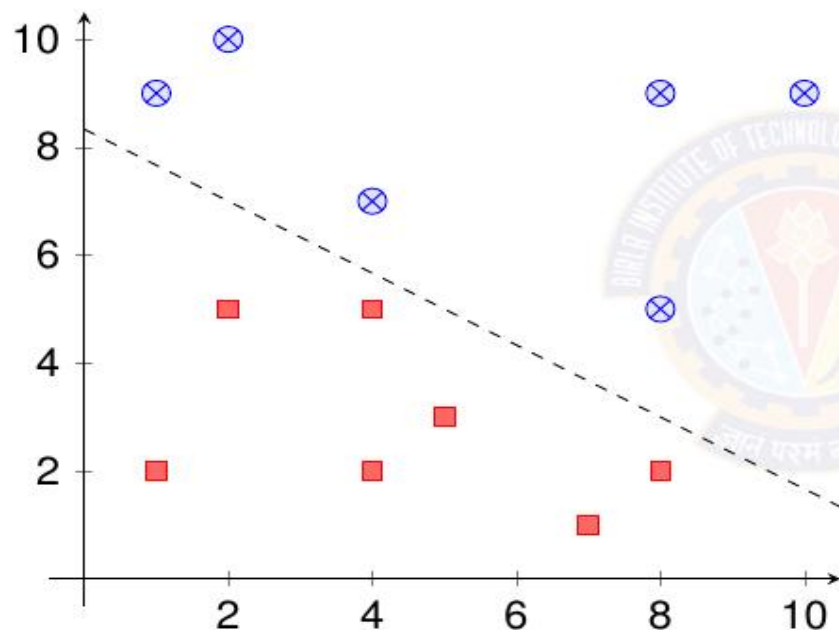
Example of Linearly Separable data



x_1	x_2	y
1	9	white
10	9	white
4	7	white
4	5	pink
5	3	pink
8	9	white
4	2	pink
2	5	pink
7	1	pink
2	10	white
8	5	white
1	2	pink
8	2	pink



Example of Linearly Separable data



- Linearly separable data.

- Line equation say

$$2x_1 + 3x_2 - 25 = 0$$

- The line becomes the **decision boundary**.

- New point (0, 0) < 25 so pink interior.

- New point (10, 10) > 25 so white interior.

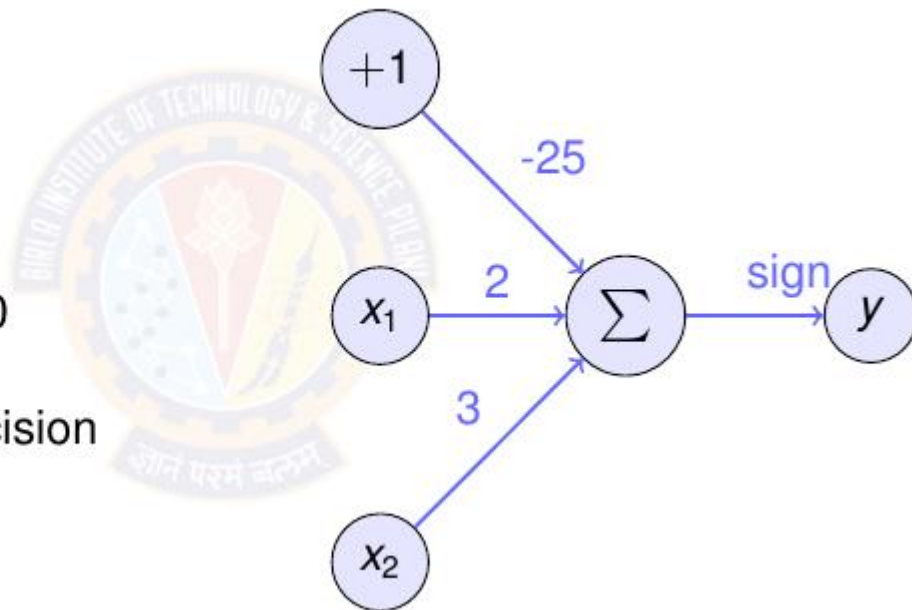
Example of Linearly Separable data



- Linearly separable data.
- Line equation say

$$2x_1 + 3x_2 - 25 = 0$$

- The line becomes the decision boundary.

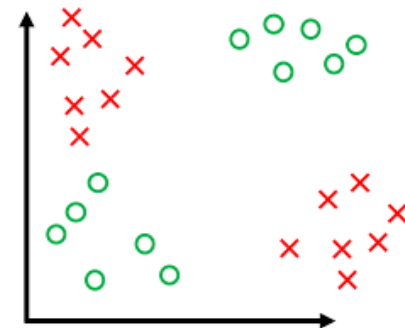
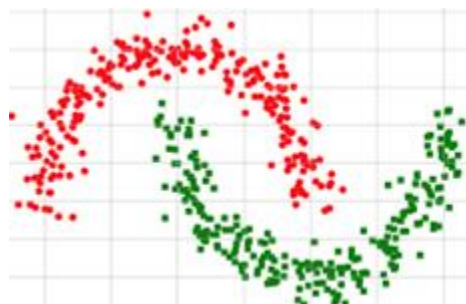
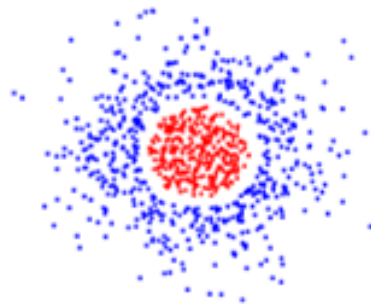


Non-Linearly Separable Data

Non-linearly Separable Data



- Two groups of data points are non-linearly separable in a 2-dimensional space if they cannot be easily separated with a linear line.

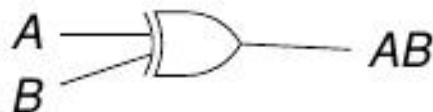


Perceptron for XOR Gate



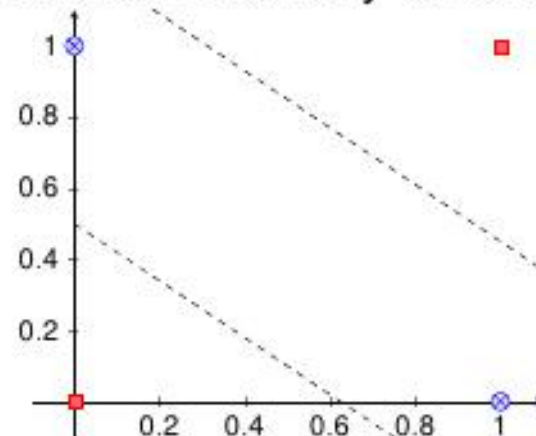
Question:

- How to represent XOR gate using a Perceptron?
- What are the parameters for the XOR Perceptron?
- Data is given below.



A	B	AB
0	0	0
0	1	1
1	0	1
1	1	0

Challenge: Data is non-linearly separable.
Decision boundary for XOR

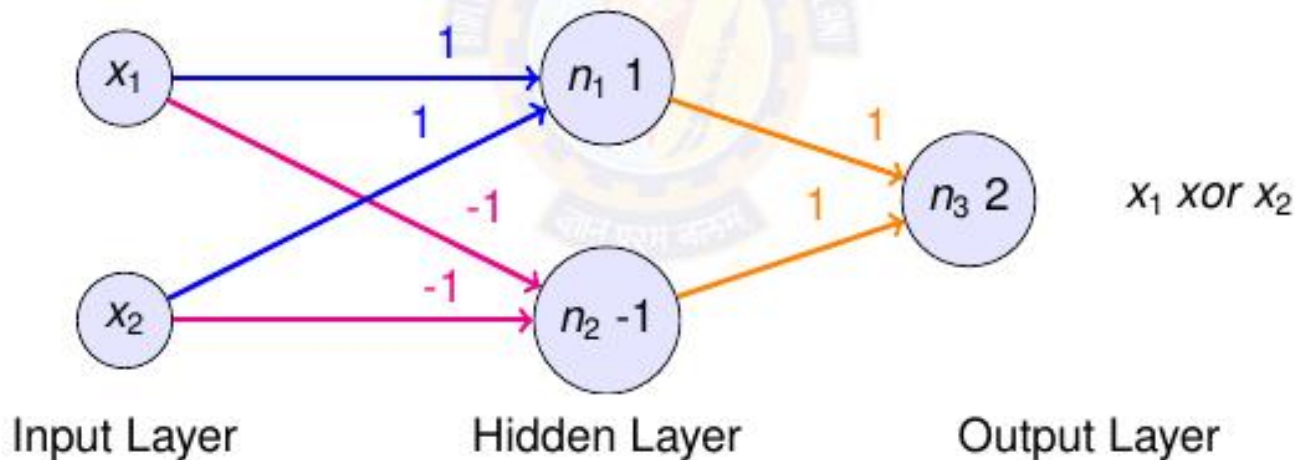


Solution for XOR

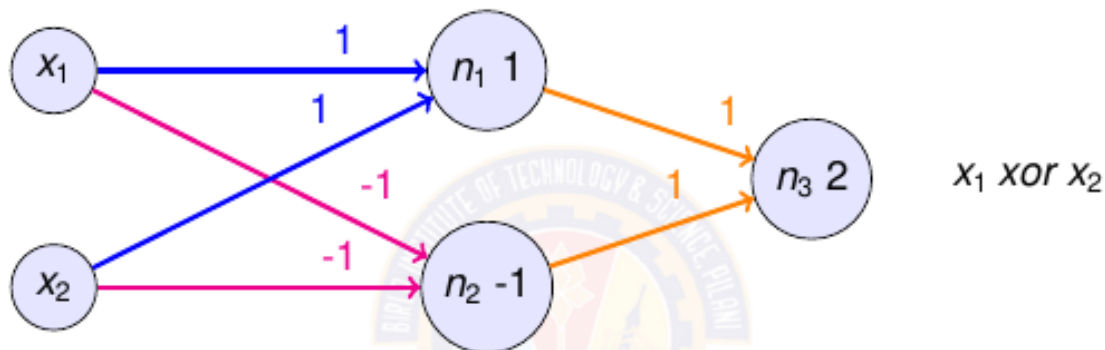


Qn: How to represent XOR gate using a Perceptron?

- Use Multilayer Perceptron (MLP)
- Introduce another layer in between the input and output.
- This in-between layer is called hidden layer.



MLP for XOR



x_1	x_2	n_1	n_2	n_3
0	0	$0 \cdot 1 + 0 \cdot 1 = 0 \not\geq th$ $n_1 = 0$	$0 \cdot (-1) + 0 \cdot (-1) = 0 > th$ $n_2 = 1$	$0 \cdot 1 + 1 \cdot 1 = 1 \not\geq th$ $n_3 = 0$
0	1	$0 \cdot 1 + 1 \cdot 1 = 1 \geq th$ $n_1 = 1$	$0 \cdot (-1) + 1 \cdot (-1) = -1 \geq th$ $n_2 = 1$	$1 \cdot 1 + 1 \cdot 1 = 2 \geq th$ $n_3 = 1$
1	0	$1 \cdot 1 + 0 \cdot 1 = 1 \geq th$ $n_1 = 1$	$1 \cdot (-1) + 0 \cdot (-1) = -1 \geq th$ $n_2 = 1$	$1 \cdot 1 + 1 \cdot 1 = 2 \geq th$ $n_3 = 1$
1	1	$1 \cdot 1 + 1 \cdot 1 = 2 \geq th$ $n_1 = 1$	$1 \cdot (-1) + 1 \cdot (-1) = -2 \not\geq th$ $n_2 = 0$	$1 \cdot 1 + 0 \cdot 1 = 1 \not\geq th$ $n_3 = 0$

Ref:
Chapter 4 of Book: Machine Learning by
Tom M. Mitchell



Thank You All!



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Deep Neural Networks

The author of this deck, Prof. Seetha Parameswaran, is gratefully acknowledging the authors who made their course materials freely available online.



Module 2



Human Brain

How do Humans Learn?

- It begins with human brain.
- Humans learn, solve problems, recognize patterns, create, think deeply about something, meditate and many many more.....
- Humans learn through **association**. [Refer to Associationism for more details.]



Observation: The Brain

- The brain is a mass of interconnected neurons.
- Number of neurons is approximately 10^{10} .
- Connections per neuron is approximately $10^{(4 \text{ to } 5)}$.
- Neuron switching time is approximately 0.001 second.
- Scene recognition time is 1 second.
- 100 inference steps doesn't seem like enough. Lot of parallel computation.



Brain: Interconnected Neurons

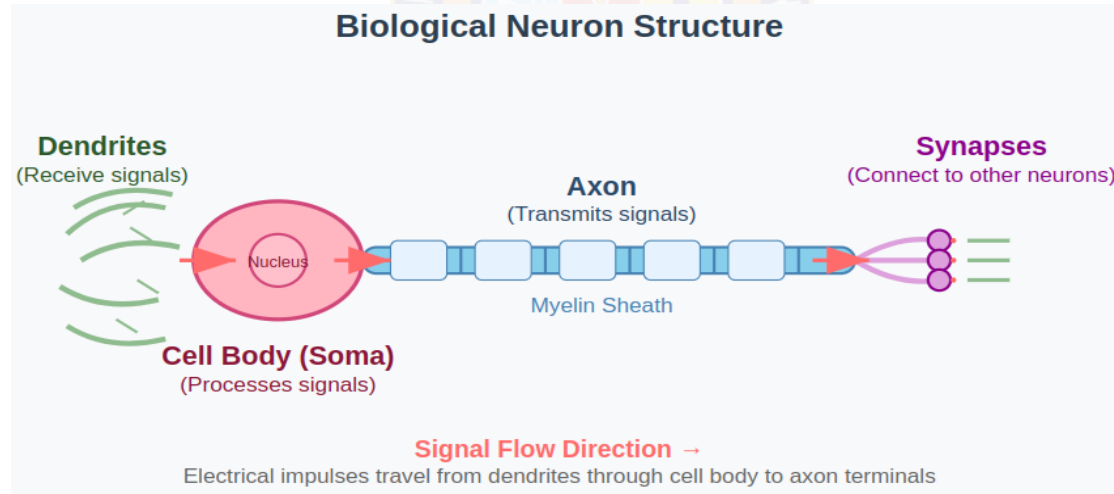
- Many neurons connect **in** to each neuron.
- Each neuron connects **out** to many neurons.



Biological Neuron

Key Components:

- **Dendrites:** Receive signals from other neurons
- **Cell Body (Soma):** Processes incoming signals
- **Axon:** Transmits output signal
- **Synapses:** Connection points between neurons

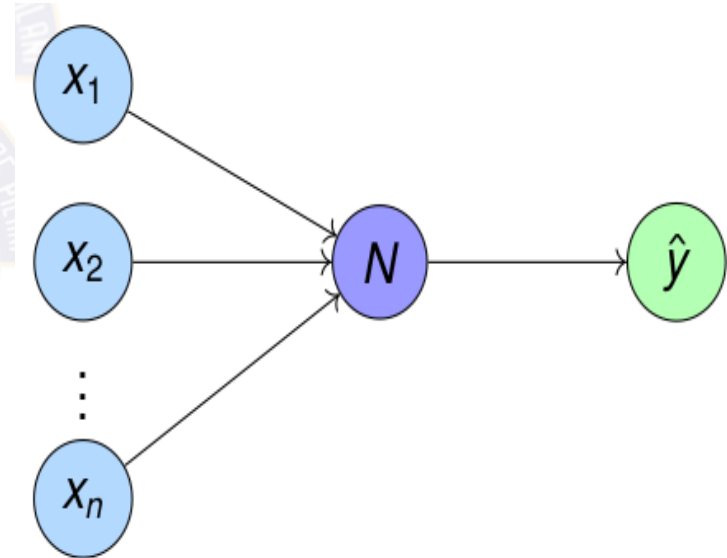




Artificial Neuron

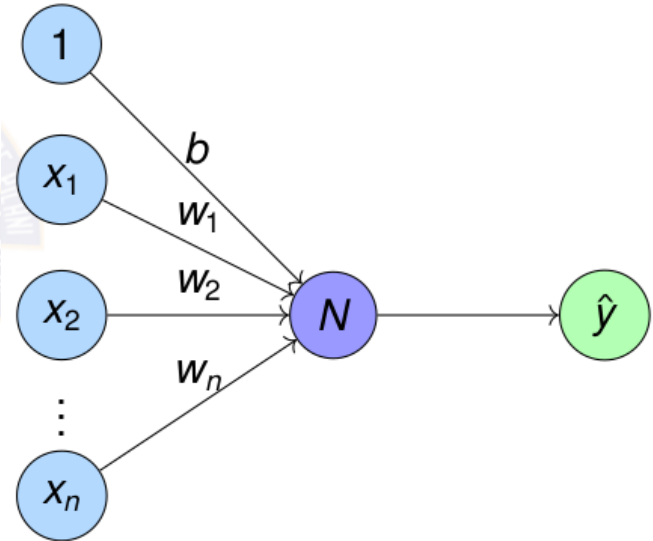
What are Artificial Neurons?

- Neuron is a processing element inspired by how the brain works.
- Similar to biological neuron, each artificial neuron will be do some computation. Each neuron is interconnected to other neurons.
- Similar to brain, the interconnections between neurons store the knowledge it learns. The knowledge is stored as parameters.



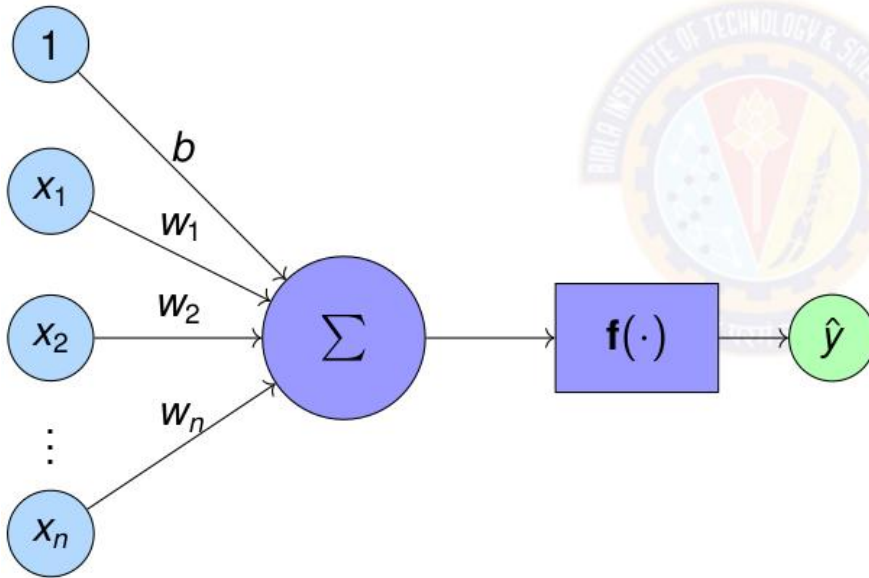
Artificial Neuron N

- Processing neuron N performs a very basic computation.
- For each feature x_i , weight w_i shows the importance to the feature
- N generates one numerical output $\hat{y}$



Artificial Neuron – Mathematical Computation

- Sum of weights w_i times corresponding features x_i
- Apply activation function $f(\cdot)$ on the sum.



$$z = \sum_{i=1}^n w_i x_i + b \quad (1)$$

$$\hat{y} = f\left(\sum_{i=1}^n w_i x_i + b\right) \quad (2)$$

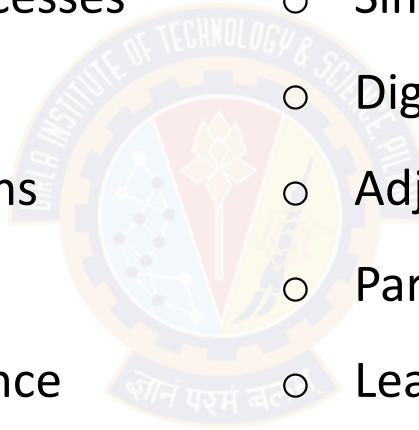
Comparison: Biological vs Artificial Neurons

Biological Neuron

- Complex biochemical processes
- Analog signal processing
- Adaptive synaptic strengths
- Parallel processing
- Learning through experience
- Fault tolerant

Artificial Neuron

- Simple mathematical model
- Digital computation
- Adjustable weights
- Parallel computation possible
- Learning through algorithms
- Deterministic behavior

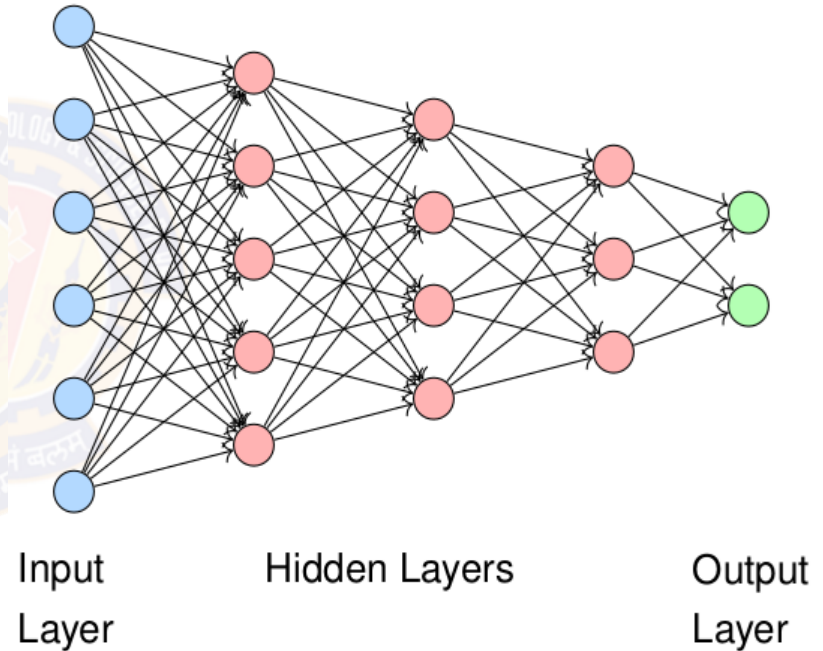




Artificial Neural Network (ANN)

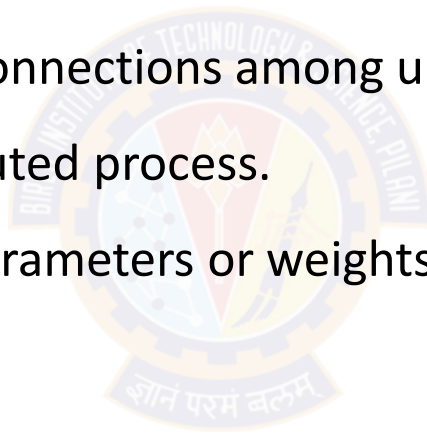
What are Artificial Neural Networks?

- Neural Network is a computational model inspired by human brain
- Interconnected neurons process information
- Interconnection can be forward, recursive or self loop
- Learn patterns from data through training
- Capable of approximating complex functions



Properties of Artificial Neural Network

- Many neuron-like threshold switching units.
- Many weighted interconnections among units.
- Highly parallel, distributed process.
- Emphasis on tuning parameters or weights automatically.



When to consider Artificial Neural Networks?

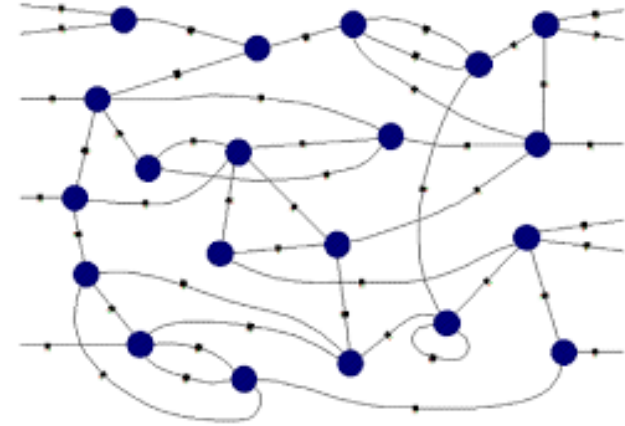
- Input is high-dimensional discrete or real-valued (e.g. raw sensor input).
- Possibly noisy data. Data has lots of errors.
- Output is discrete or real valued or a vector of values.
- Form of target function is unknown.
- Human readability, in other words, explainability , of result is unimportant.

Examples:

- Speech phoneme recognition
- Image classification
- Financial prediction

Connectionist Machines

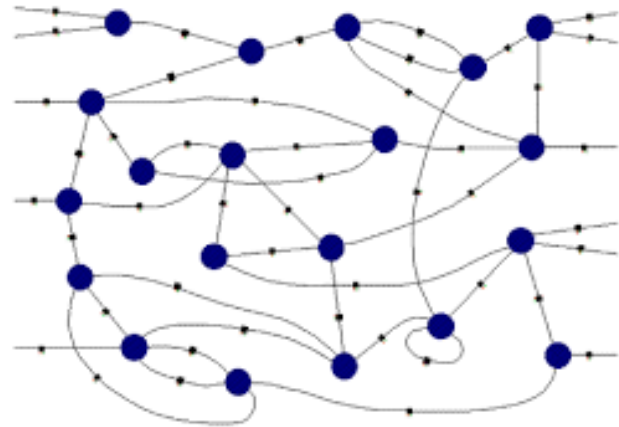
- Network of processing elements, called artificial neurons **neuron**
- The neurons are interconnected to form a network.
- Interconnection can be forward, recursive or self loop
- All world knowledge is stored in the connections between these elements.
- Neural networks are connectionist machines.



Connectionism Principles

Core Ideas:

- Intelligence emerges from simple processing neurons
- Knowledge is stored in connection weights
- Learning modifies connection strengths
- Parallel distributed processing

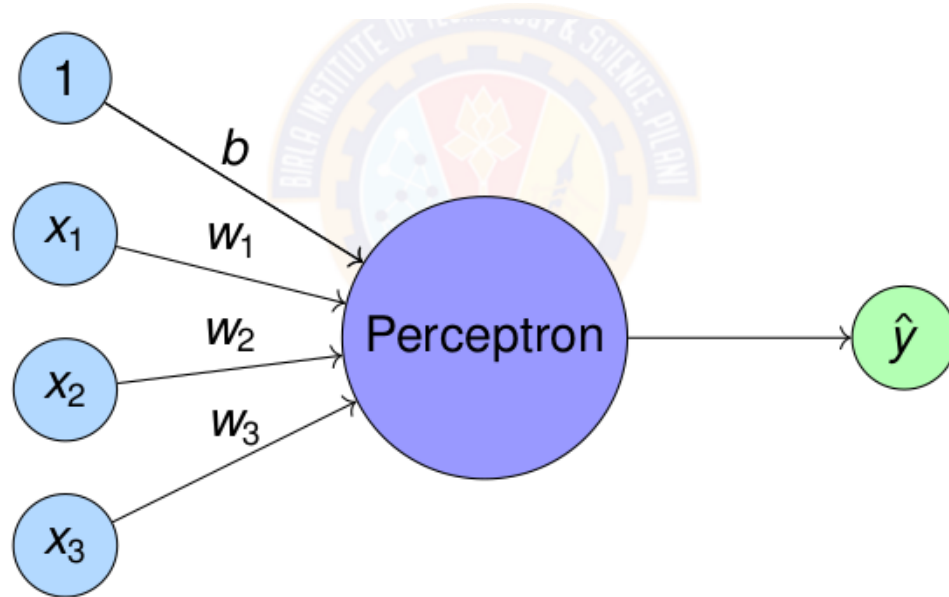




Perceptron

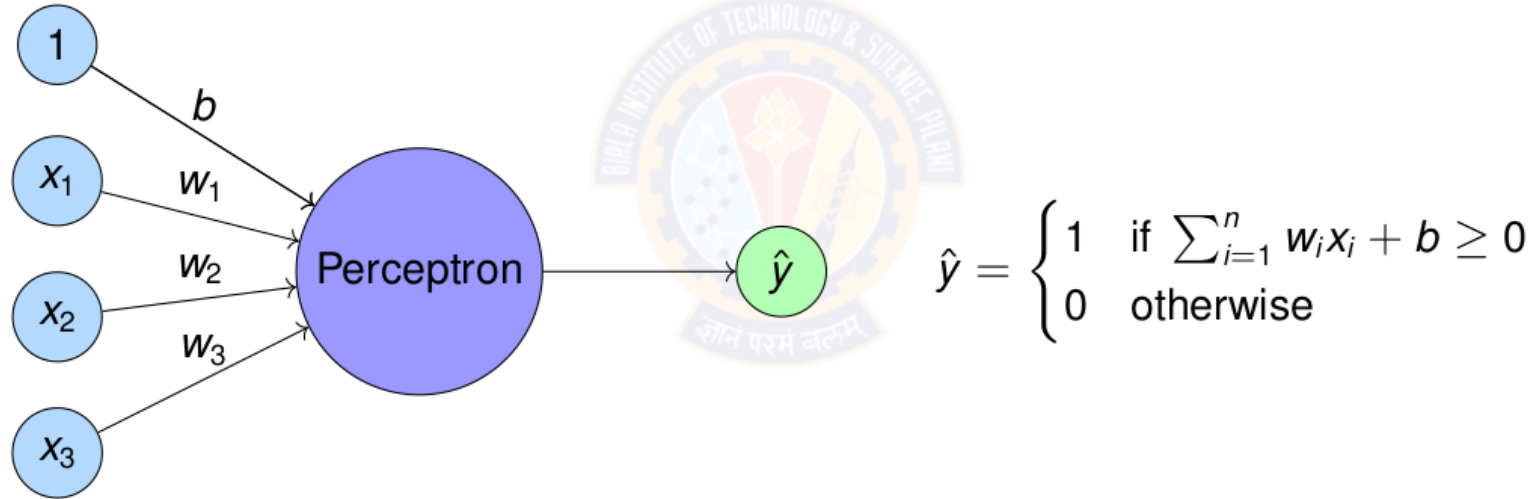
Perceptron

- Perceptron = Simplest form of artificial neuron
- Perceptron was introduced by Rosenblatt in 1958.



Perceptron

- Perceptron = Simplest form of artificial neuron

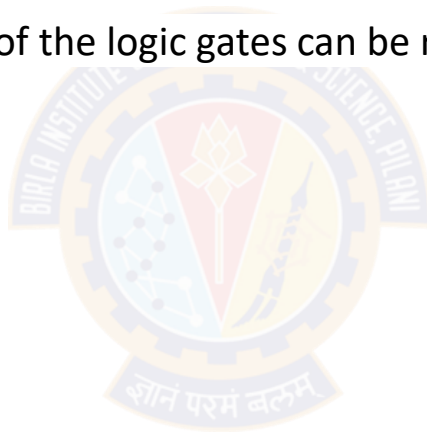




Perceptron & Logic Gates

Perceptron

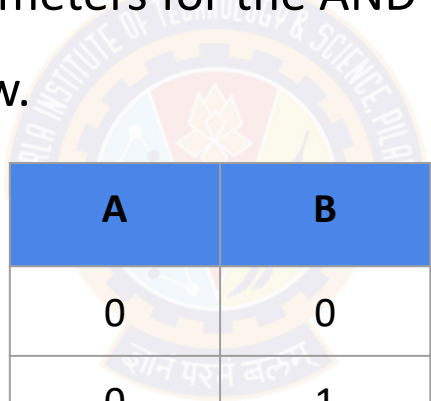
- Any linear decision can be represented by a Boolean equation.
- The quest is to find out how to represent the above using Perceptron.
- For this we test whether each of the logic gates can be represented by a Perceptron.



AND Logic Gate

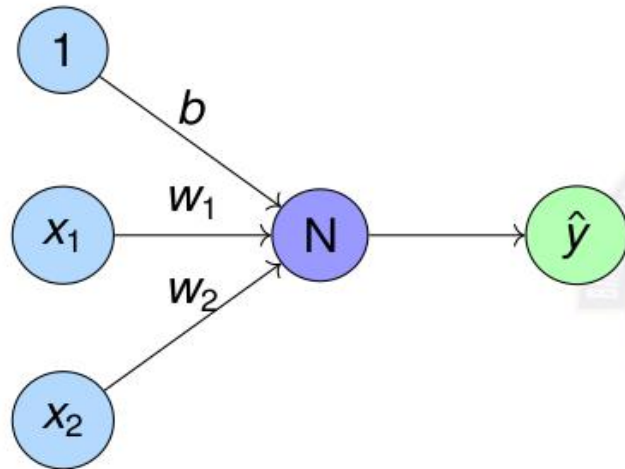
Question:

- How to represent AND gate using a perceptron?
- What are the parameters for the AND perceptron?
- Data is given below.



A	B	A and B
0	0	0
0	1	0
1	0	0
1	1	1

Perceptron for AND gate



Goal: Find parameters w_0, w_1, w_2

x_1	x_2	t	h
0	0	0	$w_0 + 0 + 0 < 0$
0	1	0	$w_0 + 0 + w_2 < 0$
1	0	0	$w_0 + w_1 + 0 < 0$
1	1	1	$w_0 + w_1 + w_2 \geq 0$

One solution is

Perceptron equation is

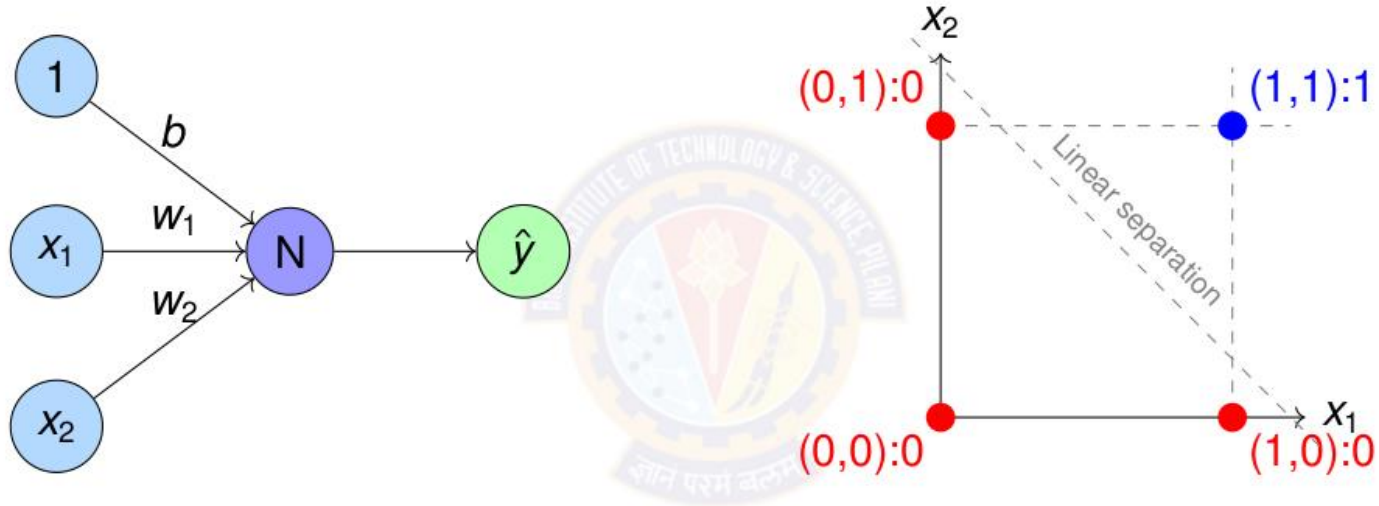
$$\hat{y} = w_0 x_0 + w_1 x_1 + w_2 x_2$$

$$w_0 = -1$$

$$w_1 = 0.75$$

$$w_2 = 0.75$$

Perceptron for AND gate



Perceptron equation is

$$\hat{y} = w_0x_0 + w_1x_1 + w_2x_2$$

One solution is

$$w_0 = -1$$

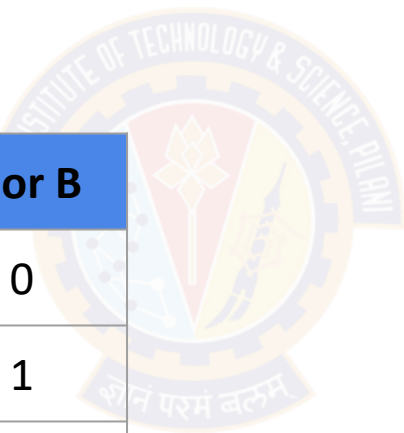
$$w_1 = 0.75$$

$$w_2 = 0.75$$

OR Logic Gate

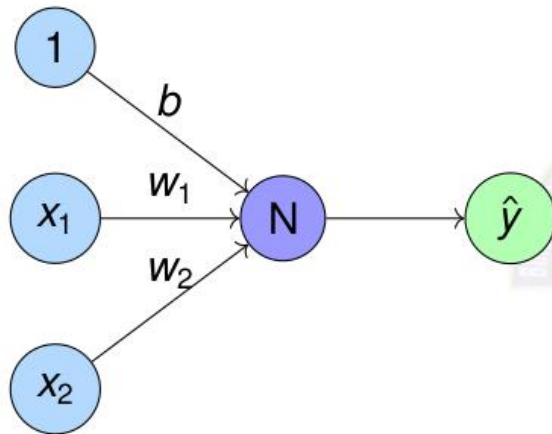
Question:

- How to represent OR gate using a perceptron?
- What are the parameters for the OR perceptron?
- Data is given below.



A	B	A or B
0	0	0
0	1	1
1	0	1
1	1	1

Perceptron for OR gate



Goal: Find parameters w_0, w_1, w_2

x_1	x_2	t	h
0	0	0	$w_0 + 0 + 0 < 0$
0	1	1	$w_0 + 0 + w_2 \geq 0$
1	0	1	$w_0 + w_1 + 0 \geq 0$
1	1	1	$w_0 + w_1 + w_2 \geq 0$

One solution is

Perceptron equation is

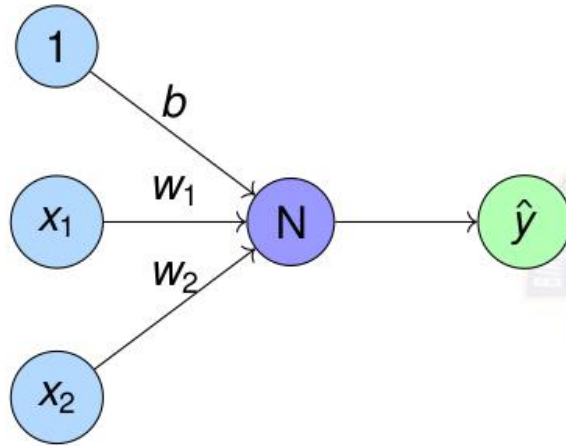
$$\hat{y} = w_0 x_0 + w_1 x_1 + w_2 x_2$$

$$w_0 = -1$$

$$w_1 = 2$$

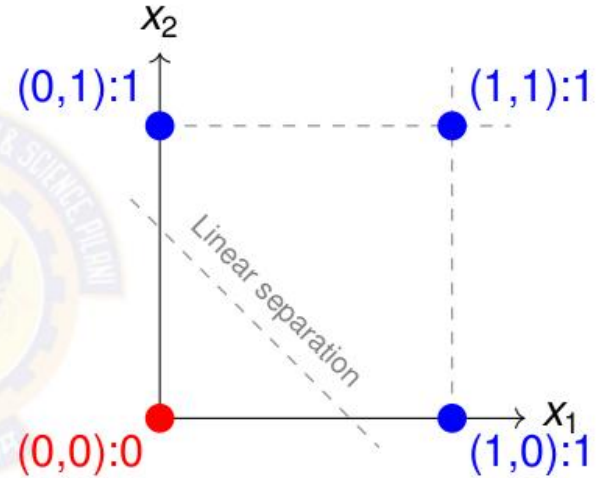
$$w_2 = 2$$

Perceptron for OR gate



Perceptron equation is

$$\hat{y} = w_0 x_0 + w_1 x_1 + w_2 x_2$$



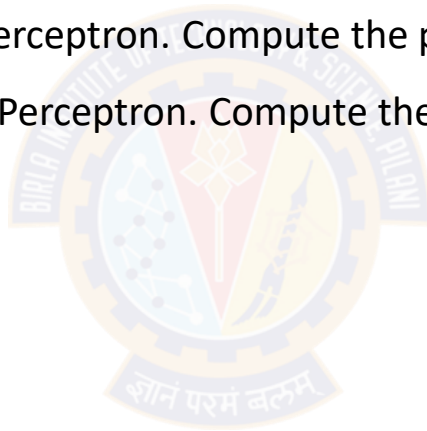
One solution is $w_0 = -1$

$$w_1 = 2$$

$$w_2 = 2$$

Exercise

1. Represent OR gate using Perceptron. Compute the parameters of the perceptron.
2. Represent NOR gate using Perceptron. Compute the parameters of the perceptron.
3. Represent NAND gate using Perceptron. Compute the parameters of the perceptron.





Perceptron Learning Algorithm

Perceptron Learning Algorithm

Goal: Find weights that correctly classify training data (input, output) pair (x, t) .

Algorithm:

- ① Initialize learning rate $\eta = 0.1$
- ② Initialize weights randomly: $w_i = \text{random}$
- ③ For each training example (x, t) :
 - ▶ Calculate output: $\hat{y} = \text{sign}(\sum_i w_i x_i + b)$
 - ▶ If $\hat{y} \neq t$, update weights:

$$w_i^{\text{new}} = w_i^{\text{old}} + \eta(t - \hat{y})x_i$$

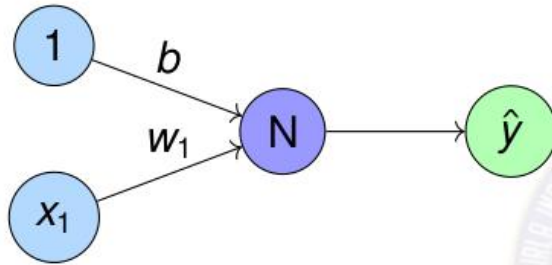
$$b^{\text{new}} = b^{\text{old}} + \eta(t - \hat{y})$$

- ④ Repeat until convergence or max iterations

Convergence Theorem:

If linearly separable data, Perceptron learning algorithm will converge in finite steps.

Perceptron Learning for NOT gate

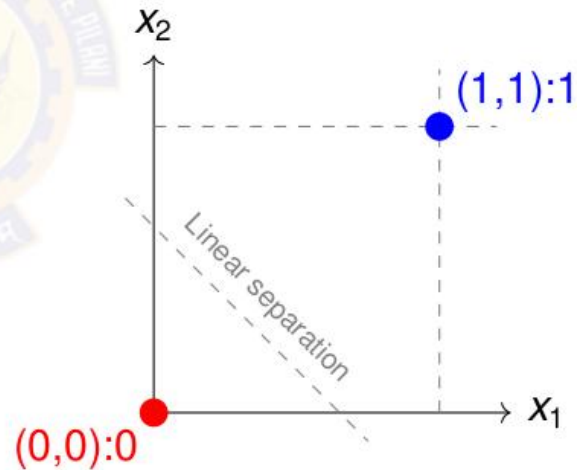


Example	x_1	t
Eg:1	-1	1
Eg:2	1	-1

Perceptron equation

$$h = \text{sign}(w_0x_0 + w_1x_1)$$

$$\hat{y} = \begin{cases} 1 & \text{if } h \geq 0 \\ -1 & \text{if } h < 0 \end{cases}$$



Perceptron Learning for NOT gate

Assume $w_0 = w_1 = 0$. Let the learning rate $= \eta = 1$.

- Epoch 1 Example 1 pair (1,-1)

$$\hat{y} = \text{sign}(w_1 x_1 + w_0) = \text{sign}(0 \cdot -1 + 0) = -1$$

Is $\hat{y}$ equal to t ? NO. Hence update weights.

$$w_i^{(new)} = w_i^{(old)} + \eta(t - \hat{y})x_i$$

$$w_1^{(new)} = w_1^{(old)} + \eta(t - \hat{y})x_1 = 0 + 1(1 - (-1)) \cdot (-1) = (-2)$$

$$w_0^{(new)} = w_0^{(old)} + \eta(t - \hat{y})x_0 = 0 + 1(1 - (-1)) = (2)$$

Perceptron Learning for NOT gate

- Epoch 1 Example 2 pair (1,-1)

$$\hat{y} = \text{sign}(w_1 x_1 + w_0) = \text{sign}((-2) \cdot 1 + 2) = \text{sign}(0) = -1$$

Is $\hat{y}$ equal to t ? YES. NO update weights.

- Epoch 2 Example 1 pair (-1,1)

$$\hat{y} = \text{sign}(w_1 x_1 + w_0) = \text{sign}((-2) \cdot (-1) + 2) = \text{sign}(4) = 1$$

Is $\hat{y}$ equal to t ? YES. NO update weights.

- Epoch 2 Example 2 pair (1,1)

$$\hat{y} = \text{sign}(w_1 x_1 + w_0) = \text{sign}((-2) \cdot 1 + 2) = \text{sign}(0) = -1$$

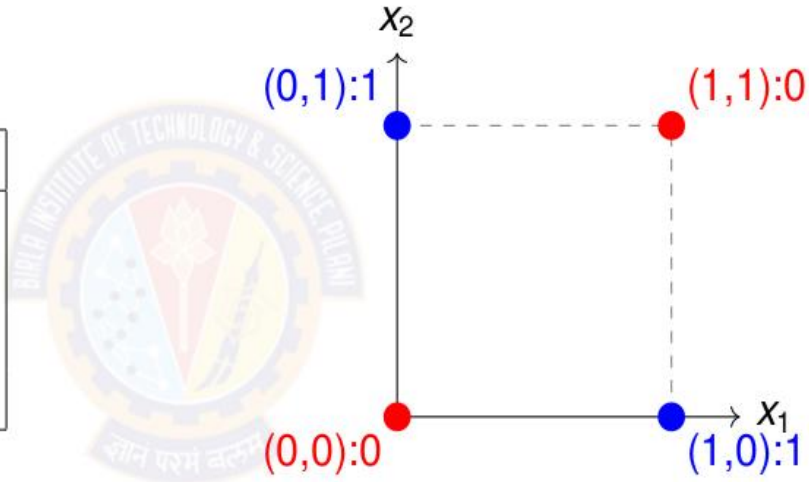
Is $\hat{y}$ equal to t ? YES. NO update weights.

- Algorithm Converges

The XOR Problem

Truth Table for XOR:

x_1	x_2	XOR Output
0	0	0
0	1	1
1	0	1
1	1	0



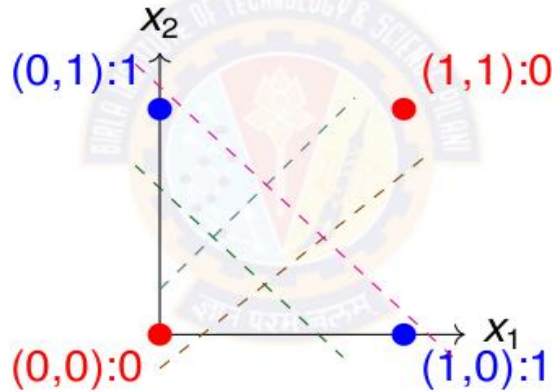
PROBLEM

XOR is not linearly separable! Single perceptron cannot solve XOR.

Why Single Perceptron Fails on XOR

Linear Decision Boundary Limitation:

Any line $w_1x_1 + w_2x_2 + b = 0$ cannot separate XOR classes.



No single line works!

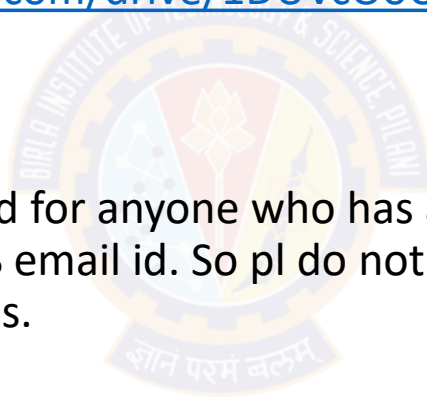
Solution: Need multiple layers (Multilayer Perceptron)

Perceptron Demo Python Code

<https://colab.research.google.com/drive/1DUVcOoUIWhl8GQKc6AWR1wi0LaMeNkgD?usp=sharing>

Student pl note:

The Python notebook is shared for anyone who has access to the link and the access is restricted to use BITS email id. So pl do not access from non-BITS email id and send requests for access.



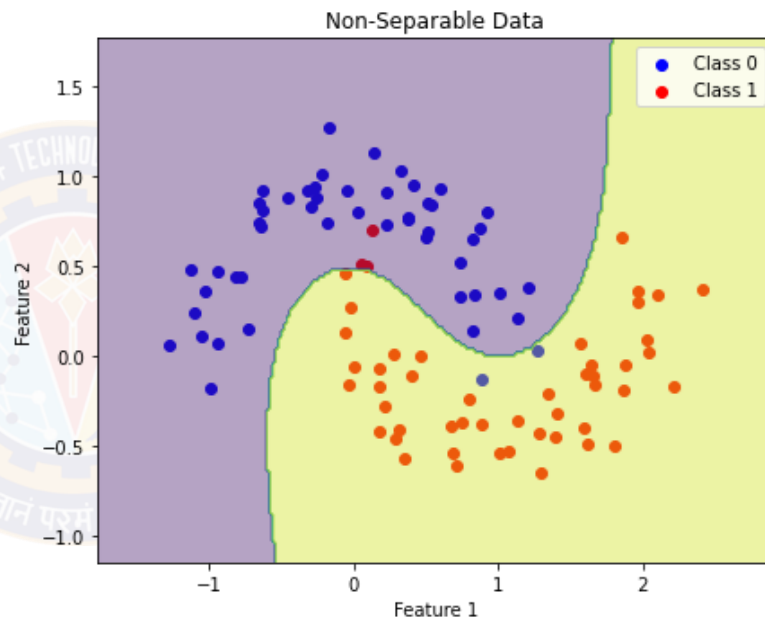
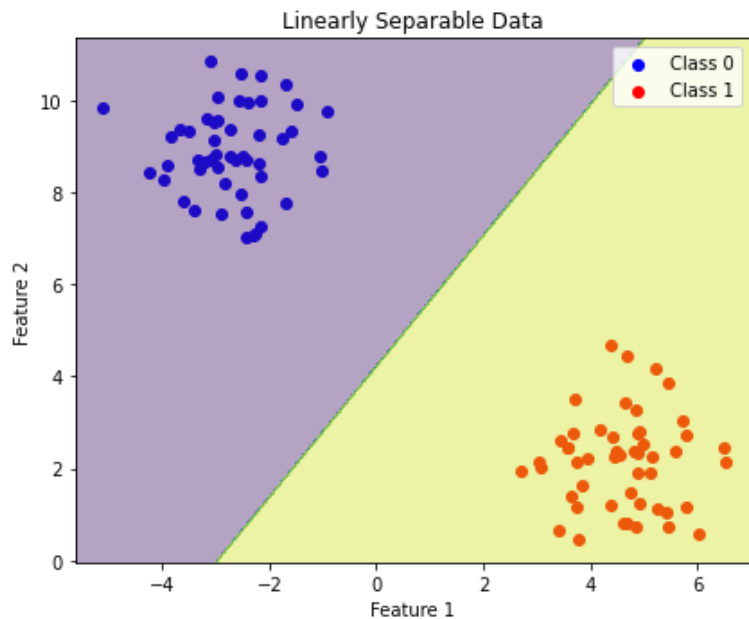


Linearly Separable Data

Linearly Separable Data

- **Two sets data points are separable in a n -dimensional space if they can be separated by an $(n-1)$ dimensional hyperplane.**
- A straight line can be drawn to separate all the data examples belonging to class +1 from all the examples belonging to the class -1. Then the two-dimensional data are clearly linearly separable.
- An infinite number of straight lines can be drawn to separate the class +1 from the class -1.

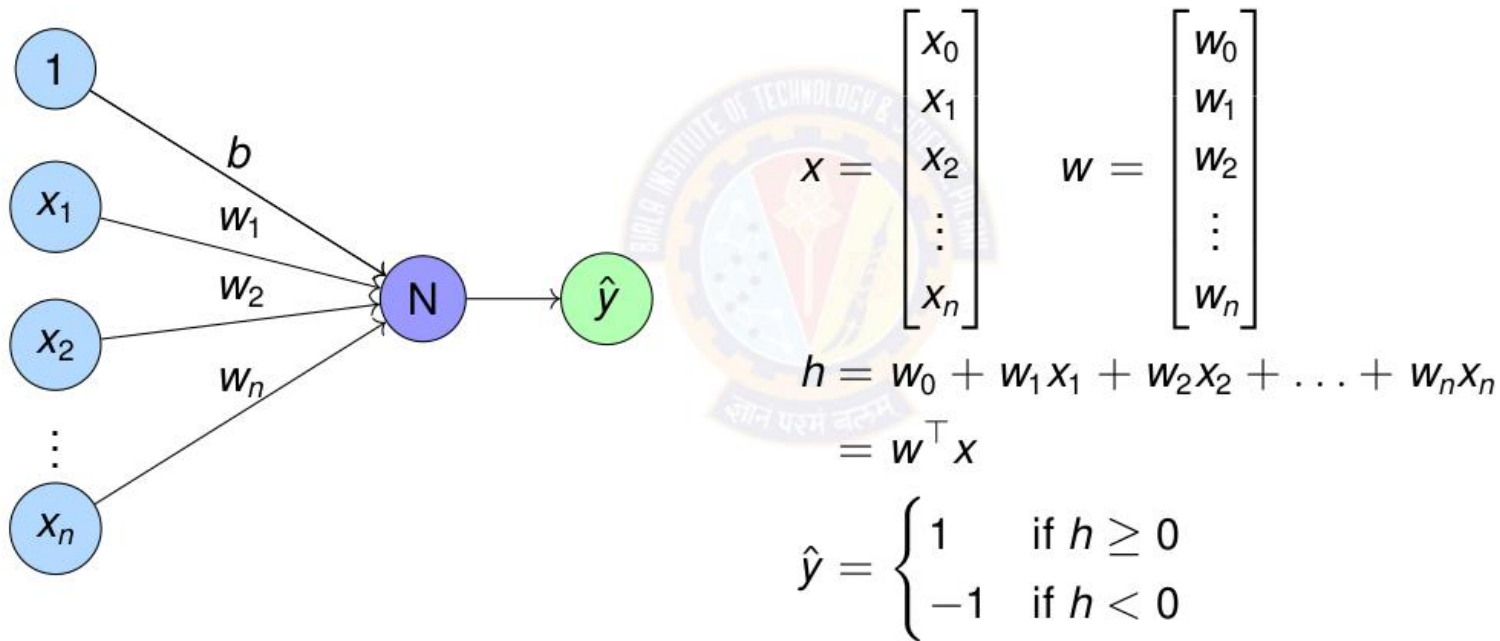
Linearly vs Non-Linearly Separable Data



Perceptron for Linearly Separable Data

- Data
 - Truth tables or set of examples
- Model
 - Perceptron
 - Inputs x are the features or columns
 - Target output t is the required label
- Objective Function
 - Deviation of desired output or target t and the computed output $\hat{y}$
- Learning Algorithm
 - Perceptron Learning algorithm
- Challenge
 - How to learn these $n + 1$ parameters $w_0, w_1, w_2, \dots, w_n$?
- Solution
 - Use Perceptron learning algorithm.

Perceptron for Linearly Separable Data

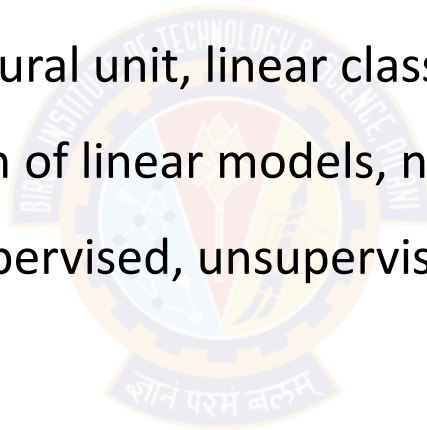


Exercise

- Represent OR gate using Perceptron. Compute the parameters of the perceptron using perceptron learning algorithm.
- Represent AND gate using Perceptron. Compute the parameters of the perceptron using perceptron learning algorithm.
- Represent NOR gate using Perceptron. Compute the parameters of the perceptron using perceptron learning algorithm.
- Represent NAND gate using Perceptron. Compute the parameters of the perceptron using perceptron learning algorithm.
- Implement a perceptron for AND and OR gates
- Sketch decision boundaries for Single perceptron

Key Concepts

- **Neural Network:** Biological inspiration and mathematical models
- **Perceptron:** Simplest neural unit, linear classification, learning algorithm
- **XOR Problem:** Limitation of linear models, need for hidden layers
- **Learning Paradigms:** Supervised, unsupervised, reinforcement learning



References

1. Zhang et al. "Dive into Deep Learning" - Chapter 1 <https://d2l.ai/>
2. http://mlsp.cs.cmu.edu/people/rsingh/docs/Chapter1_Introduction.pdf
3. Goodfellow, Bengio, Courville - "Deep Learning" Chapters 1-6
4. Rosenblatt, F. (1958) - "The Perceptron: A Probabilistic Model"
5. Minsky & Papert (1969) - "Perceptrons" (Historical perspective)



Thanks



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

DEEP NEURAL NETWORK MODULE # 3 : LINEAR NEURAL NETWORKS FOR REGRESSION

Seetha Parameswaran
BITS Pilani WILP

The instructor is gratefully acknowledging
the authors who made their course
materials freely available online.
This deck is prepared by Seetha Parameswaran.

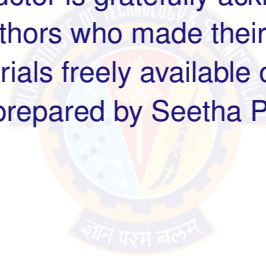


TABLE OF CONTENTS

- 1 REGRESSION
- 2 LINEAR REGRESSION
- 3 THE LINEAR MODEL: SINGLE NEURON
- 4 WORKED EXAMPLE
- 5 EVALUATION
- 6 PYTHON IMPLEMENTATION
- 7 IMPLEMENTATION TIPS
- 8 SUMMARY



WHAT IS REGRESSION?

DEFINITION

Regression is a supervised learning task that predicts a continuous-valued output based on input features.

Key Characteristics:

- Input: Feature vector $\mathbf{x} \in \mathbb{R}^d$
- Output: Real-valued target $y \in \mathbb{R}$
- Goal: Learn a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ such that $y \approx f(\mathbf{x})$

EXAMPLES OF REGRESSION PROBLEMS

① House Price Prediction

- ▶ Input: Size, location, number of rooms
- ▶ Output: Price in dollars

② Temperature Forecasting

- ▶ Input: Historical weather data, season, humidity
- ▶ Output: Temperature in degrees

③ Stock Price Prediction

- ▶ Input: Historical prices, trading volume, market indicators
- ▶ Output: Future stock price

TABLE OF CONTENTS

- 1 REGRESSION
- 2 LINEAR REGRESSION
- 3 THE LINEAR MODEL: SINGLE NEURON
- 4 WORKED EXAMPLE
- 5 EVALUATION
- 6 PYTHON IMPLEMENTATION
- 7 IMPLEMENTATION TIPS
- 8 SUMMARY



WHAT IS LINEAR REGRESSION?

DEFINITION

Linear regression assumes the output is a linear combination of the input features.

Mathematical Form:

$$y = w_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d \quad (1)$$

where:

- $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$ are the input features
- $\mathbf{w} = [w_1, w_2, \dots, w_d]^T$ are the weights
- w_0 is the bias term

EXAMPLES OF LINEAR REGRESSION

① Salary Prediction

▶ $\text{Salary} = w_0 + w_1 \times \text{Years of Experience}$

② Crop Yield Prediction

▶ $\text{Yield} = w_0 + w_1 \times \text{Rainfall} + w_2 \times \text{Fertilizer}$

③ Energy Consumption

▶ $\text{Energy} = w_0 + w_1 \times \text{Temperature} + w_2 \times \text{Hour} + w_3 \times \text{Day}$

The relationship between inputs and output is assumed to be **linear**.

VISUAL EXAMPLE: 2D LINEAR REGRESSION

2D Linear Regression: Scatter Plot with Fitted Line

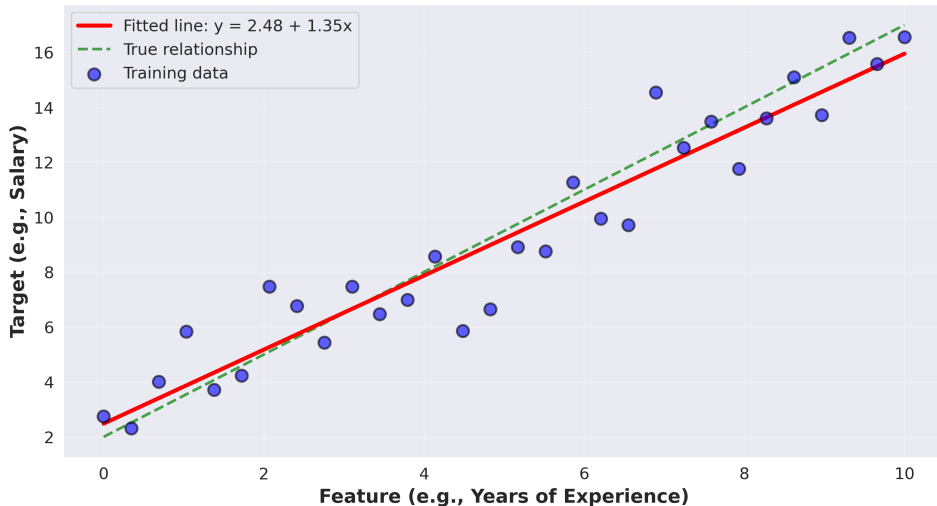


TABLE OF CONTENTS

- 1 REGRESSION
- 2 LINEAR REGRESSION
- 3 THE LINEAR MODEL: SINGLE NEURON**
- 4 WORKED EXAMPLE
- 5 EVALUATION
- 6 PYTHON IMPLEMENTATION
- 7 IMPLEMENTATION TIPS
- 8 SUMMARY



THE FOUR COMPONENTS

A complete machine learning system consists of:

- ① **Data:** d -dimensional input vectors and target outputs
- ② **Model:** Single neuron implementing linear function
- ③ **Objective Function:** Measures prediction error
- ④ **Learning Algorithm:** Gradient descent to optimize weights

MATRIX FORM OF DATA

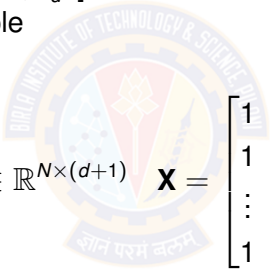
For efficient computation, we organize data into matrices:

Input Data = Augmented Features:

Add bias term: $\tilde{\mathbf{x}}^{(i)} = [1, x_1^{(i)}, \dots, x_d^{(i)}]^T$

Each row = one training example

Design Matrix: $\mathbf{X} \in \mathbb{R}^{N \times (d+1)}$ $\mathbf{X} =$

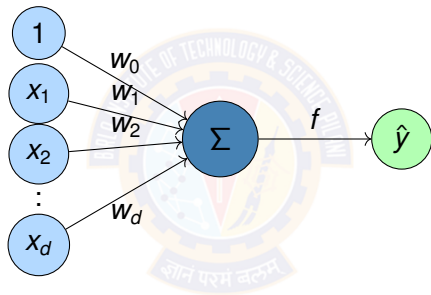

$$\begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ 1 & x_1^{(2)} & \dots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(N)} & \dots & x_d^{(N)} \end{bmatrix}$$

Target Vector: $\mathbf{y} \in \mathbb{R}^N$ $\mathbf{y} = [y^{(1)} \quad y^{(2)} \quad \dots \quad y^{(N)}]^T$

Weight Vector: $\mathbf{w} \in \mathbb{R}^{d+1}$ $\mathbf{w} = [w_0 \quad w_1 \quad w_2 \quad \dots \quad w_d]^T$

LINEAR MODEL AS A SINGLE NEURON

Model linear regression as a **single artificial neuron**:



- **Input:** d -dimensional feature vector $\mathbf{x} \in \mathbb{R}^d$
- **Output:** Single predicted value $\hat{y} \in \mathbb{R}$

LINEAR MODEL PREDICTION

Model: A single neuron computes a weighted sum and applied identity activation function.

Prediction for input $\mathbf{x}$:

$$\hat{y} = f(\mathbf{w}^T \mathbf{x}) = f(w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d) \quad (2)$$

where:

- $\mathbf{w} = [w_0, w_1, \dots, w_d]^T \in \mathbb{R}^{d+1}$ are the model parameters
- $\mathbf{x} = [1, x_1, \dots, x_d]^T \in \mathbb{R}^{d+1}$ is the input
- $f(\cdot)$ is the **identity activation** $f(z) = z$

Vectorized form: for all predictions

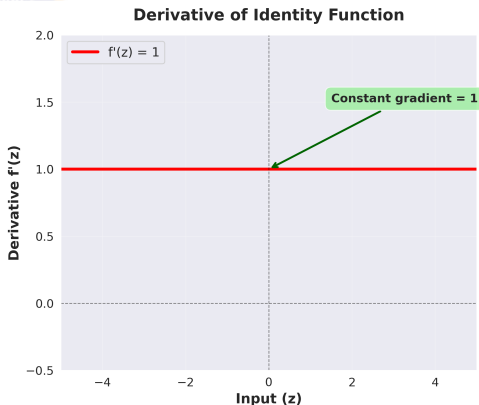
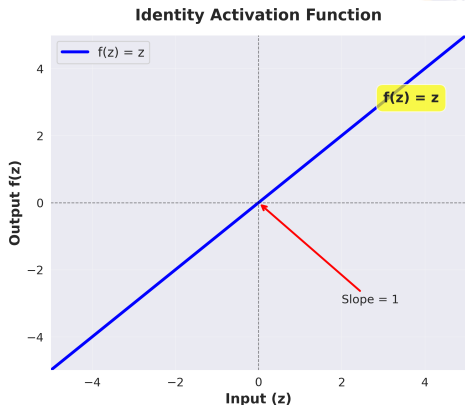
$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} \quad (3)$$

IDENTITY ACTIVATION FUNCTION

Identity Activation Function

$$f(z) = z$$

(4)



IDENTITY ACTIVATION FUNCTION

Properties of Identity Function

- Output: Continuous, $(-\infty, \infty)$
- Differentiable everywhere: $f'(z) = 1$
- Allows output to take any real value
- Gradient flows unchanged through the neuron
- Perfect for predicting continuous targets

Why identity for regression?

- Target values are continuous (prices, temperatures, etc.)
- Need to predict any real number, not just binary $\{ 0,1 \}$
- Differentiability enables gradient-based optimization
- No information loss from input to output

OBJECTIVE FUNCTION: SQUARED ERROR LOSS

We need to measure how well our model fits the data.

Loss for single example:

$$\ell(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) = \frac{1}{2} (\hat{y}^{(i)} - y^{(i)})^2 = \frac{1}{2} (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)})^2 \quad (5)$$

Total Loss (Mean Squared Error):

$$J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \frac{1}{2} (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)})^2 \quad (6)$$

Goal: Find $\mathbf{w}^*$ that minimizes $J(\mathbf{w})$

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} J(\mathbf{w}) \quad (7)$$

WHY SQUARED ERROR?

Advantages of squared error loss:

- **Differentiable:** Smooth everywhere, easy to optimize
- **Convex:** Single global minimum (for linear models)
- **Penalizes large errors:** Quadratic growth
- **Statistical interpretation:** Maximum likelihood under Gaussian noise

Vector Form:

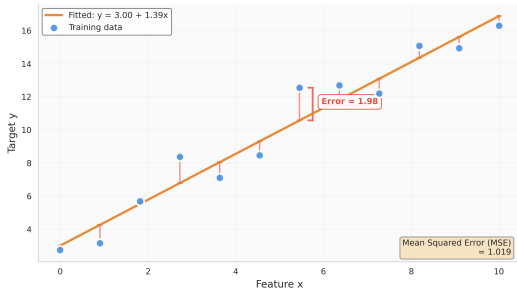
$$J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = \frac{1}{2N} (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) \quad (8)$$

where $\mathbf{X} \in \mathbb{R}^{N \times (d+1)}$ is the design matrix and $\mathbf{y} \in \mathbb{R}^N$ is the target vector.

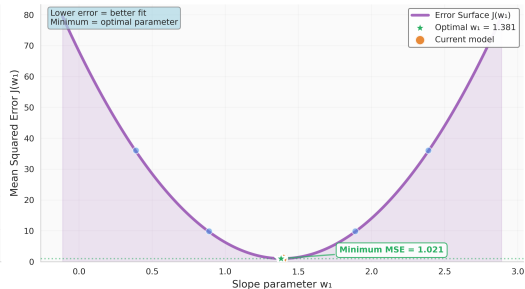
VISUALIZING THE ERROR SURFACE

Understanding Error Surface: From Individual Errors to Loss Function

(a) Linear Regression with Prediction Errors
Vertical lines show error for each data point



(b) 1D Error Surface
How error changes with slope parameter

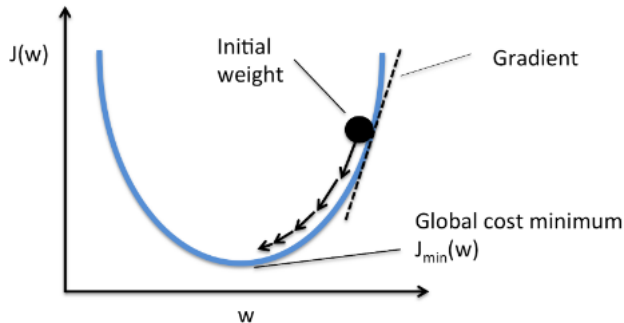


← Sum of all squared vertical errors (left) creates the parabolic error surface (right) →

The error surface is **convex** with a single global minimum.

GRADIENT DESCENT ALGORITHM

Idea: Iteratively update weights in the direction of steepest descent



where $\eta > 0$ is the **learning rate** (step size).

Intuition: Move downhill on the error surface to find the minimum.

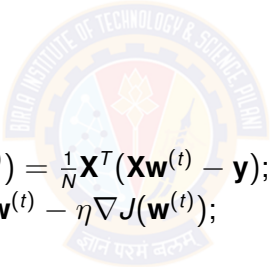
BATCH GRADIENT DESCENT ALGORITHM

Algorithm 1: Gradient Descent for Linear Regression

Input: Dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$, learning rate η , tolerance ϵ

Output: Learned weights $\mathbf{w}$

```
1 Initialize  $\mathbf{w}^{(0)} = \mathbf{0}$  (or random);
2  $t \leftarrow 0$ ;
3 while not converged do
4     Compute gradient:  $\nabla J(\mathbf{w}^{(t)}) = \frac{1}{N} \mathbf{X}^T (\mathbf{X} \mathbf{w}^{(t)} - \mathbf{y})$ ;
5     Update weights:  $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla J(\mathbf{w}^{(t)})$ ;
6     if  $\|\nabla J(\mathbf{w}^{(t+1)})\| < \epsilon$  then
7         break // Converged
8      $t \leftarrow t + 1$ ;
9 return  $\mathbf{w}^{(t+1)}$ ;
```



COMPUTING THE GRADIENT

Gradient of squared error loss:

$$\nabla J(\mathbf{w}) = \frac{\partial J}{\partial \mathbf{w}} = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)}) \mathbf{x}^{(i)} \quad (9)$$

Matrix Form:

$$\nabla J(\mathbf{w}) = \frac{1}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y}) \quad (10)$$

where $\mathbf{X}^T \in \mathbb{R}^{(d+1) \times N}$, $\mathbf{X}\mathbf{w} - \mathbf{y} \in \mathbb{R}^N$, so $\nabla J \in \mathbb{R}^{d+1}$

GRADIENT DESCENT UPDATE RULE

Update equation:

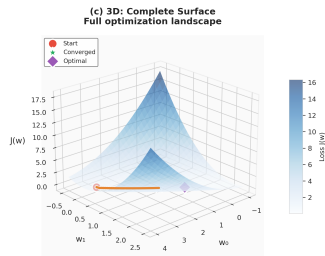
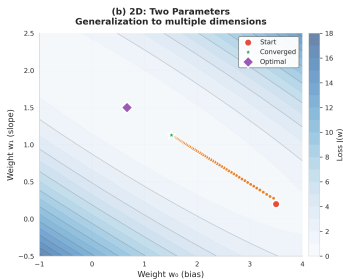
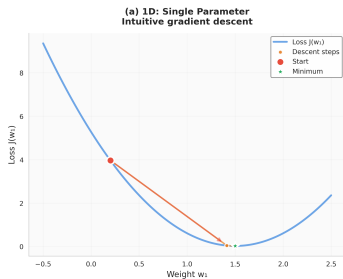
$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \frac{\eta}{N} \mathbf{X}^T (\mathbf{X} \mathbf{w}^{(t)} - \mathbf{y}) \quad (11)$$

Key Parameters:

- Learning rate η : Controls step size
 - ▶ Too large: May overshoot minimum
 - ▶ Too small: Slow convergence
- Stopping criterion: $\|\nabla J(\mathbf{w})\| < \epsilon$ or max iterations

VISUALIZING GRADIENT DESCENT

Understanding Gradient Descent: Progressive Visualization from 1D to 3D



Each step moves perpendicular to contour lines toward the minimum.

TABLE OF CONTENTS

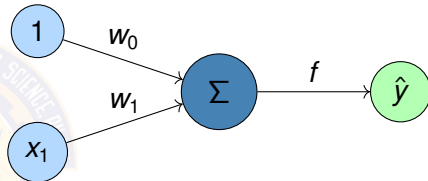
- 1 REGRESSION
- 2 LINEAR REGRESSION
- 3 THE LINEAR MODEL: SINGLE NEURON
- 4 WORKED EXAMPLE**
- 5 EVALUATION
- 6 PYTHON IMPLEMENTATION
- 7 IMPLEMENTATION TIPS
- 8 SUMMARY



NUMERICAL EXAMPLE: SETUP

Tiny Dataset: Predict house price from size

Size (x_1)	Price (y)
1	2
2	4
3	5



Matrix Formulation:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \in \mathbb{R}^{3 \times 2}, \quad \mathbf{y} = \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} \in \mathbb{R}^3, \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} \in \mathbb{R}^2$$

NUMERICAL EXAMPLE: INITIALIZATION

Hyperparameters: $\eta = 0.1$ $N = 3$

Initialize weights: $\mathbf{w}^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

Vectorized Prediction: $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$

Initial predictions: $\hat{\mathbf{y}}^{(0)} = \mathbf{X}\mathbf{w}^{(0)} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$

Loss Function: $J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$

Initial loss: $J(\mathbf{w}^{(0)}) = \frac{1}{6} \left\| \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} \right\|^2 = \frac{1}{6}(4 + 16 + 25) = 7.5$

NUMERICAL EXAMPLE: COMPUTING GRADIENT

Error vector: $\mathbf{e}^{(0)} = \hat{\mathbf{y}}^{(0)} - \mathbf{y} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} = \begin{bmatrix} -2 \\ -4 \\ -5 \end{bmatrix}$

Gradient computation: $\nabla J(\mathbf{w}^{(0)}) = \frac{1}{N} \mathbf{X}^T \mathbf{e}^{(0)}$

$$\begin{aligned} \nabla J(\mathbf{w}^{(0)}) &= \frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} -2 \\ -4 \\ -5 \end{bmatrix} \\ &= \frac{1}{3} \begin{bmatrix} -2 - 4 - 5 \\ -2 - 8 - 15 \end{bmatrix} = \begin{bmatrix} -3.67 \\ -8.33 \end{bmatrix} \end{aligned}$$

NUMERICAL EXAMPLE: WEIGHT UPDATE

Gradient descent update: $\mathbf{w}^{(1)} = \mathbf{w}^{(0)} - \eta \nabla J(\mathbf{w}^{(0)})$

Computing new weights: $\mathbf{w}^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} -3.67 \\ -8.33 \end{bmatrix} = \begin{bmatrix} 0.367 \\ 0.833 \end{bmatrix}$

New predictions: $\hat{\mathbf{y}}^{(1)} = \mathbf{X}\mathbf{w}^{(1)}$

$$\hat{\mathbf{y}}^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 0.367 \\ 0.833 \end{bmatrix} = \begin{bmatrix} 1.20 \\ 2.03 \\ 2.87 \end{bmatrix}$$

NUMERICAL EXAMPLE: NEW LOSS

New error vector: $\mathbf{e}^{(1)} = \hat{\mathbf{y}}^{(1)} - \mathbf{y} = \begin{bmatrix} 1.20 \\ 2.03 \\ 2.87 \end{bmatrix} - \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} = \begin{bmatrix} -0.80 \\ -1.97 \\ -2.13 \end{bmatrix}$

Loss after one iteration: $J(\mathbf{w}^{(1)}) = \frac{1}{6} \|\mathbf{e}^{(1)}\|^2 = \frac{1}{6} (0.64 + 3.88 + 4.54) \approx 1.51$

Progress:

- Initial loss: $J(\mathbf{w}^{(0)}) = 7.5$
- After 1 iteration: $J(\mathbf{w}^{(1)}) = 1.51$
- **80% reduction!** ✓

Continue until $\|\nabla J(\mathbf{w})\| < \epsilon$ or max iterations reached

COMPUTATIONAL GRAPH FOR ONE GRADIENT

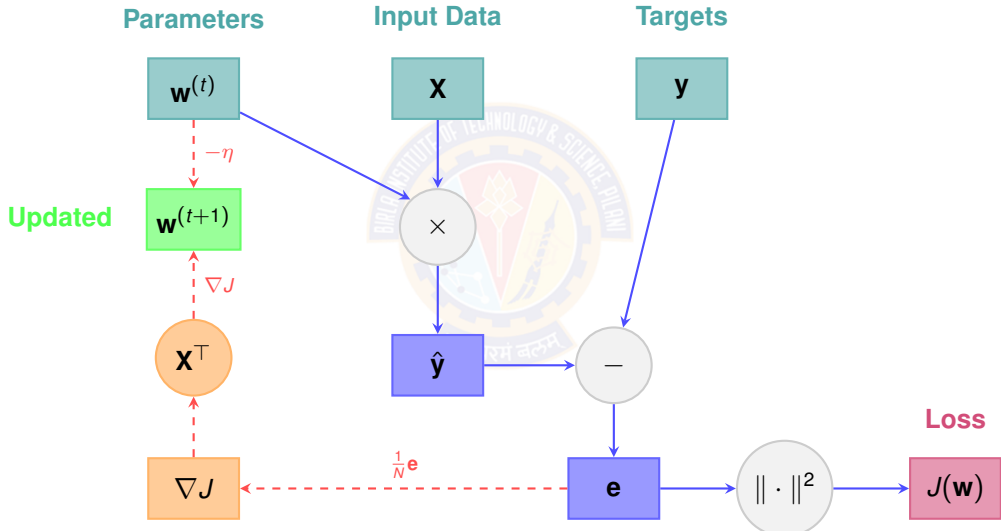


TABLE OF CONTENTS

- 1 REGRESSION
- 2 LINEAR REGRESSION
- 3 THE LINEAR MODEL: SINGLE NEURON
- 4 WORKED EXAMPLE
- 5 EVALUATION**
- 6 PYTHON IMPLEMENTATION
- 7 IMPLEMENTATION TIPS
- 8 SUMMARY



MODEL EVALUATION: TRAINING VS TEST ERROR

Key Principle: Evaluate on unseen data

Data Split:

- Training set: Used to learn weights $\mathbf{w}$ (typically 90-99%)
- Test set: Used to evaluate generalization (typically 1-10%)

Training Error:
$$J_{\text{train}} = \frac{1}{N_{\text{train}}} \sum_{i \in \text{train}} (\hat{y}^{(i)} - y^{(i)})^2$$

Test Error:
$$J_{\text{test}} = \frac{1}{N_{\text{test}}} \sum_{i \in \text{test}} (\hat{y}^{(i)} - y^{(i)})^2$$

Goal: Minimize test error (generalization performance)

EVALUATION METRICS

1. Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})^2 \quad (12)$$

2. Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})^2} \quad (13)$$

3. Mean Absolute Error (MAE): (Less sensitive to outliers than MSE)

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y^{(i)} - \hat{y}^{(i)}| \quad (14)$$

R^2 SCORE (COEFFICIENT OF DETERMINATION)

Definition: Proportion of variance explained by the model

$$R^2 = 1 - \frac{\sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)})^2}{\sum_{i=1}^N (y^{(i)} - \bar{y})^2} = 1 - \frac{SS_{\text{residual}}}{SS_{\text{total}}} \quad (15)$$

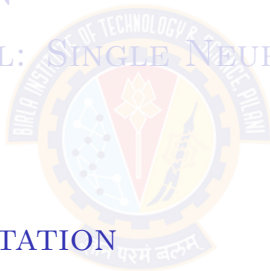
where $\bar{y} = \frac{1}{N} \sum_{i=1}^N y^{(i)}$ is the mean.

Interpretation:

- $R^2 = 1$: Perfect fit (all variance explained)
- $R^2 = 0$: Model no better than predicting mean
- $R^2 < 0$: Model worse than baseline (possible on test set)
- Typical: $0.7 < R^2 < 0.9$ indicates good fit

TABLE OF CONTENTS

- 1 REGRESSION
- 2 LINEAR REGRESSION
- 3 THE LINEAR MODEL: SINGLE NEURON
- 4 WORKED EXAMPLE
- 5 EVALUATION
- 6 PYTHON IMPLEMENTATION**
- 7 IMPLEMENTATION TIPS
- 8 SUMMARY



PERCEPTRON DEMO PYTHON CODE

- <https://colab.research.google.com/drive/10whoZYodz1H02mbVwsmTudwxaU18ukaI?usp=sharing>

Student pl note:

The Python notebook is shared for anyone who has access to the link and the access is restricted to **use BITS email id**. So please do not access from non-BITS email id and send requests for access. Access for non-BITS email id will NOT be granted.

TABLE OF CONTENTS

- 1 REGRESSION
- 2 LINEAR REGRESSION
- 3 THE LINEAR MODEL: SINGLE NEURON
- 4 WORKED EXAMPLE
- 5 EVALUATION
- 6 PYTHON IMPLEMENTATION
- 7 IMPLEMENTATION TIPS**
- 8 SUMMARY



TIPS FOR IMPLEMENTATION

1. Choosing Learning Rate η :

- Start with $\eta \in \{0.001, 0.01, 0.1, 1.0\}$
- Plot loss vs iterations for each
- If loss increases or becomes NaN $\rightarrow$ decrease η
- If convergence too slow $\rightarrow$ increase η
- Use learning rate schedules for fine-tuning

2. When to Stop Training:

- Gradient magnitude: $\|\nabla J\| < \epsilon$ (e.g., $\epsilon = 10^{-4}$)
- Loss change: $|J^{(t)} - J^{(t-1)}| < \epsilon$
- Maximum iterations reached (e.g., $T = 1000$ or $10,000$)
- Validation loss starts increasing (early stopping for overfitting)

MORE PRACTICAL TIPS

3. Weight Initialization:

- Zero initialization: $\mathbf{w}^{(0)} = \mathbf{0}$ (works well for linear regression)
- Avoid large initial values (can cause divergence or overflow)

4. Feature Engineering:

- Domain knowledge: Create meaningful derived features (e.g., price per sq ft)
- Feature selection: Remove irrelevant/redundant features

5. Feature Scaling and Normalization:

- Features with different scales can slow convergence
- 1. Min-Max Scaling: Scale to $[0, 1]$ $x'_j = \frac{x_j - \min(x_j)}{\max(x_j) - \min(x_j)}$
- 2. Standardization (Z-score): Zero mean, unit variance $x'_j = \frac{x_j - \mu_j}{\sigma_j}$

DEBUGGING CHECKLIST

Common Issues and Solutions:

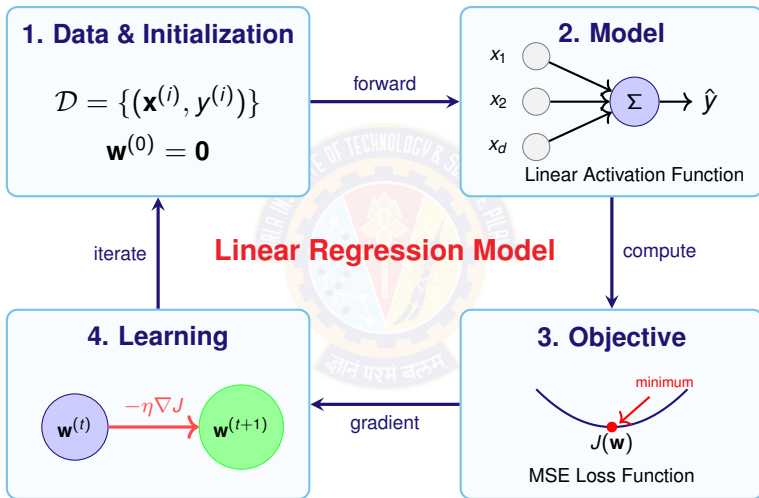
- Loss is NaN/Inf:
 - ▶ Learning rate too large
 - ▶ Numerical overflow (check feature scales)
 - ▶ Solution: Decrease η , scale features
- Training works but test error high:
 - ▶ Overfitting (too many features relative to data)
 - ▶ Solution: Regularization, more data, or simpler model
- Loss oscillating:
 - ▶ Learning rate too large
 - ▶ Solution: Decrease η or use learning rate decay
- Loss not decreasing:
 - ▶ Learning rate too small
 - ▶ Bug in gradient computation (check signs!)
 - ▶ Wrong sign in update rule (should be minus)
 - ▶ Solution: Increase η , verify code

TABLE OF CONTENTS

- 1 REGRESSION
- 2 LINEAR REGRESSION
- 3 THE LINEAR MODEL: SINGLE NEURON
- 4 WORKED EXAMPLE
- 5 EVALUATION
- 6 PYTHON IMPLEMENTATION
- 7 IMPLEMENTATION TIPS
- 8 SUMMARY



SUMMARY: PUTTING IT ALL TOGETHER



This framework extends to more complex models: neural networks, deep learning!

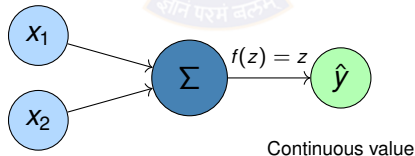
KEY TAKEAWAYS

- Regression predicts continuous outputs from input features.
- Linear regression assumes a linear relationship: $y = \mathbf{w}^T \mathbf{x} + w_0$
- A single neuron with identity activation implements linear regression.
- Squared error loss provides a convex objective function.
- Gradient descent iteratively optimizes weights: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J$
- Feature scaling is critical for fast convergence.
- Evaluation on test set ensures generalization.

LINEAR REGRESSION

Key insight: Linear relationship between inputs and output, optimized by minimizing squared errors.

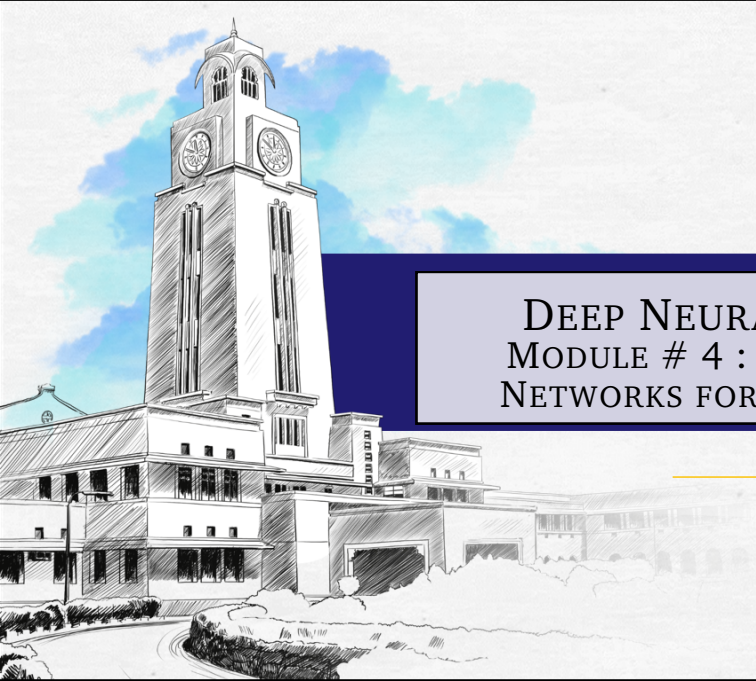
- **Problem:** Predict continuous values: $y \in \mathbb{R}$
- **Model:** Single neuron with identity activation
- **Output:** Predicted value $\hat{y} = \mathbf{w}^T \mathbf{x} \in (-\infty, \infty)$
- **Loss:** Mean Squared Error (MSE)
- **Training:** Batch Gradient Descent (GD)



REFERENCES

- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into Deep Learning*. Cambridge University Press. Chapter 3 (Linear Neural Networks for Regression). <https://d2l.ai/>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Chapter 5 (Machine Learning Basics). <https://www.deeplearningbook.org/>
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. Chapter 3 (Linear Models for Regression).
- Murphy, K. P. (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press. Chapter 11 (Linear Regression). <https://probml.github.io/pml-book/>

Thank You!



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

DEEP NEURAL NETWORK MODULE # 4 : LINEAR NEURAL NETWORKS FOR CLASSIFICATION

Seetha Parameswaran
BITS Pilani WILP

The instructor is gratefully acknowledging
the authors who made their course
materials freely available online.
This deck is prepared by Seetha Parameswaran.

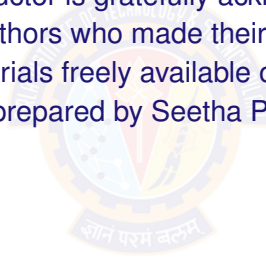


TABLE OF CONTENTS

1 CLASSIFICATION

2 BINARY CLASSIFICATION

- Logistic Regression
- The Model: Single Neuron for Binary Classification
- Binary Classification: Worked Example
- Binary Classification: Evaluation Metrics

3 MULTI-CLASS CLASSIFICATION

- Multi-class Classification Model
- Mini-batch Stochastic Gradient Descent
- Multiclass Classification: Worked Example
- Making Predictions (Inference)
- Multiclass Classification: Evaluation Metrics

4 IMPLEMENTATION TIPS AND DEBUGGING

5 SUMMARY

WHAT IS CLASSIFICATION?

DEFINITION

Classification is a supervised learning task that predicts a discrete category or class label based on input features.

Key Characteristics:

- Input: Feature vector $\mathbf{x} \in \mathbb{R}^d$
- Output: Class label $y \in \{1, 2, \dots, K\}$ (discrete)
- Goal: Learn a function $f : \mathbb{R}^d \rightarrow \{1, 2, \dots, K\}$ that assigns inputs to classes

Difference from Regression:

- Regression: Predicts **continuous** values (e.g., price, temperature)
- Classification: Predicts **discrete** categories (e.g., spam/not spam, cat/dog)

EXAMPLES OF CLASSIFICATION PROBLEMS

① Email Spam Detection

- ▶ Input: Email text, sender information, attachments
- ▶ Output: Spam or Not Spam

② Medical Diagnosis

- ▶ Input: Patient symptoms, test results, medical history
- ▶ Output: Disease present or absent

③ Image Recognition

- ▶ Input: Pixel values of an image
- ▶ Output: Object category (cat, dog, car, etc.)

④ Sentiment Analysis

- ▶ Input: Customer review text
- ▶ Output: Positive, Negative, or Neutral

TYPES OF CLASSIFICATION

1. Binary Classification

- Two classes: $y \in \{0, 1\}$ or $\{-1, +1\}$
- Examples: Spam detection, disease diagnosis, fraud detection

2. Multi-class Classification

- Multiple classes: $y \in \{1, 2, \dots, K\}$ where $K > 2$
- Examples: Digit recognition (0-9), animal classification
- Each example belongs to exactly one class

3. Multi-label Classification

- Each example can belong to multiple classes simultaneously
- Examples: Movie genres (action + comedy), document tagging

TABLE OF CONTENTS

1 CLASSIFICATION

2 BINARY CLASSIFICATION

- Logistic Regression
- The Model: Single Neuron for Binary Classification
- Binary Classification: Worked Example
- Binary Classification: Evaluation Metrics

3 MULTI-CLASS CLASSIFICATION

- Multi-class Classification Model
- Mini-batch Stochastic Gradient Descent
- Multiclass Classification: Worked Example
- Making Predictions (Inference)
- Multiclass Classification: Evaluation Metrics

4 IMPLEMENTATION TIPS AND DEBUGGING

5 SUMMARY

BINARY CLASSIFICATION PROBLEM

DEFINITION

Binary classification assigns each input to one of two classes.

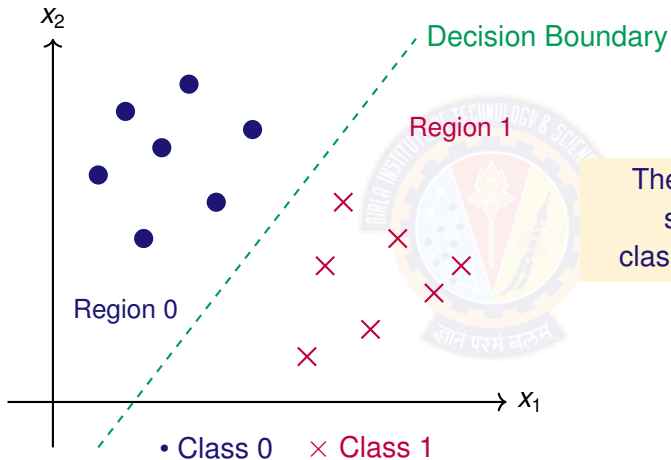
Standard Notation:

- Classes: $y \in \{0, 1\}$
 - ▶ $y = 1$: Positive class (e.g., spam, disease present)
 - ▶ $y = 0$: Negative class (e.g., not spam, disease absent)
- Dataset: $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$
- Goal: Learn $f(\mathbf{x})$ that outputs probability of class 1

Decision Boundary:

- Predict $\hat{y} = 1$ if $f(\mathbf{x}) \geq 0.5$
- Predict $\hat{y} = 0$ if $f(\mathbf{x}) < 0.5$

BINARY CLASSIFICATION: VISUAL EXAMPLE

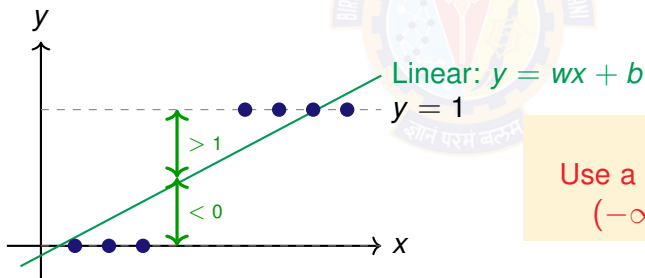


The decision boundary separates the two classes in feature space.

WHY NOT USE LINEAR REGRESSION?

Problem with Linear Regression for Classification:

- Linear regression outputs: $\hat{y} \in (-\infty, \infty)$
- We need probabilities: $\hat{y} \in [0, 1]$
- Linear model can predict values < 0 or > 1



Solution:
Use a function that maps
 $(-\infty, \infty) \rightarrow [0, 1]$

TABLE OF CONTENTS

1 CLASSIFICATION

2 BINARY CLASSIFICATION

- **Logistic Regression**

- The Model: Single Neuron for Binary Classification
- Binary Classification: Worked Example
- Binary Classification: Evaluation Metrics

3 MULTI-CLASS CLASSIFICATION

- Multi-class Classification Model
- Mini-batch Stochastic Gradient Descent
- Multiclass Classification: Worked Example
- Making Predictions (Inference)
- Multiclass Classification: Evaluation Metrics

4 IMPLEMENTATION TIPS AND DEBUGGING

5 SUMMARY

WHAT IS LOGISTIC REGRESSION?

DEFINITION

Logistic regression is a linear model for binary classification that uses the **sigmoid function** to output probabilities.

Key Idea:

- Compute linear combination: $z = \mathbf{w}^T \mathbf{x} + w_0$
- Apply sigmoid function: $\hat{y} = \sigma(z)$
- Interpret output as probability: $\hat{y} = P(y = 1 | \mathbf{x}) \in [0, 1]$

Mathematical Form:

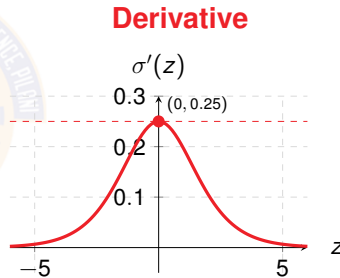
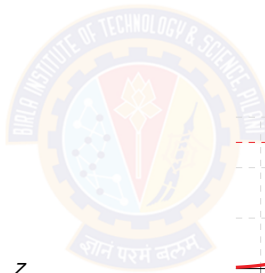
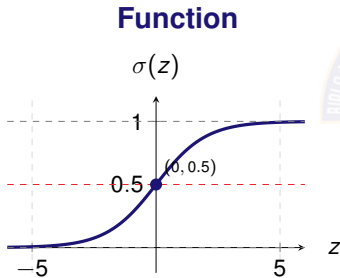
$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}} \quad (1)$$

where $\sigma(\cdot)$ is the sigmoid (logistic) function.

THE SIGMOID ACTIVATION FUNCTION

Sigmoid (Logistic) Function: $\sigma(z) = \frac{1}{1+e^{-z}}$

Derivative formula: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$



At $z = 0$: $\sigma(0) = 0.5$ $\sigma'(0) = 0.25$

PROPERTIES OF SIGMOID FUNCTION

Key Properties:

- **Range:** $\sigma(z) \in (0, 1) \rightarrow$ perfect for probabilities!
- **Monotonic:** Always increasing
- **Smooth:** Differentiable everywhere
- **Symmetry:** $\sigma(-z) = 1 - \sigma(z)$
- **Derivative:** $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

Behavior:

- When $z \rightarrow +\infty$: $\sigma(z) \rightarrow 1$ (confident class 1)
- When $z \rightarrow -\infty$: $\sigma(z) \rightarrow 0$ (confident class 0)
- When $z = 0$: $\sigma(z) = 0.5$ (uncertain)

The derivative property makes sigmoid ideal for gradient-based learning!

TABLE OF CONTENTS

1 CLASSIFICATION

2 BINARY CLASSIFICATION

- Logistic Regression
- The Model: Single Neuron for Binary Classification
- Binary Classification: Worked Example
- Binary Classification: Evaluation Metrics

3 MULTI-CLASS CLASSIFICATION

- Multi-class Classification Model
- Mini-batch Stochastic Gradient Descent
- Multiclass Classification: Worked Example
- Making Predictions (Inference)
- Multiclass Classification: Evaluation Metrics

4 IMPLEMENTATION TIPS AND DEBUGGING

5 SUMMARY

THE FOUR COMPONENTS

A complete deep learning system for classification consists of:

- ① **Data:** d -dimensional input vectors and binary labels
- ② **Model:** Single neuron with sigmoid activation
- ③ **Objective Function:** Binary cross-entropy loss
- ④ **Learning Algorithm:** Stochastic gradient descent (SGD)

Note: We use **stochastic** gradient descent instead of batch gradient descent for efficiency. (This is for learning purposes only.)

MATRIX FORM OF DATA

For efficient computation, we organize data into matrices:

Input Data = $\mathbf{x}^{(i)} = [1, x_1^{(i)}, \dots, x_d^{(i)}]^T$

Design Matrix: $\mathbf{X} \in \mathbb{R}^{N \times (d+1)}$ $\mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ 1 & x_1^{(2)} & \dots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(N)} & \dots & x_d^{(N)} \end{bmatrix}$

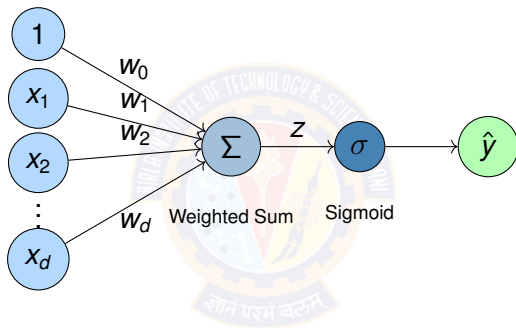
Label Vector: $\mathbf{y} \in \{0, 1\}^N$ $\mathbf{y} = [y^{(1)} \ y^{(2)} \ \dots \ y^{(N)}]^T$

Weight Vector: $\mathbf{w} \in \mathbb{R}^{d+1}$ $\mathbf{w} = [w_0 \ w_1 \ w_2 \ \dots \ w_d]^T$

Key difference from regression: $y^{(i)} \in \{0, 1\}$ (discrete), not continuous!

LOGISTIC REGRESSION AS A SINGLE NEURON

Model binary classification as a **single artificial neuron with sigmoid activation**:



- **Input:** $\mathbf{x} \in \mathbb{R}^d$
- **Linear combination:** $z = \mathbf{w}^T \mathbf{x}$
- **Activation:** Sigmoid $\hat{y} = \sigma(z)$
- **Output:** Predicted probability $\hat{y} = P(y = 1|\mathbf{x}) \in [0, 1]$

MODEL PREDICTION

Model: A single neuron computes weighted sum and applies sigmoid activation.

Prediction for input $\mathbf{x}$:

$$z = \mathbf{w}^T \mathbf{x} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d \quad (2)$$

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} \quad (3)$$

$$\hat{y} = P(y = 1 | \mathbf{x}) \in [0, 1] \quad (4)$$

where:

- z is the **logit** (pre-activation).
- $\hat{y}$ is the predicted probability.

MAKING PREDICTIONS

Decision Rule:

Given predicted probability $\hat{y} = P(y = 1|\mathbf{x})$:

$$\text{Predicted class} = \begin{cases} 1 & \text{if } \hat{y} \geq 0.5 \\ 0 & \text{if } \hat{y} < 0.5 \end{cases} \quad (5)$$

Equivalently, since $\sigma(z) = 0.5$ when $z = 0$:

$$\text{Predicted class} = \begin{cases} 1 & \text{if } \mathbf{w}^T \mathbf{x} \geq 0 \\ 0 & \text{if } \mathbf{w}^T \mathbf{x} < 0 \end{cases} \quad (6)$$

The decision boundary is: $\mathbf{w}^T \mathbf{x} = 0$ (a hyperplane in feature space)

BINARY CROSS-ENTROPY LOSS

We cannot use squared error for classification. Instead, we use **cross-entropy loss**.

Cross-entropy Loss for a single example:

$$\ell(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) = - [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] \quad (7)$$

where $\hat{y}^{(i)} = \sigma(\mathbf{w}^T \mathbf{x}^{(i)})$

Interpretation:

- If $y^{(i)} = 1$: Loss = $-\log(\hat{y}^{(i)})$
 - ▶ Penalizes if $\hat{y}^{(i)}$ is far from 1
- If $y^{(i)} = 0$: Loss = $-\log(1 - \hat{y}^{(i)})$
 - ▶ Penalizes if $\hat{y}^{(i)}$ is far from 0

TOTAL LOSS FUNCTION

Average Binary Cross-Entropy Loss:

$$J(\mathbf{w}) = \frac{-1}{N} \sum_{i=1}^N [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] \quad (8)$$

Goal:

Find $\mathbf{w}^*$ that minimizes $J(\mathbf{w})$

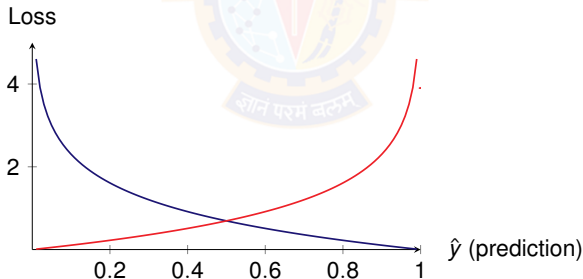
$$\mathbf{w}^* = \arg \min_{\mathbf{w}} J(\mathbf{w}) \quad (9)$$

WHY CROSS-ENTROPY LOSS?

Advantages:

- Probabilistic interpretation: Derived from maximum likelihood estimation
- Appropriate for probabilities: Heavily penalizes confident wrong predictions
- Convex: Has a single global minimum for logistic regression
- Well-behaved gradients: Works well with sigmoid activation

Loss behavior: Loss increases exponentially as prediction diverges from true label!



FROM BATCH TO STOCHASTIC GRADIENT DESCENT

Batch Gradient Descent (from Module 3):

- Uses **all** training examples in each update
- Computes gradient: $\nabla J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \nabla \ell(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)})$
- Update: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J(\mathbf{w})$

Problems with Batch GD:

- Slow for large datasets (must process all N examples per update)
- High memory requirements
- Redundant computations when examples are similar

Solution: Stochastic Gradient Descent (SGD)

- Update weights using **one random example** at a time
- Much faster iterations
- Lower memory requirements

STOCHASTIC GRADIENT DESCENT (SGD)

Key Idea:

Instead of computing gradient over entire dataset, use gradient from a **single random example**.

SGD Update Rule:

At iteration t :

- 1 Randomly select one example $(\mathbf{x}^{(i)}, y^{(i)})$
- 2 Compute gradient for this example: $\nabla \ell(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)})$ (noisy gradient estimate)
- 3 Update weights: $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla \ell(\mathbf{w}^{(t)}; \mathbf{x}^{(i)}, y^{(i)})$

The noise in SGD actually helps escape local minima and often leads to better generalization!

COMPUTING THE GRADIENT FOR SGD

Gradient of cross-entropy loss for one example:

For example $(\mathbf{x}^{(i)}, y^{(i)})$:

$$\nabla_{\mathbf{w}} \ell(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) = \frac{\partial \ell}{\partial \mathbf{w}_j} = (\hat{y}^{(i)} - y^{(i)}) \mathbf{x}^{(i)} \quad (10)$$

where $\hat{y}^{(i)} = \sigma(\mathbf{w}^T \mathbf{x}^{(i)})$ for $j = 0, 1, \dots, d$ (where $x_0^{(i)} = 1$ for bias)

Remarkably simple form!

- Error: $(\hat{y}^{(i)} - y^{(i)})$
- Direction: $\mathbf{x}^{(i)}$
- Same structure as linear regression gradient!

STOCHASTIC GRADIENT DESCENT ALGORITHM

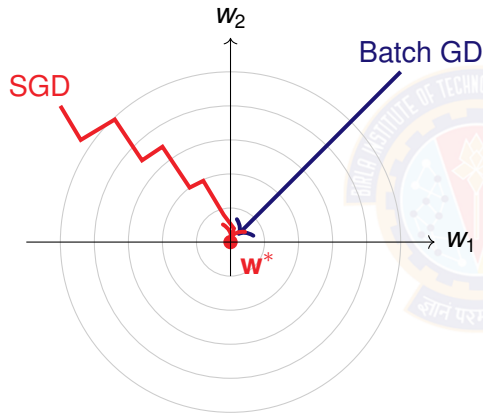
Algorithm 1: Stochastic Gradient Descent for Logistic Regression

Input: Dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$, learning rate η , epochs T

Output: Learned weights $\mathbf{w}$

```
1 Initialize  $\mathbf{w}^{(0)} = \mathbf{0}$  (or small random values);
2 for  $epoch = 1$  to  $T$  do
3     Shuffle dataset  $\mathcal{D}$ ;
4     for each example  $(\mathbf{x}^{(i)}, y^{(i)})$  in  $\mathcal{D}$  do
5         Compute prediction:  $\hat{y}^{(i)} = \sigma(\mathbf{w}^T \mathbf{x}^{(i)})$ ;
6         Compute gradient:  $\nabla \ell = (\hat{y}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$ ;
7         Update weights:  $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \ell$ ;
8     Optional: Compute total loss  $J(\mathbf{w})$  for monitoring;
9 return  $\mathbf{w}$ ;
```

SGD vs BATCH GD: VISUAL COMPARISON



Batch GD: Smooth, deterministic path

SGD: Noisy, stochastic path but faster iterations

BENEFITS AND CHALLENGES OF SGD

Benefits:

- **Speed:** Much faster per iteration (only one example)
- **Memory:** Can handle datasets that don't fit in memory
- **Online learning:** Can update model as new data arrives
- **Generalization:** Noise helps escape sharp minima
- **Scalability:** Works well with massive datasets

Challenges:

- **Noisy updates:** Path to minimum is not smooth
- **Learning rate:** More sensitive to η choice
- **Convergence:** May oscillate near minimum
- **Hyperparameter tuning:** Requires careful tuning

Common compromise: Mini-batch SGD

TABLE OF CONTENTS

1 CLASSIFICATION

2 BINARY CLASSIFICATION

- Logistic Regression
- The Model: Single Neuron for Binary Classification
- **Binary Classification: Worked Example**
- Binary Classification: Evaluation Metrics

3 MULTI-CLASS CLASSIFICATION

- Multi-class Classification Model
- Mini-batch Stochastic Gradient Descent
- Multiclass Classification: Worked Example
- Making Predictions (Inference)
- Multiclass Classification: Evaluation Metrics

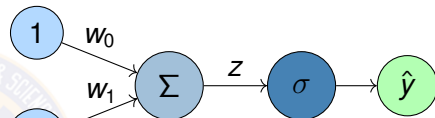
4 IMPLEMENTATION TIPS AND DEBUGGING

5 SUMMARY

NUMERICAL EXAMPLE: SETUP

Tiny Dataset: Classify whether a student passes based on study hours

Hours (x_1)	Pass (y)
1	0
2	0
3	1
4	1



Matrix Formulation:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

NUMERICAL EXAMPLE: INITIALIZATION

Hyperparameters: $\eta = 0.5$, $N = 4$

Initialize weights: $\mathbf{w}^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

Sigmoid function: $\sigma(z) = \frac{1}{1 + e^{-z}}$

Loss function: $\ell(\mathbf{w}; \mathbf{x}, y) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$

Initial predictions with $\mathbf{w}^{(0)} = \begin{bmatrix} 0 & 0 \end{bmatrix}^T$: For all examples: $z = 0 \Rightarrow \hat{y} = \sigma(0) = 0.5$

$$\hat{\mathbf{y}}^{(0)} = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

NUMERICAL EXAMPLE: FIRST SGD ITERATION

Randomly select first example: $(\mathbf{x}^{(1)}, y^{(1)}) = ([1 \ 1], 0)$

Step 1 - Forward pass: $z^{(1)} = \mathbf{x}^{(1)} \mathbf{w}^{(0)} = [1 \ 1] \begin{bmatrix} 0 \\ 0 \end{bmatrix} = 0$

$$\hat{y}^{(1)} = \sigma(0) = 0.5$$

Step 2 - Compute error: $\text{error} = \hat{y}^{(1)} - y^{(1)} = 0.5 - 0 = 0.5$

Step 3 - Compute gradient: $\nabla \ell = (\hat{y}^{(1)} - y^{(1)}) \mathbf{x}^{(1)} = 0.5 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$

Step 4 - Update weights: $\mathbf{w}^{(1)} = \mathbf{w}^{(0)} - \eta \nabla \ell$

$$= \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.5 \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} -0.25 \\ -0.25 \end{bmatrix}$$

NUMERICAL EXAMPLE: SECOND SGD ITERATION

Randomly select second example: $(\mathbf{x}^{(3)}, y^{(3)}) = ([1 \ 3], 1)$

Step 1: Forward pass: $z^{(3)} = \mathbf{X}^{(3)}\mathbf{w}^{(1)} = [1 \ 3] \begin{bmatrix} -0.25 \\ -0.25 \end{bmatrix} = -1.0$

$$\hat{y}^{(3)} = \sigma(-1.0) = \frac{1}{1 + e^{1.0}} \approx 0.269$$

Step 2: Compute error: $\text{error} = 0.269 - 1 = -0.731$

Step 3: Compute gradient: $\nabla \ell = -0.731 \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} -0.731 \\ -2.193 \end{bmatrix}$

Step 4: Update weights: $\mathbf{w}^{(2)} = \begin{bmatrix} -0.25 \\ -0.25 \end{bmatrix} - 0.5 \begin{bmatrix} -0.731 \\ -2.193 \end{bmatrix} = \begin{bmatrix} 0.116 \\ 0.847 \end{bmatrix}$

NUMERICAL EXAMPLE: CHECKING PROGRESS

After 2 SGD iterations, let's check predictions:

With $\mathbf{w}^{(2)} = \begin{bmatrix} 0.116 \\ 0.847 \end{bmatrix}$:

Example 1: $\hat{y}^{(1)} = \sigma(0.116 + 0.847 \cdot 1) \approx \sigma(0.963) \approx 0.724$

Example 2: $\hat{y}^{(2)} = \sigma(0.116 + 0.847 \cdot 2) \approx \sigma(1.810) \approx 0.859$

Example 3: $\hat{y}^{(3)} = \sigma(0.116 + 0.847 \cdot 3) \approx \sigma(2.657) \approx 0.934$

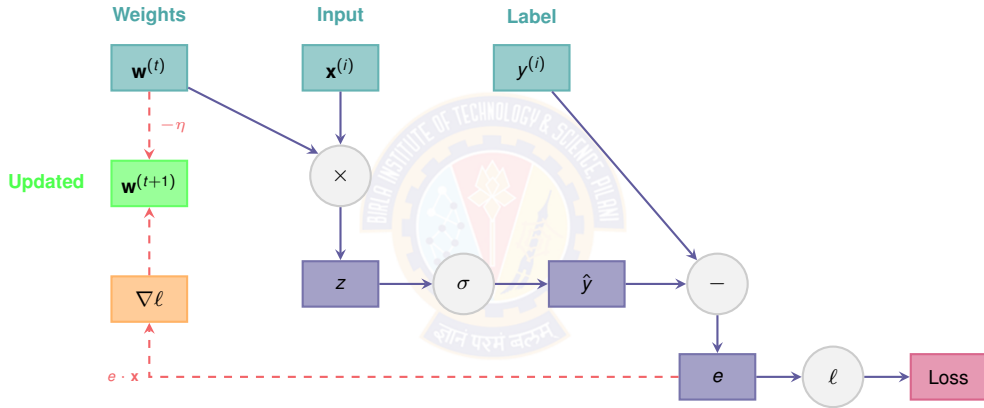
Example 4: $\hat{y}^{(4)} = \sigma(0.116 + 0.847 \cdot 4) \approx \sigma(3.504) \approx 0.971$

Progress:

- Examples 1, 2 (true label 0): predictions moving toward 0 ✓
- Examples 3, 4 (true label 1): predictions moving toward 1 ✓

Continue SGD for multiple epochs until convergence!

COMPUTATIONAL GRAPH FOR ONE SGD STEP



Single example used per iteration — much faster than batch GD!

TABLE OF CONTENTS

1 CLASSIFICATION

2 BINARY CLASSIFICATION

- Logistic Regression
- The Model: Single Neuron for Binary Classification
- Binary Classification: Worked Example
- Binary Classification: Evaluation Metrics

3 MULTI-CLASS CLASSIFICATION

- Multi-class Classification Model
- Mini-batch Stochastic Gradient Descent
- Multiclass Classification: Worked Example
- Making Predictions (Inference)
- Multiclass Classification: Evaluation Metrics

4 IMPLEMENTATION TIPS AND DEBUGGING

5 SUMMARY

EVALUATING CLASSIFICATION MODELS

Unlike regression, we need different metrics for classification.

Key Questions:

- How often does the model predict correctly? (Accuracy)
- What types of errors does it make? (Confusion Matrix)
- How well does it identify positive cases? (Precision, Recall)

Data Split:

- Training set: Learn weights $\mathbf{w}$ (typically 90-99%)
- Validation set: Tune hyperparameters (typically 10-15%)
- Test set: Final evaluation (typically 1-10%)

Always evaluate on unseen test data to measure generalization!

CONFUSION MATRIX

The confusion matrix shows all prediction outcomes:

		Predicted	
		Positive (1)	Negative (0)
Actual	Positive (1)	True Positive (TP)	False Negative (FN)
	Negative (0)	False Positive (FP)	True Negative (TN)

Definitions:

- **TP**: Correctly predicted positive
- **TN**: Correctly predicted negative
- **FP**: Incorrectly predicted positive (Type I error)
- **FN**: Incorrectly predicted negative (Type II error)

CLASSIFICATION METRICS I

1. **Accuracy:** Overall correctness

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (11)$$

2. **Precision:** Of predicted positives, how many are correct?

$$\text{Precision} = \frac{TP}{TP + FP} \quad (12)$$

CLASSIFICATION METRICS II

3. Recall (Sensitivity): Of actual positives, how many did we find?

$$\text{Recall} = \frac{TP}{TP + FN} \quad (13)$$

4. F1 Score: Harmonic mean of precision and recall

$$F1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (14)$$

WHEN TO USE WHICH METRIC?

Accuracy:

- Use when classes are balanced
- **Misleading with imbalanced data!** (e.g., 95% negative class)

Precision:

- Important when False Positives are costly
- Example: Spam detection (don't want to mark real emails as spam)

Recall:

- Important when False Negatives are costly
- Example: Disease detection (don't want to miss sick patients)

F1 Score:

- Use when you need balance between precision and recall
- Good for imbalanced datasets

EXAMPLE: COMPUTING METRICS

Given predictions:

True	Pred	True	Pred
1	1 (TP)	0	0 (TN)
1	1 (TP)	0	0 (TN)
1	0 (FN)	0	1 (FP)
1	1 (TP)	0	0 (TN)

$$\text{Accuracy} = \frac{3 + 3}{3 + 3 + 1 + 1} = \frac{6}{8} = 0.75$$

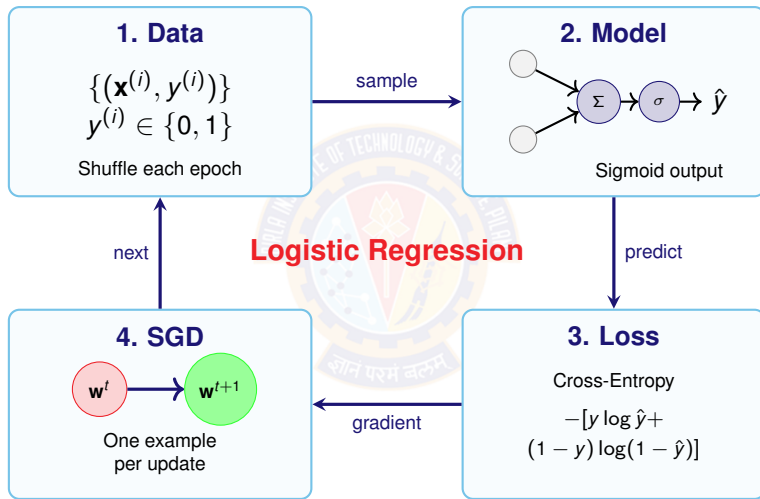
$$\text{Precision} = \frac{3}{3 + 1} = 0.75$$

$$\text{Recall} = \frac{3}{3 + 1} = 0.75$$

$$F1 = 2 \cdot \frac{0.75 \times 0.75}{0.75 + 0.75} = 0.75$$

Counts: TP = 3, TN = 3, FP = 1, FN = 1

SUMMARY: BINARY CLASSIFICATION



This framework is the foundation for neural networks!

KEY TAKEAWAYS

- Binary classification predicts discrete categories using $y \in \{0, 1\}$.
- Sigmoid activation $\sigma(z) = \frac{1}{1+e^{-z}}$ maps to probabilities $[0, 1]$.
- Logistic regression = linear model + sigmoid activation.
- Cross-entropy loss is the appropriate objective for classification.
- Stochastic Gradient Descent (SGD) updates weights one example at a time — faster and more scalable.
- Feature scaling is essential for convergence and numerical stability.
- Evaluation metrics: Accuracy, Precision, Recall, F1 Score, Confusion Matrix.
- This framework extends naturally to multi-class classification and deep networks.

TABLE OF CONTENTS

1 CLASSIFICATION

2 BINARY CLASSIFICATION

- Logistic Regression
- The Model: Single Neuron for Binary Classification
- Binary Classification: Worked Example
- Binary Classification: Evaluation Metrics

3 MULTI-CLASS CLASSIFICATION

- Multi-class Classification Model
- Mini-batch Stochastic Gradient Descent
- Multiclass Classification: Worked Example
- Making Predictions (Inference)
- Multiclass Classification: Evaluation Metrics

4 IMPLEMENTATION TIPS AND DEBUGGING

5 SUMMARY

WHAT IS MULTI-CLASS CLASSIFICATION?

DEFINITION

Multi-class classification assigns each input to one of K classes, where $K > 2$.

Key Characteristics:

- Classes: $y \in \{1, 2, 3, \dots, K\}$ (or $\{0, 1, 2, \dots, K - 1\}$)
- Each example belongs to **exactly one** class
- Output: Probability distribution over all K classes
- Probabilities sum to 1: $\sum_{k=1}^K P(y = k|\mathbf{x}) = 1$

Example:

- Input: Image of a handwritten digit
- Output: Probabilities for each of 10 classes (0, 1, 2, ..., 9)
- Prediction: Class with highest probability

EXAMPLES OF MULTI-CLASS CLASSIFICATION

① Handwritten Digit Recognition (MNIST)

- ▶ Input: 28×28 pixel image
- ▶ Output: One of 10 digits (0-9)

② Image Classification (ImageNet)

- ▶ Input: Color image
- ▶ Output: One of 1000 object categories (cat, dog, car, plane, ...)

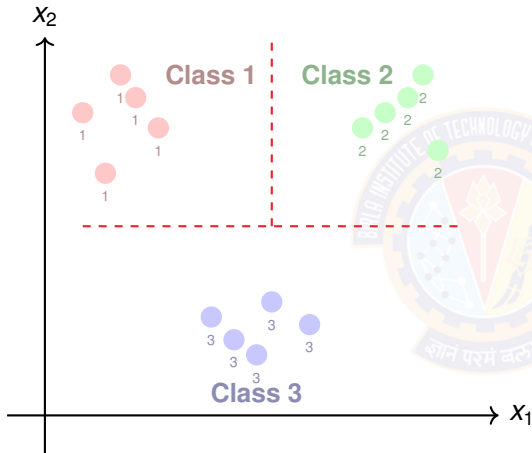
③ Text Classification

- ▶ Input: News article text
- ▶ Output: Topic category (politics, sports, technology, entertainment, business)

④ Speech Recognition

- ▶ Input: Audio features
- ▶ Output: Phoneme class (multiple phonemes in a language)

MULTI-CLASS CLASSIFICATION: VISUAL EXAMPLE



Key insight: We need **multiple output neurons** (one per class) and a way to convert their outputs into a valid probability distribution.

Decision boundaries separate $K = 3$ classes in feature space.

TABLE OF CONTENTS

1 CLASSIFICATION

2 BINARY CLASSIFICATION

- Logistic Regression
- The Model: Single Neuron for Binary Classification
- Binary Classification: Worked Example
- Binary Classification: Evaluation Metrics

3 MULTI-CLASS CLASSIFICATION

- **Multi-class Classification Model**
- Mini-batch Stochastic Gradient Descent
- Multiclass Classification: Worked Example
- Making Predictions (Inference)
- Multiclass Classification: Evaluation Metrics

4 IMPLEMENTATION TIPS AND DEBUGGING

5 SUMMARY

THE FOUR COMPONENTS FOR MULTI-CLASS

A complete multi-class classification system consists of:

- ① **Data:** Input features and class labels $y \in \{1, 2, \dots, K\}$
- ② **Model:** Linear model with K output neurons (no hidden layers)
- ③ **Activation:** Softmax function to produce probability distribution
- ④ **Objective Function:** Categorical cross-entropy loss
- ⑤ **Learning Algorithm:** Mini-batch stochastic gradient descent

Note: We now use **mini-batch SGD** instead of single-example SGD for better efficiency and stability.

DATA REPRESENTATION

Input Features:

$$\mathbf{X} \in \mathbb{R}^{N \times (d+1)} \quad \mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ 1 & x_1^{(2)} & \dots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(N)} & \dots & x_d^{(N)} \end{bmatrix}$$

Class Labels: Two representations

1. Integer encoding: $\mathbf{y} \in \{1, 2, \dots, K\}^N$

$$\mathbf{y} = [3 \quad 1 \quad 2 \quad 1 \quad 3]^T$$

2. One-hot encoding: $\mathbf{Y} \in \{0, 1\}^{N \times K}$ (preferred for computation)

ONE-HOT ENCODING

Definition: Convert class label to binary vector of length K

For class k : All zeros except 1 at position k

Example with $K = 4$ classes:

Class 1: $\mathbf{y} = [1, 0, 0, 0]^T$

Class 2: $\mathbf{y} = [0, 1, 0, 0]^T$

Class 3: $\mathbf{y} = [0, 0, 1, 0]^T$

Class 4: $\mathbf{y} = [0, 0, 0, 1]^T$

Why one-hot?

- Treats all classes equally (no implicit ordering)
- Matches the format of softmax output probabilities
- Simplifies loss computation
- Standard in deep learning frameworks

WEIGHT MATRIX FOR MULTI-CLASS CLASSIFICATION

For K classes and d input features:

Weight Matrix: $\mathbf{W} \in \mathbb{R}^{(d+1) \times K}$

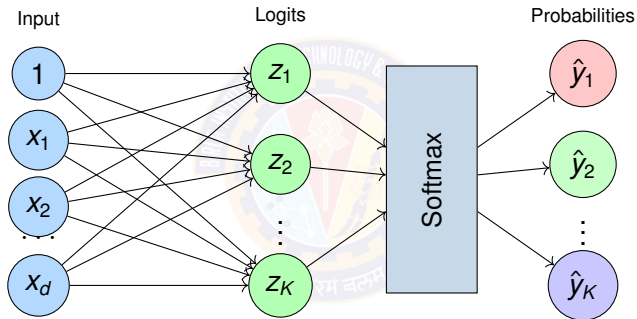
$$\mathbf{W} = \begin{bmatrix} w_{0,1} & w_{0,2} & \cdots & w_{0,K} \\ w_{1,1} & w_{1,2} & \cdots & w_{1,K} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,K} \\ \vdots & \vdots & \ddots & \vdots \\ w_{d,1} & w_{d,2} & \cdots & w_{d,K} \end{bmatrix}$$

- Each **column** $\mathbf{w}_k$ represents weights for class k
- Each class has its own linear model
- Row 0: Bias terms for all classes
- Total parameters: $(d + 1) \times K$

Compare to binary: 1 weight vector vs K weight vectors!

MULTI-CLASS MODEL ARCHITECTURE

Architecture: Multiple output neurons with softmax activation



$$\mathbf{z} = \mathbf{W}^T \mathbf{x} \text{ (linear)} \rightarrow \hat{\mathbf{y}} = \text{softmax}(\mathbf{z}) \text{ (activation)}$$

LINEAR MODEL: COMPUTING LOGITS

Step 1: Compute logits (pre-activation values)

For a single input $\mathbf{x}$:

$$\mathbf{z} = \mathbf{W}^T \mathbf{x} = \begin{bmatrix} \mathbf{w}_1^T \mathbf{x} \\ \mathbf{w}_2^T \mathbf{x} \\ \vdots \\ \mathbf{w}_K^T \mathbf{x} \end{bmatrix} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_K \end{bmatrix} \quad (15)$$

where $z_k = \mathbf{w}_k^T \mathbf{x} = w_{0,k} + w_{1,k}x_1 + \cdots + w_{d,k}x_d$

For entire dataset (batch):

$$\mathbf{Z} = \mathbf{XW} \in \mathbb{R}^{N \times K} \quad (16)$$

Each row of $\mathbf{Z}$ contains logits for one example.

Logits are **unbounded** values in $(-\infty, \infty) \rightarrow$ not probabilities yet!

THE SOFTMAX FUNCTION

Purpose: Convert K logits into a valid probability distribution

DEFINITION:

For logit vector $\mathbf{z} = [z_1, z_2, \dots, z_K]^T$

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \quad \text{for } k = 1, 2, \dots, K \quad (17)$$

Output: Probability vector $\hat{\mathbf{y}} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_K]^T$

$$\hat{y}_k = P(y = k|\mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \quad (18)$$

Interpretation: $\hat{y}_k$ is the predicted probability for class k

PROPERTIES OF SOFTMAX

Key Properties:

- ① Output range: $\hat{y}_k \in (0, 1)$ for all k
- ② Probability distribution: $\sum_{k=1}^K \hat{y}_k = 1$
- ③ Preserves order: If $z_i > z_j$, then $\hat{y}_i > \hat{y}_j$
- ④ Translation invariant: $\text{softmax}(\mathbf{z}) = \text{softmax}(\mathbf{z} + c)$ for any constant c
- ⑤ Differentiable: Enables gradient-based learning
- ⑥ Reduces to sigmoid: When $K = 2$, softmax is equivalent to sigmoid

The exponential amplifies differences: larger logits get much larger probabilities!

SOFTMAX EXAMPLE: NUMERICAL COMPUTATION

Given logits: $\mathbf{z} = [2.0, 1.0, 0.1]^T$ for $K = 3$ classes

Step 1: Compute exponentials

$$e^{z_1} = e^{2.0} \approx 7.39$$

$$e^{z_2} = e^{1.0} \approx 2.72$$

$$e^{z_3} = e^{0.1} \approx 1.11$$

Step 2: Compute sum

$$\sum_{j=1}^3 e^{z_j} = 7.39 + 2.72 + 1.11 = 11.22$$

Step 3: Normalize

$$\hat{y}_1 = \frac{7.39}{11.22} \approx 0.659$$

$$\hat{y}_2 = \frac{2.72}{11.22} \approx 0.242$$

$$\hat{y}_3 = \frac{1.11}{11.22} \approx 0.099$$

Step 4: Predicted class:

$\arg \max_k \hat{y}_k = 1$ (highest probability)



COMPLETE FORWARD PASS

For a batch of N examples $\mathbf{X}$:

Step 1: Compute logits $\mathbf{Z} = \mathbf{XW} \in \mathbb{R}^{N \times K}$ (19)

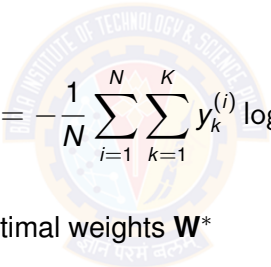
Step 2: Apply softmax $\hat{\mathbf{Y}} = \text{softmax}(\mathbf{Z}) \in [0, 1]^{N \times K}$ (20)
(softmax applied row-wise to each example)

Step 3: Predict class $\hat{k} = \arg \max_k \hat{y}_k$ (21)

Each row of $\hat{\mathbf{Y}}$ is a probability distribution over K classes.

CATEGORICAL CROSS-ENTROPY LOSS

Total loss over dataset:


$$J(\mathbf{W}) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_k^{(i)} \log(\hat{y}_k^{(i)}) \quad (22)$$

Goal: Minimize $J(\mathbf{W})$ to find optimal weights $\mathbf{W}^*$

UNDERSTANDING CATEGORICAL CROSS-ENTROPY

Intuition:

- Loss depends only on predicted probability for the **true class**
- Heavily penalizes **confident wrong predictions**
- Encourages high probability for correct class

Example: True class is 2 ($\mathbf{y} = [0, 1, 0]$)

Prediction	$\hat{y}_1$	$\hat{y}_2$	Loss = $-\log(\hat{y}_2)$
Good	0.1	0.8	0.22
Medium	0.3	0.5	0.69
Bad	0.7	0.2	1.61
Very bad	0.9	0.05	3.00

Loss increases exponentially as predicted probability for true class decreases!

WHY CATEGORICAL CROSS-ENTROPY?

Advantages:

- Probabilistic interpretation: Derived from maximum likelihood
- Appropriate for multi-class: Naturally handles K classes
- Works well with softmax: Numerically stable gradient
- Penalizes wrong confidence: Heavily penalizes confident mistakes
- Convex: For linear models (like ours)

Connection to binary cross-entropy:

When $K = 2$:

$$\text{Categorical: } - [y_1 \log(\hat{y}_1) + y_2 \log(\hat{y}_2)]$$

$$\text{Binary: } - [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

They are equivalent! (with $y_2 = y$ and $y_1 = 1 - y$)

TABLE OF CONTENTS

1 CLASSIFICATION

2 BINARY CLASSIFICATION

- Logistic Regression
- The Model: Single Neuron for Binary Classification
- Binary Classification: Worked Example
- Binary Classification: Evaluation Metrics

3 MULTI-CLASS CLASSIFICATION

- Multi-class Classification Model
- **Mini-batch Stochastic Gradient Descent**
- Multiclass Classification: Worked Example
- Making Predictions (Inference)
- Multiclass Classification: Evaluation Metrics

4 IMPLEMENTATION TIPS AND DEBUGGING

5 SUMMARY

FROM SGD TO MINI-BATCH SGD

Recap: Gradient Descent Variants

1. Batch Gradient Descent:

- Uses **all** N examples per update
- Slow but stable, accurate gradient

2. Stochastic Gradient Descent (SGD):

- Uses **1** random example per update
- Fast but noisy, unstable

3. Mini-batch SGD (Best of both worlds):

- Uses **small batch** of B examples per update ($1 < B \ll N$)
- Typical batch sizes: $B \in \{32, 64, 128, 256, 512\}$
- Good balance of speed and stability
- **Standard in modern deep learning!**

WHY MINI-BATCH SGD?

Advantages of mini-batches:

① Computational efficiency:

- ▶ Vectorized operations (matrix multiplication)
- ▶ Better GPU utilization
- ▶ Faster than processing examples one-by-one

② Gradient estimation:

- ▶ Less noisy than single-example SGD
- ▶ More stable convergence
- ▶ Better approximation of true gradient

③ Generalization:

- ▶ Noise helps escape sharp minima
- ▶ Often leads to better test performance

④ Memory efficiency:

- ▶ Can handle datasets larger than memory
- ▶ Load and process data in batches

MINI-BATCH SGD: KEY CONCEPTS

- **Batch size (B):** Number of examples per update
 - ▶ Small: 32-64 (more noise, faster iterations)
 - ▶ Medium: 128-256 (balanced)
 - ▶ Large: 512-1024 (less noise, but slower convergence)
- **Iteration:** One weight update (processing one mini-batch)
- **Epoch:** One complete pass through the entire dataset

$$\text{Iterations per epoch} = \left\lceil \frac{N}{B} \right\rceil \quad (23)$$

Example: $N = 60,000$ samples, $B = 128$

- ▶ Iterations per epoch: $\lceil 60000/128 \rceil = 469$
- ▶ After 10 epochs: $469 \times 10 = 4690$ weight updates

COMPUTING GRADIENTS FOR MINI-BATCH

Gradient for a mini-batch of size B :

Let $\mathcal{B}$ be a mini-batch of B examples: $\mathcal{B} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i \in \mathcal{I}}$

Average gradient over mini-batch:

$$\nabla J_{\mathcal{B}}(\mathbf{W}) = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \ell(\mathbf{W}; \mathbf{x}^{(i)}, \mathbf{y}^{(i)}) \quad (24)$$

For categorical cross-entropy with softmax:

$$\nabla_{\mathbf{W}} J_{\mathcal{B}} = \frac{1}{B} \mathbf{X}_{\mathcal{B}}^T (\hat{\mathbf{Y}}_{\mathcal{B}} - \mathbf{Y}_{\mathcal{B}}) \quad (25)$$

where:

- $\mathbf{X}_{\mathcal{B}} \in \mathbb{R}^{B \times (d+1)}$: Mini-batch inputs
- $\hat{\mathbf{Y}}_{\mathcal{B}} \in \mathbb{R}^{B \times K}$: Predicted probabilities
- $\mathbf{Y}_{\mathcal{B}} \in \mathbb{R}^{B \times K}$: One-hot true labels

MINI-BATCH SGD UPDATE RULE

Update at iteration t :

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla J_{\mathcal{B}}(\mathbf{w}^{(t)}) \quad (26)$$

Expanded form:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \frac{\eta}{B} \mathbf{X}_{\mathcal{B}}^T (\hat{\mathbf{Y}}_{\mathcal{B}} - \mathbf{Y}_{\mathcal{B}}) \quad (27)$$

MINI-BATCH SGD ALGORITHM

Algorithm 2: Mini-batch SGD for Multi-class Classification

Input: Dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^N$, batch size B , learning rate η , epochs T

Output: Learned weight matrix $\mathbf{W}$

```
1 Initialize  $\mathbf{W} \sim \mathcal{N}(0, 0.01)$  // Small random values
2 for  $epoch = 1$  to  $T$  do
3     Shuffle dataset  $\mathcal{D}$ ;
4     for each mini-batch  $\mathcal{B}$  of size  $B$  do
5         // Forward pass
6          $\mathbf{Z}_{\mathcal{B}} = \mathbf{X}_{\mathcal{B}}\mathbf{W}$  // Compute logits
7          $\hat{\mathbf{Y}}_{\mathcal{B}} = \text{softmax}(\mathbf{Z}_{\mathcal{B}})$  // Apply softmax
8          $\nabla J = \frac{1}{B}\mathbf{X}_{\mathcal{B}}^T(\hat{\mathbf{Y}}_{\mathcal{B}} - \mathbf{Y}_{\mathcal{B}})$  // Compute gradient
9          $\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla J$  // Update weights
10        Optional: Compute total loss for monitoring;
11 return  $\mathbf{W}$ ;
```

CHOOSING BATCH SIZE

Batch Size	Advantages	Disadvantages
Small (32-64)	• Fast iterations • More updates per epoch • Better generalization • Escapes sharp minima	• Noisy gradients • Unstable training • Poor GPU utilization
Medium (128-256)	• Good balance • Stable training • Good GPU usage • Standard choice	• May need tuning
Large (512-1024)	• Stable gradients • Better GPU utilization • Faster per-epoch time	• Slow convergence • May generalize worse • More memory

TABLE OF CONTENTS

1 CLASSIFICATION

2 BINARY CLASSIFICATION

- Logistic Regression
- The Model: Single Neuron for Binary Classification
- Binary Classification: Worked Example
- Binary Classification: Evaluation Metrics

3 MULTI-CLASS CLASSIFICATION

- Multi-class Classification Model
- Mini-batch Stochastic Gradient Descent
- **Multiclass Classification: Worked Example**
- Making Predictions (Inference)
- Multiclass Classification: Evaluation Metrics

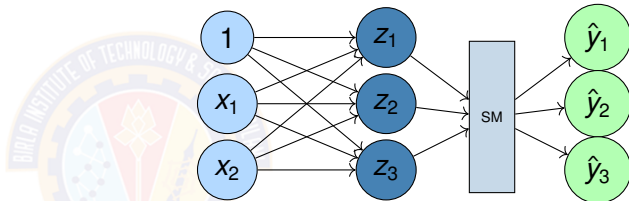
4 IMPLEMENTATION TIPS AND DEBUGGING

5 SUMMARY

NUMERICAL EXAMPLE: SETUP

Tiny Dataset: Classify 4 data points into 3 classes

x_1	x_2	Class
1	2	1
2	1	2
2	3	1
3	2	3



Parameters:

- $K = 3$ classes, $d = 2$ features, $N = 4$ examples
- Batch size: $B = 2$ (mini-batch)
- Learning rate: $\eta = 0.1$

NUMERICAL EXAMPLE: DATA REPRESENTATION

Design Matrix: (with bias column)

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \\ 1 & 2 & 3 \\ 1 & 3 & 2 \end{bmatrix} \in \mathbb{R}^{4 \times 3}$$

One-hot Labels:

$$\mathbf{Y} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 3}$$

(class 1)
(class 2)
(class 1)
(class 3)

Weight Matrix: Initialize randomly

$$\mathbf{W}^{(0)} = \begin{bmatrix} 0.1 & -0.1 & 0.2 \\ 0.2 & 0.1 & -0.1 \\ -0.1 & 0.2 & 0.1 \end{bmatrix} \in \mathbb{R}^{3 \times 3}$$

NUMERICAL EXAMPLE: FIRST MINI-BATCH (EXAMPLE)

Mini-batch: First 2 examples after shuffling

$$\mathbf{X}_{\mathcal{B}} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \end{bmatrix}, \mathbf{Y}_{\mathcal{B}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Step 1: Compute logits

$$\begin{aligned} \mathbf{Z}_{\mathcal{B}} &= \mathbf{X}_{\mathcal{B}} \mathbf{W}^{(0)} \\ &= \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \end{bmatrix} \begin{bmatrix} 0.1 & -0.1 & 0.2 \\ 0.2 & 0.1 & -0.1 \\ -0.1 & 0.2 & 0.1 \end{bmatrix} \\ &= \begin{bmatrix} 0.1 & 0.4 & 0.3 \\ 0.4 & 0.2 & 0.1 \end{bmatrix} \end{aligned}$$

NUMERICAL EXAMPLE: SOFTMAX COMPUTATION

Step 2: Apply softmax to each row

Softmax for example 1: $\hat{\mathbf{y}}^{(1)} \approx \begin{bmatrix} 0.280 \\ 0.378 \\ 0.342 \end{bmatrix}$

Similarly for example 2: $\hat{\mathbf{y}}^{(2)} \approx [0.387, 0.315, 0.298]^T$

For the entire batch: $\hat{\mathbf{Y}}_{\mathcal{B}} = \begin{bmatrix} 0.280 & 0.378 & 0.342 \\ 0.387 & 0.315 & 0.298 \end{bmatrix}$

NUMERICAL EXAMPLE: GRADIENT AND UPDATE

Step 3: Compute gradient

$$\begin{aligned}\nabla J &= \frac{1}{B} \mathbf{X}_B^T (\hat{\mathbf{Y}}_B - \mathbf{Y}_B) \\ \hat{\mathbf{Y}}_B - \mathbf{Y}_B &= \begin{bmatrix} 0.280 & 0.378 & 0.342 \\ 0.387 & 0.315 & 0.298 \end{bmatrix} - \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \\ &= \begin{bmatrix} -0.720 & 0.378 & 0.342 \\ 0.387 & -0.685 & 0.298 \end{bmatrix} \\ \nabla J &= \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} -0.720 & 0.378 & 0.342 \\ 0.387 & -0.685 & 0.298 \end{bmatrix} \\ &\approx \begin{bmatrix} -0.167 & -0.154 & 0.320 \\ -0.024 & -0.496 & 0.469 \\ -0.527 & 0.036 & 0.491 \end{bmatrix}\end{aligned}$$

NUMERICAL EXAMPLE: WEIGHT UPDATE

Step 4: Update weights

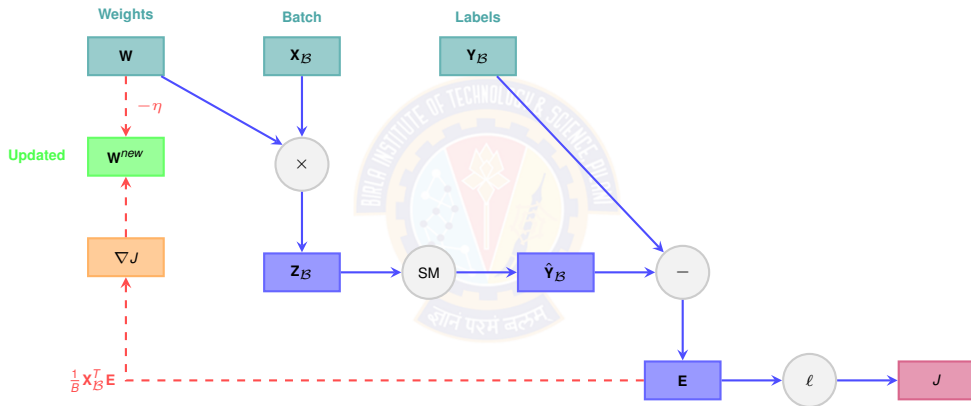
$$\begin{aligned}\mathbf{W}^{(1)} &= \mathbf{W}^{(0)} - \eta \nabla J \\ &= \begin{bmatrix} 0.1 & -0.1 & 0.2 \\ 0.2 & 0.1 & -0.1 \\ -0.1 & 0.2 & 0.1 \end{bmatrix} - 0.1 \begin{bmatrix} -0.167 & -0.154 & 0.320 \\ -0.024 & -0.496 & 0.469 \\ -0.527 & 0.036 & 0.491 \end{bmatrix} \\ &= \begin{bmatrix} 0.117 & -0.085 & 0.168 \\ 0.202 & 0.150 & -0.147 \\ -0.047 & 0.196 & 0.051 \end{bmatrix}\end{aligned}$$

Progress:

- Weights adjusted based on mini-batch gradient
- Process next mini-batch (examples 3-4) with updated weights
- Continue for multiple epochs until convergence

This completes one iteration of mini-batch SGD!

COMPUTATIONAL GRAPH FOR MINI-BATCH FORWARD



Mini-batch processes B examples simultaneously — efficient and stable!

TABLE OF CONTENTS

1 CLASSIFICATION

2 BINARY CLASSIFICATION

- Logistic Regression
- The Model: Single Neuron for Binary Classification
- Binary Classification: Worked Example
- Binary Classification: Evaluation Metrics

3 MULTI-CLASS CLASSIFICATION

- Multi-class Classification Model
- Mini-batch Stochastic Gradient Descent
- Multiclass Classification: Worked Example
- **Making Predictions (Inference)**
- Multiclass Classification: Evaluation Metrics

4 IMPLEMENTATION TIPS AND DEBUGGING

5 SUMMARY

INFERENCE: MAKING PREDICTIONS

After training, use the model to classify new examples.

Inference process for input $\mathbf{x}_{\text{new}}$:

Compute logits: $\mathbf{z} = \mathbf{W}^T \mathbf{x}_{\text{new}}$ (28)

Apply softmax: $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$ (29)

Predict class: $\hat{k} = \arg \max_{k=1, \dots, K} \hat{y}_k$ (30)

Optional: Report confidence scores (the probabilities $\hat{\mathbf{y}}$)

INFERENCE EXAMPLE

Trained model with $K = 3$ classes, new input $\mathbf{x} = [1, 2.5, 1.8]^T$

Step 1: Compute logits $\mathbf{z} = \mathbf{W}^T \mathbf{x} = [1.2, 2.5, 0.8]^T$

Step 2: Apply softmax $e^{1.2} \approx 3.32, \quad e^{2.5} \approx 12.18, \quad e^{0.8} \approx 2.23$

$$\text{Sum} = 3.32 + 12.18 + 2.23 = 17.73$$

$$\hat{\mathbf{y}} = \begin{bmatrix} 3.32/17.73 \\ 12.18/17.73 \\ 2.23/17.73 \end{bmatrix} = \begin{bmatrix} 0.187 \\ 0.687 \\ 0.126 \end{bmatrix}$$

Step 3: Predict class $\hat{k} = \arg \max_k \hat{y}_k = 2$ (Class 2 with 68.7% confidence)

TABLE OF CONTENTS

1 CLASSIFICATION

2 BINARY CLASSIFICATION

- Logistic Regression
- The Model: Single Neuron for Binary Classification
- Binary Classification: Worked Example
- Binary Classification: Evaluation Metrics

3 MULTI-CLASS CLASSIFICATION

- Multi-class Classification Model
- Mini-batch Stochastic Gradient Descent
- Multiclass Classification: Worked Example
- Making Predictions (Inference)
- **Multiclass Classification: Evaluation Metrics**

4 IMPLEMENTATION TIPS AND DEBUGGING

5 SUMMARY

EVALUATION METRICS FOR MULTI-CLASS

1. Accuracy: Overall correctness

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{N} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\hat{y}^{(i)} = y^{(i)}] \quad (31)$$

2. Confusion Matrix: $K \times K$ matrix

- Row i , Column j : Number of class i examples predicted as class j
- Diagonal: Correct predictions
- Off-diagonal: Misclassifications

3. Per-class Precision and Recall:

- Compute for each class separately (one-vs-all)
- Average across classes (macro-average or weighted average)

CONFUSION MATRIX FOR MULTI-CLASS

Example: 3-class confusion matrix

		Predicted		
		Class 1	Class 2	Class 3
Actual	Class 1	45	3	2
	Class 2	5	38	7
	Class 3	2	4	44

Interpretation:

- Diagonal (green): Correct predictions (45, 38, 44)
- Off-diagonal (red): Misclassifications
- Row sums: Total actual examples per class (50, 50, 50)
- Column sums: Total predicted examples per class

Accuracy: $(45 + 38 + 44)/150 = 0.847$ or 84.7%

PER-CLASS METRICS

For each class k , compute:

Precision for class k : Of all predictions for class k , what fraction were correct?

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k} \quad (32)$$

Recall for class k : Of all actual class k examples, what fraction did we identify?

$$\text{Recall}_k = \frac{TP_k}{TP_k + FN_k} \quad (33)$$

F1 Score for class k :

$$F1_k = 2 \cdot \frac{\text{Precision}_k \times \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k} \quad (34)$$

AVERAGING STRATEGIES

How to combine per-class metrics into overall score?

1. **Macro-average**: Simple average across classes

$$\text{Precision}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \text{Precision}_k \quad (35)$$

- Treats all classes equally
- Good when classes are equally important

2. **Weighted average**: Weight by class frequency

$$\text{Precision}_{\text{weighted}} = \sum_{k=1}^K w_k \cdot \text{Precision}_k \quad w_k = N_k / N \quad (36)$$

- Accounts for class imbalance
- Good when some classes are more common

TOP-K ACCURACY

For problems with many classes (e.g., ImageNet with 1000 classes):

Top-K Accuracy: Correct if true class is in top K predictions

$$\text{Top-K Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[y^{(i)} \in \text{Top-K}(\hat{\mathbf{y}}^{(i)})] \quad (37)$$

Example: Image classification with 1000 classes

- Top-1 accuracy: Is the highest probability class correct?
- Top-5 accuracy: Is the true class in the top 5 predictions?

Why useful?

- More forgiving metric for large K
- Captures whether model has correct class among top candidates
- Standard in ImageNet challenge

SUMMARY: MULTICLASS CLASSIFICATION

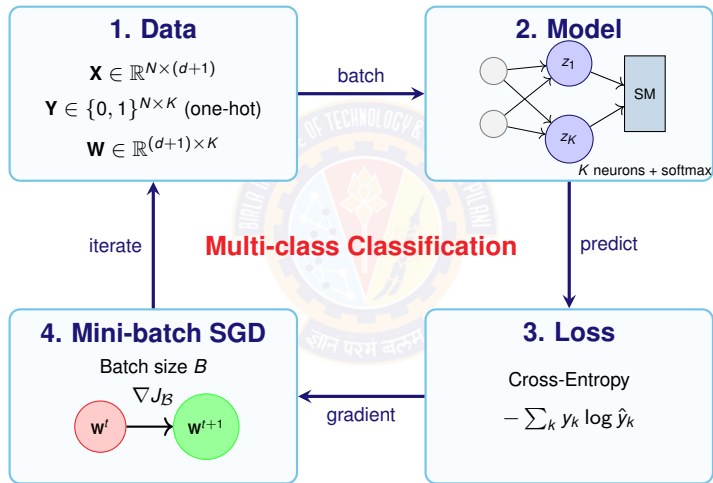


TABLE OF CONTENTS

1 CLASSIFICATION

2 BINARY CLASSIFICATION

- Logistic Regression
- The Model: Single Neuron for Binary Classification
- Binary Classification: Worked Example
- Binary Classification: Evaluation Metrics

3 MULTI-CLASS CLASSIFICATION

- Multi-class Classification Model
- Mini-batch Stochastic Gradient Descent
- Multiclass Classification: Worked Example
- Making Predictions (Inference)
- Multiclass Classification: Evaluation Metrics

4 IMPLEMENTATION TIPS AND DEBUGGING

5 SUMMARY

IMPLEMENTATION TIPS

- 1. Learning Rate & Epochs:

- ▶ Start: $\eta \in \{0.01, 0.1, 1.0\}$
- ▶ Typical: 10-100 epochs
- ▶ Use early stopping

- 2. Weight Initialization:

- ▶ $w_j \sim \mathcal{N}(0, 0.01)$
- ▶ Avoid zeros or large values

- 3. Batch Size:

- ▶ Start: $B \in \{64, 128, 256\}$
- ▶ Powers of 2 preferred
- ▶ Larger: stable, slower
- ▶ Smaller: noisy, faster

- 4. Data Handling:

- ▶ **Always shuffle** each epoch
- ▶ Check class balance

- 5. Feature Scaling: **Essential!**

- ▶ Standardization: $x'_j = \frac{x_j - \mu_j}{\sigma_j}$
- ▶ Prevents sigmoid saturation

- 6. Numerical Stability:

- ▶ Use: $\log(\sigma(z))$
- ▶ Stable softmax: $c = \max_k z_k$
- ▶ $\text{SM}(z)_k = \frac{e^{z_k - c}}{\sum_j e^{z_j - c}}$
- ▶ Log-sum-exp trick
- ▶ Prevents overflow/underflow



DEBUGGING CHECKLIST

- Loss is NaN/Inf:
 - ▶ η too large $\rightarrow$ decrease
 - ▶ Check $\log(0)$ in loss
 - ▶ Scale features
 - ▶ Use stable softmax
- Predicts same class:
 - ▶ Check class balance
 - ▶ Verify softmax/loss
 - ▶ η might be too small
 - ▶ Verify one-hot encoding
- Loss oscillates:
 - ▶ η too large
 - ▶ Use mini-batches
 - ▶ Apply learning rate decay
- Softmax outputs $\neq 1$:
 - ▶ Implementation bug
 - ▶ Numerical precision
 - ▶ Use stable softmax
- Loss not decreasing:
 - ▶ Check gradient signs
 - ▶ η too small/large
 - ▶ Verify feature scaling
 - ▶ Verify data shuffling
 - ▶ Check one-hot encoding
- High train, low test accuracy:
 - ▶ Overfitting
 - ▶ Add regularization
 - ▶ Get more training data
 - ▶ Simpler model
- Sanity checks:
 - ▶ Tiny dataset $\rightarrow$ 100% accuracy
 - ▶ Verify $\hat{y} \in [0, 1]$
 - ▶ Verify $\sum_k \hat{y}_k = 1$
 - ▶ Shuffling improves results
 - ▶ Print confusion matrix

TABLE OF CONTENTS

1 CLASSIFICATION

2 BINARY CLASSIFICATION

- Logistic Regression
- The Model: Single Neuron for Binary Classification
- Binary Classification: Worked Example
- Binary Classification: Evaluation Metrics

3 MULTI-CLASS CLASSIFICATION

- Multi-class Classification Model
- Mini-batch Stochastic Gradient Descent
- Multiclass Classification: Worked Example
- Making Predictions (Inference)
- Multiclass Classification: Evaluation Metrics

4 IMPLEMENTATION TIPS AND DEBUGGING

5 SUMMARY

ACTIVATION FUNCTIONS

Property	Identity	Sigmoid	Softmax
Formula	$f(z) = z$	$\sigma(z) = \frac{1}{1+e^{-z}}$	$\text{SM}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$
Output Range	$(-\infty, \infty)$	$[0, 1]$	$[0, 1]^K, \sum_k = 1$
Use Case	Regression	Binary classification	Multi-class
# Outputs	1	1	K (coupled)

LOSS FUNCTIONS

Aspect	MSE	Binary CE	Categorical CE
Formula	$\frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})^2$	$-\frac{1}{N} \sum_{i=1}^N [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$	$-\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_k^{(i)} \log \hat{y}_k^{(i)}$
Task	Regression	Binary classification	Multi-class classification
Output Type	Continuous value	Probability (2 classes)	Probability distribution (K classes)
Target y	Real number	$\{0, 1\}$	One-hot vector
Prediction $\hat{y}$	Any real number	Probability $[0, 1]$	Probability vector, $\sum_k \hat{y}_k = 1$
Activation	Identity	Sigmoid	Softmax
Penalizes	Distance from target	Confident wrong predictions	Wrong class probability
Properties	Convex, Quadratic penalty	Convex, Probabilistic, Logarithmic penalty	Convex, Extends binary CE, Multi-class penalty

GRADIENT DESCENT VARIANTS

Aspect	Batch GD	Stochastic GD	Mini-batch GD
Examples/update	All N	1	B (batch size)
Gradient	$\nabla J = \frac{1}{N} \sum_{i=1}^N \nabla \ell_i$	$\nabla \ell_i$ for random i	$\nabla J_B = \frac{1}{B} \sum_{i \in B} \nabla \ell_i$
Update rule	$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J$	$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \ell_i$	$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J_B$
Iterations/epoch	1	N	$\lceil N/B \rceil$
Speed per iteration	Slow	Fast	Medium
Convergence	Smooth, deterministic	Noisy, oscillating	Balanced
Memory	High (all data)	Low (1 example)	Medium (B examples)
Gradient quality	Exact	Noisy estimate	Good estimate
Advantages	Stable, Precise gradient, Reproducible	Fast iterations, Online learning, Escapes local minima	Best balance , GPU efficient, Stable & fast
Disadvantages	Slow for large N , High memory, NO online	Very noisy, Unstable, Hard to parallelize	Extra hyperparameter (B), Needs tuning
Best for	Small datasets	Online learning	Deep learning (std)
Typical B	N (all data)	1	32, 64, 128, 256

COMPARISON OF LINEAR MODELS

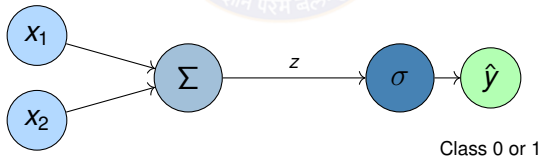
Aspect	Regression	Binary	Multi-class
Output	Continuous $y \in \mathbb{R}$	Discrete $y \in \{0, 1\}$	Discrete $y \in \{1, \dots, K\}$
Activation	Identity: $f(z) = z$	Sigmoid: $\sigma(z)$	Softmax: $\text{SM}(\mathbf{z})$
Loss Function	Mean Squared Error	Binary Cross-Entropy	Categorical Cross-Entropy
Output Range	$(-\infty, \infty)$	$[0, 1]$	$[0, 1]^K, \sum_k = 1$
Interpretation	Predicted value	$P(y = 1 \mathbf{x})$	$P(y = k \mathbf{x})$ for all k
Output neurons	1	1	K
Weights	Vector $\mathbf{w} \in \mathbb{R}^{d+1}$	Vector $\mathbf{w} \in \mathbb{R}^{d+1}$	Matrix $\mathbf{W} \in \mathbb{R}^{(d+1) \times K}$
Evaluation	MSE, RMSE, R^2	Accuracy, Precision, Recall, F1	Confusion matrix, Top-K
Learning	Batch GD or SGD	SGD or Mini-batch	Mini-batch SGD
Examples	House prices, temperature	Spam detection, diagnosis	Digit recognition, ImageNet

RECAP: BINARY CLASSIFICATION

LOGISTIC REGRESSION

Key insight: Sigmoid maps linear combination to probability; optimized by minimizing cross-entropy loss.

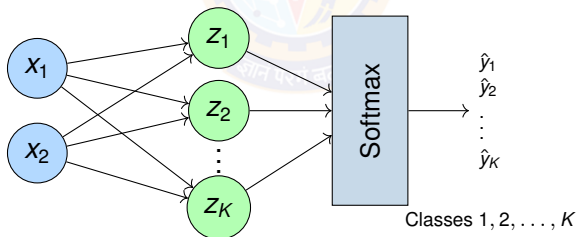
- **Problem:** Classify into two classes: $y \in \{0, 1\}$
- **Model:** Single neuron with sigmoid activation
- **Output:** Probability $\hat{y} = P(y = 1|\mathbf{x}) \in [0, 1]$
- **Loss:** Binary cross-entropy
- **Training:** Stochastic Gradient Descent (SGD)



RECAP: MULTI-CLASS CLASSIFICATION

Key insight: Multiple outputs coupled through softmax create valid probability distribution over all classes.

- **Problem:** Classify into K classes: $y \in \{1, 2, \dots, K\}$
- **Model:** K output neurons with softmax activation
- **Output:** Probability distribution $\hat{\mathbf{y}} = [\hat{y}_1, \dots, \hat{y}_K]^T$, where $\sum_{k=1}^K \hat{y}_k = 1$
- **Loss:** Categorical cross-entropy
- **Training:** Mini-batch Stochastic Gradient Descent



REFERENCES

- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into Deep Learning*. Cambridge University Press. Chapter 4 (Linear Neural Networks for Classification). <https://d2l.ai/>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Chapter 6 (Deep Feedforward Networks). <https://www.deeplearningbook.org/>
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. Chapter 4 (Linear Models for Classification).
- Murphy, K. P. (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press. Chapter 10 (Logistic Regression). <https://probml.github.io/pml-book/>

Thank You!